

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH
CITY**

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

**FACULTY OF COMPUTER SCIENCE AND
ENGINEERING**



REPORT

GRADUATION PROJECT

**DEVELOPMENT OF AN ONLINE
LIBRARY MANAGEMENT SYSTEM**

MAJOR: COMPUTER SCIENCE

COUNCIL: Council 2CC Graduation Project

Advisor 1: Dr. Truong Tuan Anh

Advisor 2: MSc. Vu Ngoc Tu

Committee Secretary: MSc. Mai Duc Trung

—o0o—

Student 1: Nguyen Anh Khoa (2211614)

Student 2: Nguyen Tran Bao Khoa (2211610)

Student 3: Mai Nam Phuong (2252656)

HO CHI MINH CITY, OCTOBER 2026

Acknowledgments

First and foremost, our team would like to express our sincere gratitude to Dr. Truong Tuan Anh – lecturer-in-charge – and MSc. Vu Ngoc Tu – who directly guided our team throughout the graduation project during Semester 253.

Thanks to their dedicated guidance and valuable feedback, our team was able to make timely adjustments, address shortcomings, and complete the content of the project. Under their orientation, our team recognized the practical market demand for an online library management system while gaining a clearer understanding of the strengths and limitations of existing systems to propose appropriate enhancement solutions for our graduation project.

We would also like to express our deepest gratitude to our families, who have always encouraged, motivated, and created the best conditions for us to focus on completing this project. This served as an immense source of motivation helping us persevere throughout the process.

In addition, we would like to thank our friends who accompanied, gave feedback, and shared experiences with us during our work. Your encouragement and support helped us overcome challenging moments when feeling confused about how to execute the next steps of the project.

Finally, we sincerely thank everyone for all their valuable help, encouragement, and guidance, enabling us to successfully complete our graduation project titled “Development of an Online Library Management System”.

Declaration of Authorship

We hereby declare that this project is our own research work, conducted under the supervision and guidance of Dr. Truong Tuan Anh and MSc. Vu Ngoc Tu. The research results presented in this project have not been previously published in any form. All materials used in this project have been collected from various reference sources and systems.

These materials are fully cited in the references section. Apart from the aforementioned sources, all other report contents and implementation results were independently created by our team.

We take full responsibility should any violation arise concerning the statements declared herein.

Ho Chi Minh City, October 2026

Nguyen Anh Khoa

Nguyen Tran Bao Khoa

Mai Nam Phuong

Abstract

The project titled “Development of an Online Library Management System” was conducted to fulfill digital transformation demands in library resource management and utilization. In the context of traditional libraries gradually transitioning to digital environments, constructing a modern, efficient management system capable of providing better support for users is an imperative requirement. The project focuses on researching, designing, and deploying a library system as a Web Application targeted at small-to-medium scale libraries, such as school libraries, faculty libraries, or learning resource centers.

The system is constructed to manage diverse types of library resources, including printed books, journals, and digital assets such as E-books and PDF documents. Furthermore, the system goes beyond supporting basic administrative functions by aiming to enhance user experience through the integration of modern technologies, particularly Artificial Intelligence (AI).

Within the system, the primary user roles consist of readers, librarians, and administrators. Each role possesses distinct functional requirements and usage patterns, which are appropriately supported by the system.

For the reader role, the system provides flexible document search and retrieval functions through keywords or intelligent search tools. Additionally, the system incorporates a document recommendation mechanism based on reading history and user behavior, helping readers quickly access documents tailored to their learning and research needs. Readers can manage their borrowing history, track loan-return statuses, and interact with the system via a chatbot for prompt assistance with queries.

For the librarian role, the system supports managing resource catalogs, updating document metadata, and processing borrowing and returning operations. Furthermore, librarians can monitor resource availability, control data, and ensure efficient library operations. Implementing the system minimizes manual procedures, thereby increasing productivity and reducing management errors.

For the administrator role, the system provides management privileges over all system operations, including user account management, system configuration, and data monitoring. In addition, administrators can review statistics related to borrowing and returning activities to support managerial decision-making and rational library resource development.

A key highlight of the system is the integration of Artificial Intelligence capabilities, such as document recommendations, natural language search, and an interactive assistant chatbot. These features significantly enhance user-system interaction and elevate the overall user experience compared to traditional library systems.

Architecturally, the system is designed following a web application model with key components: user interface, server-side business processing logic, and data storage databases. This organization ensures flexibility, scalability, and maintainability for future upgrades.

Throughout the development of the Online Library Management System, this project aims to optimize resource management workflows, enhance operational efficiency, and deliver an improved user experience in searching for and utilizing academic materials. Simultaneously, the project demonstrates the practical applicability of modern technologies, especially AI, in developing information systems for educational and research environments.

Contents

1	General Topic Introduction	1
1.1	Project Introduction	1
1.1.1	Motivation for Topic Selection	1
1.2	Project Objectives	1
1.2.1	Overview of Proposed System	2
1.3	Project Benefits	2
1.3.1	Building a Modern Online Library Management System	2
1.3.2	Effective Support for Library Management	2
1.3.3	Optimizing Reader Resource Exploitation	2
1.3.4	Enhancing Interaction Between Users and the Library System	2
1.4	Project Scope and Limitations	2
1.4.1	User Groups	2
1.4.2	Scope of Artificial Intelligence Application	3
1.4.3	Project Limitations	3
1.5	General Topic Significance	3
1.5.1	Practical Significance	3
1.5.2	Scientific Significance	3
2	Overview Analysis of Relevant Market Systems	4
2.1	Library Resource Management and Search System – Ho Chi Minh City University of Technology (HCMUT)	4
2.1.1	Strengths	5
2.1.2	Weaknesses	5
2.2	Online Library System – University of London (The Online Library – UoL)	5
2.2.1	Strengths	5
2.2.2	Weaknesses	6
2.3	University of Delaware Library System (UD Library)	6
2.3.1	Strengths	6
2.3.2	Weaknesses	6
2.4	Open Source Library Management System – Koha ILS	6
2.4.1	Strengths	7
2.4.2	Weaknesses	7
3	System Requirements	8
3.1	Functional Requirements	8
3.1.1	User Account Management	8
3.1.2	Resource Catalog Management	8
3.1.3	Circulation Management	8
3.1.4	AI-Powered Document Recommendation Engine	8
3.1.5	AI Natural Language Search and Intelligent Interaction	9
3.1.6	AI-Assisted Cataloging and Summarization	9
3.1.7	Notifications and Communication	9
3.1.8	Reporting and Analytics	9
3.1.9	System Administration and Configuration	9
3.2	Non-Functional Requirements	9
4	Detailed Use-Case Specifications	11
4.1	User Account and Access Management	11
4.1.1	Overview	11
4.1.2	Detailed Specifications	11
4.2	Catalog and Metadata Management	14
4.2.1	Overview	14
4.2.2	Detailed Specifications	14
4.3	Document Circulation Management	17
4.3.1	Overview	17
4.3.2	Detailed Specifications	17
4.4	Resource Access and Exploitation	21
4.4.1	Overview	21
4.4.2	Detailed Specifications	21

4.5	Intelligent Interaction	22
4.5.1	Overview	22
4.5.2	Detailed Specifications	22
4.6	Notifications and Communication	25
4.6.1	Overview	25
4.6.2	Detailed Specifications	25
4.7	Reporting and System Administration	28
4.7.1	Overview	28
4.7.2	Detailed Specifications	28
5	System Analysis and Design	31
5.1	System Architecture	31
5.1.1	Presentation Tier	31
5.1.2	Application Tier	31
5.1.3	Data Tier	32
5.2	Database Design	33
5.2.1	Entity Descriptions	33
5.2.2	Enhanced Entity-Relationship Diagram (EERD)	34
5.2.3	Database Schema	35
5.3	Class Diagram	36
5.3.1	Account and Access Management	36
5.3.2	Book Catalog Management	37
5.3.3	Document Circulation Management	38
5.3.4	Resource Search and Exploitation	38
5.3.5	AI Virtual Assistant and Recommendations	39
5.3.6	Notifications and Communication	39
5.3.7	System Administration	39
5.4	Activity Diagrams	41
5.4.1	Account Registration	41
5.4.2	System Login	43
5.4.3	Password Reset	44
5.4.4	View Loan/Return History	45
5.4.5	User Account Management	46
5.4.6	Add / Edit / Delete Catalog Record	47
5.4.7	Batch Catalog Data Ingestion	48
5.4.8	Automated Cataloging via ISBN	49
5.4.9	Physical Material Checkout	50
5.4.10	Material Return	51
5.4.11	Calculate and Assess Overdue Fines	52
5.4.12	Online Loan Renewal	53
5.4.13	Place Item Hold	54
5.4.14	Document Catalog Search	55
5.5	Sequence Diagrams	56
5.5.1	Account Registration	56
5.5.2	Account Login	57
5.5.3	Account Password Reset	58
5.5.4	Document Search	58
5.5.5	Add / Edit / Delete Document	59
5.5.5.a	Add Document	59
5.5.5.b	Edit Document	59
5.5.5.c	Delete Document	60
5.5.6	Batch Document Import	60
5.5.7	Reserve Document	61
5.5.8	Fine Calculation Update	61
5.5.9	Process Document Loan	62
5.5.10	Extend Document Loan	63
5.5.11	Return Document	64

6	Implementation	65
6.1	Technology Stack	65
6.1.1	Front-end: React.js and Vite	65
6.1.2	Back-end: Node.js and Express	65
6.1.3	Dedicated AI Microservice: Python and FastAPI	65
6.1.4	ORM: Prisma	66
6.1.5	Relational Database: PostgreSQL	66
6.1.6	Vector Store and Embeddings: pgvector	66
6.1.7	AI / LLM Capabilities: Embeddings, RAG, Summarization, and Chatbot	66
6.1.8	Hybrid Search Strategy	67
6.1.9	Security and Authentication: JWT, RBAC, and OTP	67
6.1.10	External Integrations: Open Library, Google Books, and Nodemailer	67
6.2	Authentication Module	67
6.2.1	User Interface Implementation	67
6.2.2	Technical Implementation Highlights	69
6.3	Reader Module	69
6.3.1	User Interface Implementation	70
6.3.2	Technical Implementation Highlights	73
6.4	Librarian Module	73
6.4.1	User Interface Implementation	73
6.4.2	Technical Implementation Highlights	77
6.5	Administrator Module	77
6.5.1	User Interface Implementation	77
6.5.2	Technical Implementation Highlights	80
7	System Testing	81
7.1	Test Objectives and Environment	81
7.2	Unit Testing	81
7.3	Integration Testing	82
7.4	System Testing	83
7.5	Code Quality Testing	84
7.6	Test Summary	85
8	AI Features Evaluation	86
8.1	Why Traditional Testing is Insufficient for AI	86
8.2	AI-as-a-Judge Methodology	86
8.3	Chatbot RAG Evaluation	87
8.4	Semantic Search Evaluation	89
8.5	Limitations and Future Work	91
9	Summary and Conclusion	92
9.1	Project Accomplishments and Implementation Summary	92
9.2	Summary of Completed Project Lifecycle	92
10	References	92
11	Appendix	94

List of Figures

1	Main interface of the lib.hcmut.edu.vn library portal and print material catalog	4
2	Institutional Repository Search Interface and Account Login Module on the OPAC portal	4
3	Unified discovery search interface and live support tool at The Online Library - UoL	5
4	DELCAT unified discovery service interface and supporting AI tool at UD Library	6
5	Koha Staff Client administrative interface module	7
6	User Account and Access Management Use-Case Diagram	11
7	Catalog & AI Metadata Management Use-Case Diagram	14
8	Document Circulation Management Use-Case Diagram	17
9	Resource Access & Exploitation Use-Case Diagram	21
10	Intelligent Interaction & AI Use-Case Diagram	22
11	Notifications & Communication Use-Case Diagram	25
12	Reporting & Administration Use-Case Diagram	28
13	Overall System Architecture Diagram of the Online Library Management System	31
14	EERD Diagram of the Online Library Management System	34
15	Relational Database Schema for the Online Library Management System	35
16	Class Diagram for Account and Access Management	36
17	Class Diagram for Book Catalog Management	37
18	Class Diagram for Document Circulation Management	38
19	Class Diagram for Resource Search and Exploitation	38
20	Class Diagram for AI Virtual Assistant and Recommendations	39
21	Class Diagram for Notifications and Communication	39
22	Class Diagram for System Administration	39
23	Activity Diagram for Account Registration	41
24	Activity Diagram for System Login	43
25	Activity Diagram for Password Reset	44
26	Activity Diagram for View Loan/Return History	45
27	Activity Diagram for User Account Management	46
28	Activity Diagram for Add / Edit / Delete Catalog Record	47
29	Activity Diagram for Batch Catalog Data Ingestion	48
30	Activity Diagram for Automated Cataloging via ISBN	49
31	Activity Diagram for Physical Material Checkout	50
32	Activity Diagram for Material Return	51
33	Activity Diagram for Calculate and Assess Overdue Fines	52
34	Activity Diagram for Online Loan Renewal	53
35	Activity Diagram for Place Item Hold	54
36	Activity Diagram for Document Catalog Search	55
37	Sequence Diagram for Account Registration	56
38	Sequence Diagram for Account Login	57
39	Sequence Diagram for Account Password Reset	58
40	Sequence Diagram for Document Search	58
41	Sequence Diagram for Add Document	59
42	Sequence Diagram for Edit Document	59
43	Sequence Diagram for Delete Document	60
44	Sequence Diagram for Batch Document Import	60
45	Sequence Diagram for Reserve Document	61
46	Sequence Diagram for Fine Calculation Update	61
47	Sequence Diagram for Process Document Loan	62
48	Sequence Diagram for Extend Document Loan	63
49	Sequence Diagram for Return Document	64
50	BkLib Login Interface	68
51	BkLib Account Registration Interface	68
52	Three-step Modal Workflow for Password Recovery	69
53	Reader Module Homepage Interface	70
54	Catalog Search Interface	70
55	Active Loans Management Interface	71
56	Loan History Interface	71
57	Active Holds Interface	72
58	Virtual Assistant Chat Window Interface	72

59	Personal Profile and Settings Interface	73
60	Librarian Module Homepage Interface	74
61	Circulation Management Interface	74
62	Catalog Management Interface	75
63	Communication Management Interface	75
64	Librarian Reports and Analytics Interface	76
65	Librarian Personal Profile and Settings Interface	76
66	Administrator System Overview Dashboard	77
67	User Account Management Interface	78
68	System Configuration Interface	78
69	Branch Management Interface	79
70	System Audit Logs Interface	79
71	Backup and Disaster Recovery Interface	80
72	Unit test results	82
73	Integration test results	83
74	System test results	84
75	Quality test results	85
76	Summary of test results	85

List of Tables

1	Use-case Specification: Account Registration	11
2	Use-case Specification: Password Reset	12
3	Use-case Specification: System Login	12
4	Use-case Specification: Update User Profile	13
5	Use-case Specification: View Loan/Return History	13
6	Use-case Specification: User Account Management	14
7	Use-case Specification: Add / Edit / Delete Catalog Record	15
8	Use-case Specification: Batch Catalog Data Ingestion	16
9	Use-case Specification: Automated Cataloging via ISBN	16
10	Use-case Specification: Automated Document Summarization	17
11	Use-case Specification: Physical Material Checkout	18
12	Use-case Specification: Material Return	19
13	Use-case Specification: Calculate and Assess Overdue Fines	19
14	Use-case Specification: Online Loan Renewal	20
15	Use-case Specification: Place Item Hold	20
16	Use-case Specification: Inter-Branch Loan Request	21
17	Use-case Specification: Document Catalog Search	22
18	Use-case Specification: Rate and Review Materials	22
19	Use-case Specification: Natural Language Search	23
20	Use-case Specification: Chatbot Virtual Assistant Interaction	24
21	Use-case Specification: View Personalized Recommendations	25
22	Use-case Specification: Configure Notification Preferences	26
23	Use-case Specification: View System Notifications	27
24	Use-case Specification: Broadcast Mass Notifications	27
25	Use-case Specification: View Operational Dashboard	28
26	Use-case Specification: Export Periodic Statistical Reports	29
27	Use-case Specification: Configure System Parameters	29
28	Use-case Specification: Manage Branch Libraries	29
29	Use-case Specification: View Audit Logs	30
30	System Entities and Database Definitions	33
31	Scope and Results of Unit Testing	81
32	Scope and Results of Integration Testing	82
33	System Testing Workflows	83
34	Source Code Quality Test Results	84
35	Backend Test Summary	85
36	Deterministic vs. Non-deterministic Testing	86
37	AI Evaluation Metrics Definitions	86
38	Chatbot Evaluation — Group 1: Policy and FAQ Queries	88
39	Sample Ground Truth vs. Chatbot Answer — FAQ Case #2	88
40	Chatbot Evaluation — Group 2: Personal Account Queries (Function Calling)	89
41	Sample Function Calling Interaction — Case #12	89
42	Chatbot Evaluation — Overall Metrics Summary	89
43	Semantic Search Evaluation Test Cases	90
44	Semantic Search Failure Example — Data Sparsity	90
45	Architectural Fix Options for Language Mismatch	91
46	Summary of Completed Project Deliverables	92

1 General Topic Introduction

1.1 Project Introduction

In the context of rapid digital transformation, traditional libraries are gradually transforming into modern online learning resource centers. However, the greatest challenge today extends beyond mere digitization of book titles to managing and exploiting those resources effectively. Existing library management systems often focus primarily on basic administrative tasks, rendering interactions between readers and collections passive and fragmented.

For library managers, manually operating vast resource catalogs—ranging from physical books and journals to digital assets—frequently leads to delays in updating loan statuses and preservation records. The lack of automated analytics tools places heavy pressure on member management and circulation tracking, particularly when forecasting reading demand to optimize collection acquisition.

For readers, searching for materials on legacy systems is often difficult due to rigid keyword matching, lacking flexibility and personalization. Modern users desire a smarter interactive space where they can receive book recommendations tailored to their interests, reading history, and personal behavior without spending excessive time on manual searches. Simultaneously, the demand for immediate assistance via virtual assistants to resolve inquiries regarding rules or document shelf locations is becoming essential.

Recognizing these pressing requirements, the project "Development of an Online Library Management System" was conceived to research, design, and deploy a comprehensive technological platform. This platform not only thoroughly resolves resource management and operational challenges—such as book catalogs, journals, digital assets, and circulation workflows—but also integrates Artificial Intelligence (AI) to enhance collection exploitation. The core focus of the project is creating a smart interactive ecosystem based on a combination of modern administrative models that automate operations and centralize resource management, personalized user experiences through intelligent search tools and automated recommendation systems based on actual reading behavior, and automated support from Chatbots and virtual assistants to deliver immediate query responses and operational guidance to users anytime, anywhere.

By applying software technologies and AI algorithms, the system focuses on optimizing administrative workflows for librarians while providing readers with rapid search capabilities and access to materials tailored to individual needs.

1.1.1 Motivation for Topic Selection

During our academic studies and library usage in a university environment, our team observed multiple limitations in existing library systems. Document searches rely heavily on simple keywords, yielding results that often fail to align with user needs. Furthermore, systems lack the capability to recommend materials based on user preferences or borrowing history, requiring readers to spend significant time locating suitable resources.

Additionally, management activities such as tracking loans/returns, updating item statuses, or providing user support remain heavily dependent on manual operations. This not only reduces operational efficiency but also negatively impacts user experience.

Recognizing these realities, our team chose the topic "Development of an Online Library Management System" with the objective of building a modern system capable of automation and intelligent support, contributing to enhanced management efficiency and improved user satisfaction.

1.2 Project Objectives

The primary objective of this project is to build an online library management system as a Web Application to effectively support both library administrators and readers in exploiting and utilizing academic resources.

The system is developed to provide a centralized management platform for library operations, including catalog management, loan and return lifecycle tracking, and membership administration. Concurrently, the system aims to enhance user experience through intelligent search tools and recommendation mechanisms based on user preferences and reading history.

Furthermore, the project aims to integrate artificial intelligence technologies to assist users during system interactions. Features such as chatbots or virtual assistants are researched and implemented to deliver instant answers to frequently asked questions, system usage guidance, and rapid document retrieval support.

Through constructing this system, the project strives to create a smart digital library platform that optimizes administrative workflows for library managers while delivering convenient, effective resource discovery and access experiences for readers.

1.2.1 Overview of Proposed System

The online library management system is built as a web application, allowing users to access and interact via standard web browsers without requiring additional software installations. Overall, the system comprises core components including the user interface, server-side business logic processing engine, and database storing resource metadata and user profiles.

In addition to basic components, the system integrates artificial intelligence modules to support advanced capabilities such as document recommendations, intelligent search, and chatbot user assistance. These modules operate on data gathered from user behavior and system resource metadata to produce relevant, personalized results.

1.3 Project Benefits

The project delivers tangible benefits to library resource management and exploitation.

1.3.1 Building a Modern Online Library Management System

The project focuses on developing a web-based library management system enabling resource administration and access through an online environment. The system integrates data processing technologies and artificial intelligence to enhance administrative control and support users during document search and exploitation.

1.3.2 Effective Support for Library Management

The system enables management of diverse resource catalogs, including physical books, journals, and digital assets. Features tracking loan-return-preservation statuses, member accounts, and usage analytics empower library staff to operate efficiently. Statistical insights from the system also inform decisions regarding collection acquisition and adjustments.

1.3.3 Optimizing Reader Resource Exploitation

Intelligent search capabilities, automated recommendation engines, and virtual assistant features allow users to quickly locate materials matching their learning and research needs. The system also personalizes user experiences based on reading history and search patterns, boosting resource exploitation efficiency.

1.3.4 Enhancing Interaction Between Users and the Library System

Integrating chatbots or virtual assistants ensures users receive prompt support when interacting with the system. Inquiries regarding borrowing policies, document location, or system operation are answered immediately, enhancing overall user experience during library interactions.

1.4 Project Scope and Limitations

The scope of the project focuses on researching and building an online library management system as a Web Application. Direct system actors comprise: Administrators (Admin), Librarians (Librarian), and Readers (Reader).

1.4.1 User Groups

Reader: Primary system users who search, query, and borrow materials. Readers also receive personalized book recommendations, manage loan histories, and interact with the system via search features or chatbots.

Librarian: Responsible for managing catalog entries, processing loan and return transactions, and updating resource statuses. Librarians also utilize administrative tools for data entry and system information control.

Administrator (Admin): Manages the entire system, including parameter configuration, user account management, and overall system monitoring.

The system manages common library resources such as physical books, printed journals, and digital assets in electronic file formats like E-books or PDF documents. Core functionalities encompass catalog management, member account management, loan-return status tracking, document search support, and personalized recommendation generation.

Within the project scope, the system is designed for small to medium-scale libraries, such as departmental libraries, school libraries, or learning resource centers. Target end-users include students, faculty members, and researchers requiring access to academic materials.

AI features are implemented at a foundational level, detailed in the artificial intelligence application scope section below.

1.4.2 Scope of Artificial Intelligence Application

Within the project scope, artificial intelligence features are deployed at a foundational level to demonstrate integration potential within a library management system. Specifically, the system utilizes existing models and services to implement functionalities such as behavioral recommendation algorithms, natural language search, and Q&A chatbots for common inquiries.

Due to time and resource constraints, the project does not focus on building or training AI models from scratch, but primarily leverages existing tools and APIs to deploy and integrate them into the system. This ensures project feasibility while demonstrating a modern technological approach.

1.4.3 Project Limitations

Although the system is designed with comprehensive features, certain limitations exist. The system is currently developed exclusively as a web application and does not support native mobile platforms. AI capabilities are implemented at a foundational level and have not reached the optimization of large commercial platforms.

Additionally, the system does not handle extreme large-scale scenarios involving millions of records or deep integration with external third-party library networks. Advanced capabilities will be considered for future research and subsequent software releases.

1.5 General Topic Significance

1.5.1 Practical Significance

The online library management system developed in this project enhances operational efficiency for libraries in a digital environment. Centralizing resource and user management reduces manual operations while ensuring data accuracy and real-time synchronization.

Furthermore, the system significantly improves reader experiences through intelligent search and recommendation tools. Users can rapidly discover resources tailored to their academic needs without navigating cumbersome search procedures.

Additionally, integrating chatbots or virtual assistants delivers instant user support, reducing the workload for library staff in addressing routine inquiries.

1.5.2 Scientific Significance

From a research standpoint, the project clarifies methodologies for building and deploying a library management system using modern software technologies. System design encompasses software architecture engineering, database design, and resource/user management implementation.

The project also explores integrating artificial intelligence models into information systems to personalize user experiences. Applying recommendation algorithms and user-assistive chatbots demonstrates AI's potential in elevating knowledge resource utilization.

By combining web development technologies, data management, and artificial intelligence, the project proposes a smart library management system model that can serve as a benchmark for future research and practical implementations.

In subsequent chapters, the report focuses on system requirement analysis, overall architecture design, and core feature implementation to build a complete library management system.

2 Overview Analysis of Relevant Market Systems

2.1 Library Resource Management and Search System – Ho Chi Minh City University of Technology (HCMUT)

The library resource management and search system at Ho Chi Minh City University of Technology (VNU-HCM) currently operates through two independent yet complementary component portals: the Official Library Information Portal (<https://lib.hcmut.edu.vn/>) and the Online Public Access Catalog system (OPAC at <https://opac.lib.hcmut.edu.vn/>). This combination aims to manage and serve two core resource streams in parallel: traditional printed materials and institutional repository assets.

For the main library information portal (lib.hcmut.edu.vn), the system serves as a central hub managing and presenting all print materials, including textbooks, reference books, and academic journals across various disciplines physically stored in campus stacks (such as the Loan Collection in Building B2, Reference Collection in Building B4 at Ly Thuong Kiet Campus, and the Library at Di An Campus). Users on this interface primarily access general announcements, loan rules, or utilize the quick search bar to verify the physical availability of print volumes.

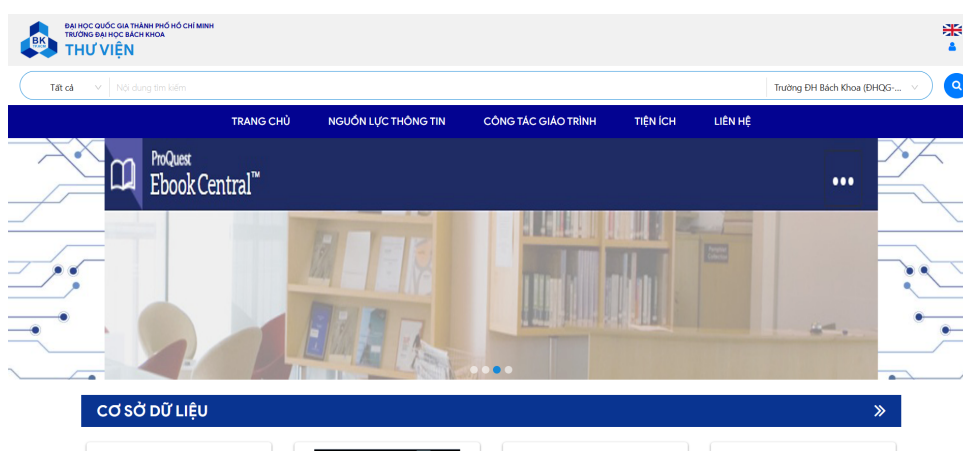


Figure 1: Main interface of the lib.hcmut.edu.vn library portal and print material catalog

Concurrently, to support in-depth academic research, the university operates the OPAC portal (opac.lib.hcmut.edu.vn), serving as a specialized search gateway for institutional repository assets. This portal archives and digitizes intellectual assets produced within the university, including student graduation theses, master's theses, doctoral dissertations, and scientific research projects conducted by faculty members. The OPAC interface offers advanced metadata filtering by Author, Advisor, Defense Year, or Academic Field.

The OPAC system is deeply integrated with the university's Single Sign-On authentication system (SSO/BKPay). Upon successfully logging into the "Personal Account" module, readers can manage loan activities such as viewing borrowed items, checking loan histories, tracking overdue fines, requesting loan renewals, or placing holds on items currently checked out.



Figure 2: Institutional Repository Search Interface and Account Login Module on the OPAC portal

Although both portals satisfy core operational needs, data fragmentation between print materials and digital/institutional repositories creates friction in student user experiences. This limitation motivates our team to

construct a modern library management system that unifies resource data and optimizes operational workflows through modern software solutions.

2.1.1 Strengths

- Specialized interface for institutional repositories optimized for querying academic metadata fields.
- Online loan-return and hold placement workflows fully compatible with practical library policies.
- Single Sign-On (SSO) authentication mechanism synchronizing reader accounts while securing digital resources.

2.1.2 Weaknesses

- Maintaining two independent portals obligates readers to execute repetitive searches across separate domain names.
- Search mechanisms rely strictly on exact keyword matching, showing limited natural language understanding.
- Static platform architecture lacking automated recommendation features and 24/7 AI virtual assistant tools.
- Advanced search interfaces are poorly optimized for mobile devices, exhibiting procedural interaction flows.

2.2 Online Library System – University of London (The Online Library – UoL)

The University of London (UoL) Online Library system at <https://onlinelibrary.london.ac.uk/> represents a flagship digital library (E-library) model designed specifically to support global distance-learning students. Unlike traditional physical collection management models, this system operates almost entirely digitally, focusing on managing, storing, and distributing access rights to academic databases, electronic journals (E-journals), and electronic books (E-books).

A key technical highlight of the system is the deployment of a centralized Discovery Service. Rather than requiring users to log into publisher platforms individually, a single unified search bar (One-stop search) retrieves full-text items across hundreds of database sources. Furthermore, to compensate for the absence of in-person assistance, the system features a live support tool ("Chat with us") functioning as a 24/7 help desk to resolve technical inquiries and guide student literature searches.

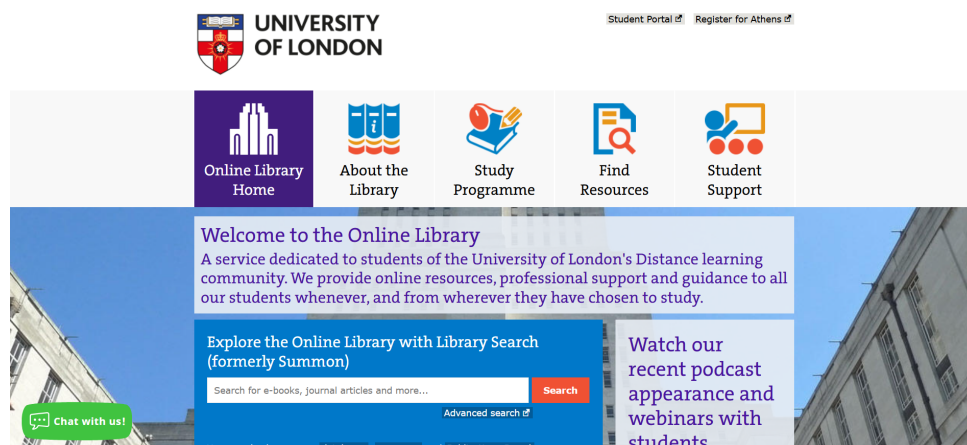


Figure 3: Unified discovery search interface and live support tool at The Online Library - UoL

Analyzing the UoL system provides critical insights into optimizing digital user experience while highlighting technological gaps in intelligent algorithmic user interaction.

2.2.1 Strengths

- Provides a single access gateway for all digital resources, reducing search latency for users.
- Employs modern authentication protocols (OpenAthens), enabling seamless remote off-campus access without requiring a VPN.
- Categorizes digital materials by specific academic disciplines to guide student literature exploration efficiently.

2.2.2 Weaknesses

- Model is restricted strictly to digital assets, entirely omitting physical book circulation and stack management workflows.
- Centralized discovery searching can induce information overload and lacks behavioral analytics for personalized recommendation.
- Chatbot support operates on static pre-scripted rules, failing to interpret complex queries and relying heavily on human librarian intervention.

2.3 University of Delaware Library System (UD Library)

The University of Delaware Library (UD Library) system at <https://library.udel.edu/> exemplifies a highly digitized multi-branch university library governance model. Unlike UoL's pure digital focus, UD Library tackles a more complex challenge: managing and distributing data across central and departmental libraries, while pioneering AI virtual assistant integration in student support workflows.

At the core of this system lies **DELCAT** (Unified Discovery Service), serving as a single interface connecting users with millions of records across diverse formats. Notably, UD Library has begun deploying conversational natural language AI solutions alongside traditional search mechanisms.

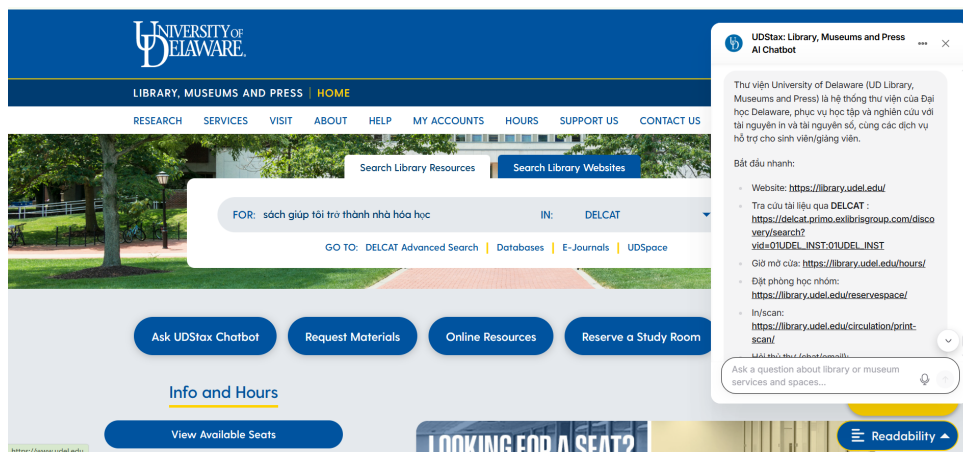


Figure 4: DELCAT unified discovery service interface and supporting AI tool at UD Library

Investigating UD Library offers important architectural perspectives on constructing an "All-in-one" unified discovery platform and operating conversational AI assistants in large academic environments.

2.3.1 Strengths

- Integrates virtual assistants capable of processing natural language to automate routine procedural Q&A.
- DELCAT platform unifies print book and electronic resource discovery within a single user interface.
- Connects non-traditional library services, such as group study room reservations, printing services, and interlibrary loans.

2.3.2 Weaknesses

- Chatbot operates independently as a procedural advisory channel without deep integration into search engines for direct book recommendation.
- Unified discovery architecture frequently causes information overload by returning excessive search result volumes.

2.4 Open Source Library Management System – Koha ILS

Koha ILS is the world's most widely adopted open-source Integrated Library System (ILS), utilized by numerous academic institutions and public library networks. Unlike user-facing OPAC portals or Discovery platforms analyzed previously, Koha represents a backend core system focused on digitizing and automating complex administrative workflows for library staff (Librarians).

Through its Staff Client administration portal, Koha offers comprehensive modules for managing physical and digital libraries, including: Circulation, International Cataloging, Patron Management, and Statistical Reporting. Studying Koha provides valuable reference standards for designing database schemas and business rule logic in our project.

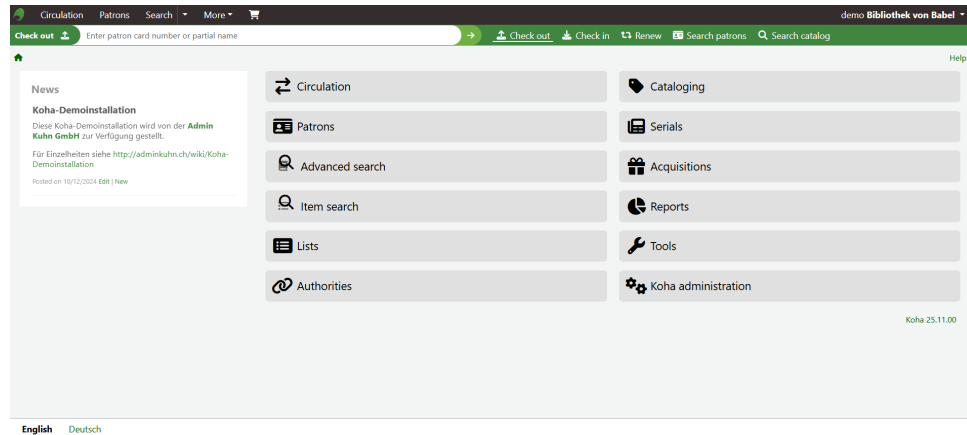


Figure 5: Koha Staff Client administrative interface module

Analyzing Koha helps clearly delineate essential core features required in library management software while exposing user experience friction present in legacy administrative systems.

2.4.1 Strengths

- Strictly complies with international library metadata standards (MARC21, Z39.50), guaranteeing data consistency and interoperability.
- Circulation module provides exhaustive rule matrices for configuring borrowing, returning, renewals, and overdue fine logic tailored to user groups.
- Features built-in REST APIs allowing third-party applications to interact and extract data for platform extension.
- Reporting system supports direct custom SQL queries, delivering maximum flexibility for administrative reporting.

2.4.2 Weaknesses

- Staff Client interface (UI/UX) is verbose, adhering to traditional form-heavy layouts that increase librarian training curves.
- Data cataloging workflows remain heavily manual, lacking intelligent automated classification or metadata extraction mechanisms.
- Monolithic system architecture and complex relational database schemas hamper rapid decoupling or custom modular feature additions.

3 System Requirements

3.1 Functional Requirements

3.1.1 User Account Management

- The system must allow new users (readers) to register accounts by providing full name, email address, student or staff ID, and password.
- The system must authenticate users via username or email and password. Optional Multi-Factor Authentication (MFA) must be supported for administrator accounts.
- The system must support at least three user roles: Reader, Librarian, and Administrator, each endowed with distinct authorization privileges.
- Users must be able to view and update their personal profile information, including contact details and notification preferences.
- The system must enable users to reset passwords via verification links dispatched to their registered email addresses.
- Administrators must be empowered to activate, deactivate, or temporarily suspend any user account.
- The system must log borrowing history for each reader, accessible to the respective reader and authorized librarians.

3.1.2 Resource Catalog Management

- The system must maintain a catalog of all library assets, including physical books and periodicals.
- Each catalog record must store: title, author(s), ISBN/ISSN, publisher, publication year, edition, genre/subject, language, physical location (shelf/classification code), total copy count, and available copy count.
- Librarians must be able to add, edit, and delete catalog records via the administration interface.
- The system must support batch catalog data ingestion via CSV or MARC21 file formats.
- The system must provide full-text search capability across title, author, ISBN, subject, and keyword fields.
- Search results must support filtering by category, language, availability status, publication year range, and media format.
- The system must display real-time copy availability for each item, including available, checked out, and reserved counts.

3.1.3 Circulation Management

- The system must allow readers to check out physical materials by barcode scanning or manual entry at circulation desks, or by placing online pickup requests.
- The system must enforce role-based loan limits (e.g., maximum 5 items for students, maximum 15 items for faculty).
- The system must assign due dates to borrowed items based on configurable loan duration rules per media type.
- The system must allow readers to renew checked-out items online up to a configurable maximum count, provided no hold requests exist for those items.
- The system must allow readers to place holds on items currently checked out by other users.
- The system must automatically notify the next user in a hold queue once a reserved item becomes available.
- The system must process item returns and immediately update copy availability in the catalog.
- The system must automatically calculate overdue fines based on daily configurable rates per material type.
- The system must enable readers to view active loans, due dates, holds, and outstanding fines from their user dashboard.
- The system must support inter-branch borrowing, enabling readers to request materials from other branch libraries in the network.

3.1.4 AI-Powered Document Recommendation Engine

- The system must generate personalized book and resource recommendations for readers based on borrowing history, search logs, and rated items.
- The recommendation engine must combine collaborative filtering (analyzing preferences of similar users) and content-based filtering (analyzing item metadata attributes).

3.1.5 AI Natural Language Search and Intelligent Interaction

- The system must provide a natural language search interface allowing readers to query the catalog using conversational phrasing.
- The natural language search module must analyze user intent, extract attributes (genre, target audience, topic, length), and map them to catalog metadata.
- The system must display relevance-ranked natural language search results alongside standard keyword search outputs.
- The system must integrate a Virtual Assistant Chatbot for direct user interaction, resolving FAQ inquiries regarding library policies, operating hours, and loan procedures.

3.1.6 AI-Assisted Cataloging and Summarization

- Automatically auto-fill book metadata (Title, Author, Description) when librarians input an ISBN.
- AI automatically generates concise summary abstracts for newly cataloged books for reader previews.

3.1.7 Notifications and Communication

- The system must dispatch automated email/SMS notifications for: 3-day pre-due-date reminders, overdue alerts, hold availability notices, and account status updates.
- Readers must be able to configure notification channels and frequencies within profile settings.
- The system must provide an in-app notification center displaying unread system alerts.
- Librarians must be able to broadcast mass announcements (e.g., library closure notices) to all users or target groups.

3.1.8 Reporting and Analytics

- The system must generate analytical reports regarding: top-borrowed items, overdue items, fine collection summaries, new user registrations, and catalog growth metrics.
- The system must feature a real-time dashboard for librarians displaying key operational metrics: active loans, overdue counts, copy availability, and daily transaction volumes.
- Reports must support export to PDF and Excel formats.

3.1.9 System Administration and Configuration

- Administrators must be able to configure global parameters including loan durations, fine rates, role borrowing limits, and hold expiration thresholds.
- The system must support multi-branch library management.
- The system must record full audit logs for administrative actions (configuration changes, account edits, batch ingestion).

3.2 Non-Functional Requirements

To ensure stable, secure operations and a smooth user experience, the system complies with the following non-functional standards:

- **Page Load Time:** Main homepage, document search, and book detail pages must complete loading within under 3 seconds (optimized to 1.5 seconds) on both desktop and mobile platforms.
- **API Response Time:**
 - Basic operational APIs (borrowing cart checks, loan history) must respond within 0.5 seconds.
 - Intelligent processing APIs (AI Virtual Assistant, text summarization, semantic search) must return results within 3 seconds.
- **Load Capacity:** The system must stably handle 1,000 to 2,000 concurrent reader sessions without server degradation.

- **UI/UX Consistency:** Typography, brand color palettes, navigation layouts, and action buttons (Renew, Hold, Process) must remain consistent across all views.
- **Data Security & Privacy:**
 - User account passwords must be hashed using strong algorithms (bcrypt) prior to database storage; plain-text logging is strictly prohibited.
 - Client-server data transmissions must be encrypted over TLS/HTTPS protocols.
- **Application Security:** Source code must enforce strict parameterization to prevent web vulnerabilities including SQL Injection and Cross-Site Scripting (XSS).
- **Uptime:** Server infrastructure must maintain a minimum uptime of 99.5% per year, excluding scheduled maintenance windows.
- **Disaster Recovery:** Database backups are executed daily. In hardware failure events, recovery to the latest backup state must occur within 4 hours.
- **Integrability:** Modular architecture using standard REST APIs facilitates seamless integration with external services (university portals, Google Books API).

4 Detailed Use-Case Specifications

4.1 User Account and Access Management

4.1.1 Overview

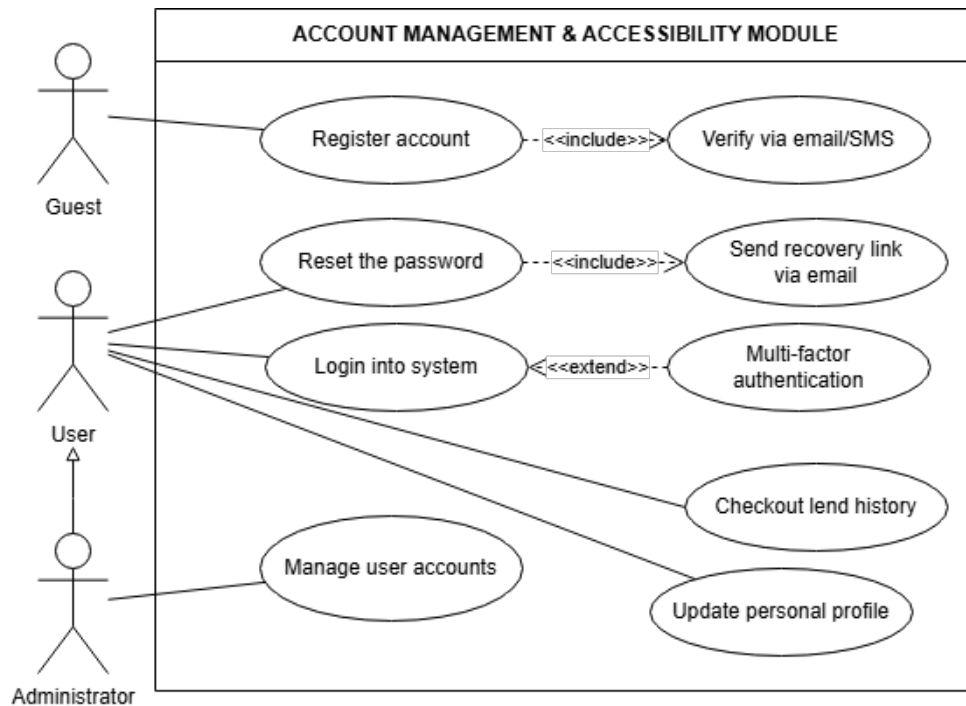


Figure 6: User Account and Access Management Use-Case Diagram

4.1.2 Detailed Specifications

Use-case Name	Account Registration
Use-case ID	UC-ACC-01
Overview	Allows unregistered users to sign up and become system members.
Actor	Guest
Pre-conditions	Guest accesses the registration page of the system.
Trigger Condition	Guest clicks the "Register" button.
Flow of Events	1. System displays the registration form. 2. Guest inputs required fields (Full name, Email, Password, etc.). 3. System validates input data integrity. 4. [Include] System executes "Send Verification Email/SMS" containing an OTP or activation link to the registered contact info. 5. Guest inputs the verification code or clicks the activation link. 6. System verifies the token, creates a new account, and persists it to the database. 7. System displays a success message.
Post-conditions	Account is created successfully and status transitions to "Activated".
Exceptions	Ex1: Email/Phone number already exists → System reports an error and requests re-entry. Ex2: Invalid verification code entered beyond maximum attempts → Cancel registration session.

Table 1: Use-case Specification: Account Registration

Use-case Name	Password Reset
Use-case ID	UC-ACC-02
Overview	Assists users in recovering account access when forgetting login passwords.
Actor	User (All Roles)
Pre-conditions	User possesses an existing system account but cannot recall password.
Trigger Condition	User clicks "Forgot Password" on the login view.
Flow of Events	1. System prompts user for registered email. 2. User inputs email and submits. 3. System verifies email existence in the database. 4. [Include] System triggers "Send Recovery Link via Email" flow. 5. User accesses the reset link received in email. 6. System renders password reset screen. 7. User inputs new password and confirms. 8. System hashes and updates new password, showing success alert.
Post-conditions	User password is updated successfully.
Exceptions	Ex1: Email does not exist → Display "No account found associated with this email". Ex2: Reset link expired → Prompt user to restart the reset process.

Table 2: Use-case Specification: Password Reset

Use-case Name	System Login
Use-case ID	UC-ACC-03
Overview	Authenticates user identity to grant system access matching role privileges.
Actor	User (All Roles)
Pre-conditions	User possesses an active account in the system.
Trigger Condition	User accesses the login view and enters credentials.
Flow of Events	1. User submits identity credentials (Email/Username) and Password. 2. System validates input formats and matches credentials against DB. 3. [Extend] If Multi-Factor Authentication (MFA) is configured, system triggers "MFA Verification", requesting OTP from app/SMS. 4. Initializes user session (Session/JWT Token). 5. Redirects user to the dashboard corresponding to their assigned Role.
Post-conditions	User logs in successfully; system records login timestamp audit.
Exceptions	Ex1: Incorrect credentials → Display "Incorrect account or password". Ex2: Account suspended → Display "Your account is locked. Please contact Admin". Ex3: MFA code failed 3 attempts → Lock login session temporarily for 15 minutes.

Table 3: Use-case Specification: System Login

Use-case Name	Update User Profile
Use-case ID	UC-ACC-04
Overview	Allows users to modify personal profile information on the platform.
Actor	User
Pre-conditions	User is currently logged into the system.
Trigger Condition	User accesses "Profile Settings" and selects "Edit".
Flow of Events	1. System populates form with current profile data. 2. User updates permissible fields (Phone, address, avatar image...). 3. User clicks "Save". 4. System validates field constraints. 5. Updates database record and displays success notification.
Post-conditions	User profile is updated.
Exceptions	Ex1: Invalid avatar format or file size exceeded → Warning alert and file rejection.

Table 4: Use-case Specification: Update User Profile

Use-case Name	View Loan/Return History
Use-case ID	UC-ACC-05
Overview	Enables users to review past and active item borrowings along with their statuses.
Actor	User
Pre-conditions	User is logged into the system.
Trigger Condition	User selects "Borrowing History" in personal dashboard.
Flow of Events	1. System receives user request. 2. Queries database for circulation records tied to user ID. 3. Renders loan list (Item title, borrow date, due date, return date, status, fines). 4. User can filter results by date range or status.
Post-conditions	System state remains unchanged.
Exceptions	Ex1: No loan records found → Display "You have no borrowing history".

Table 5: Use-case Specification: View Loan/Return History

Use-case Name	User Account Management
Use-case ID	UC-ACC-06
Overview	Empowers administrators to search, view, authorize, activate, or suspend user accounts.
Actor	Administrator (Admin)
Pre-conditions	Administrator is logged in with administrative access privileges.
Trigger Condition	Admin navigates to "User Management" menu.
Flow of Events	1. System displays table of all user accounts. 2. Admin filters or searches accounts by role or account status. 3. Admin selects a specific user to perform actions (Lock/Unlock, Reset Password, Change Role). 4. Admin confirms the administrative operation. 5. System persists changes to DB and writes audit log. 6. System displays success confirmation.
Post-conditions	Target account status or permission set is updated.
Exceptions	Ex1: Admin attempts to lock their own account → System rejects action and raises logic violation error.

Table 6: Use-case Specification: User Account Management

4.2 Catalog and Metadata Management

4.2.1 Overview

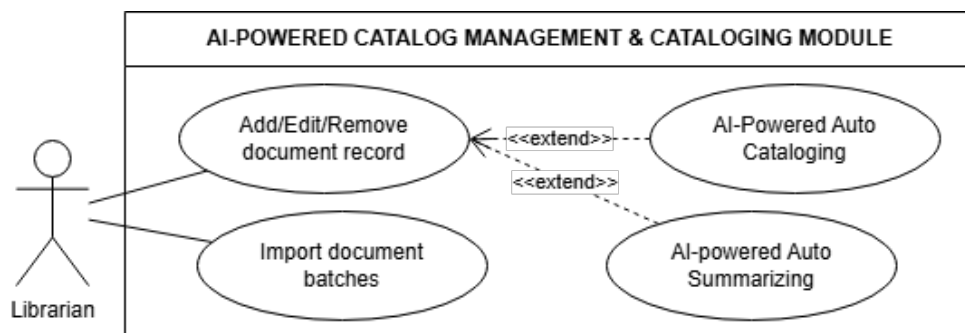


Figure 7: Catalog & AI Metadata Management Use-Case Diagram

4.2.2 Detailed Specifications

Use-case Name	Add / Edit / Delete Catalog Record
Use-case ID	UC-CAT-01
Overview	Librarians perform catalog management operations (CRUD), updating metadata or removing lost/damaged copies.
Actor	Librarian
Pre-conditions	Librarian is logged into the system.
Trigger Condition	Librarian navigates to "Catalog Management".
Flow of Events	1. System displays list of current catalog items. 2. Librarian selects action: Add New, Edit, or Delete. 3. If Delete: Librarian confirms deletion, system checks active loan constraints and executes Soft Delete. 4. If Add/Edit: System renders item form (Title, Author, ISBN, Category, Copies...). ** Extension Points: ** - [Extend] Librarian can execute "Automated Cataloging via ISBN" (UC-CAT-03) to auto-fill form. - [Extend] Librarian can execute "Automated Document Summarization" (UC-CAT-04) if description is missing. 5. Librarian reviews form fields and clicks "Save". 6. System validates fields, persists to DB, and displays success alert.

Post-conditions	Catalog record is updated successfully.
Exceptions	Ex1: Required fields missing → System highlights missing inputs and requests completion. Ex2: Duplicate ISBN code → Display "ISBN already exists in catalog".

Table 7: *Use-case Specification: Add / Edit / Delete Catalog Record*

Use-case Name	Batch Catalog Data Ingestion
Use-case ID	UC-CAT-02
Overview	Allows librarians to import large volumes of catalog records via standard data files (CSV, Excel).
Actor	Librarian
Pre-conditions	Librarian is logged into the system.
Trigger Condition	Librarian clicks "Batch Import" in catalog management.
Flow of Events	1. System presents upload interface and sample template file. 2. Librarian uploads catalog dataset file. 3. System parses file and displays preview table distinguishing valid vs invalid records. 4. Librarian confirms import operation. 5. System writes valid rows to DB and generates execution report (success vs error row counts).
Post-conditions	Multiple catalog records added to system.
Exceptions	Ex1: Unsupported file format → System rejects upload: "Unsupported file format". Ex2: File contains duplicate ISBNs → System skips duplicate lines, logs errors, and imports remaining valid records.

Table 8: Use-case Specification: Batch Catalog Data Ingestion

Use-case Name	Automated Cataloging via ISBN
Use-case ID	UC-CAT-03
Overview	AI-assisted feature to automatically retrieve and populate book metadata (Title, Author, Publisher, Genre) from external APIs using ISBN lookup.
Actor	Librarian
Pre-conditions	Currently executing flow of "Add / Edit / Delete Catalog Record".
Trigger Condition	Librarian enters ISBN and clicks "Auto-Fetch Metadata".
Flow of Events	1. Librarian enters ISBN into corresponding input field. 2. System dispatches API request to open catalog endpoints (Google Books, Open Library). 3. System receives JSON response, parsing entity attributes. 4. System auto-populates form fields with parsed attributes. 5. Librarian reviews, edits if needed, and proceeds with base cataloging flow.
Post-conditions	Catalog form is auto-filled, reducing manual entry time for librarians.
Exceptions	Ex1: ISBN not found in external databases → System alerts "No metadata found, please enter manually". Ex2: Third-party API timeout → System alerts timeout error and asks librarian to retry.

Table 9: Use-case Specification: Automated Cataloging via ISBN

Use-case Name	Automated Document Summarization
Use-case ID	UC-CAT-04
Overview	Uses Large Language Models (LLMs) to automatically generate concise abstract summaries (150-300 words) for catalog items lacking publisher descriptions.
Actor	Librarian
Pre-conditions	Executing "Add / Edit / Delete Catalog Record" flow. Title and Author fields populated.
Trigger Condition	Librarian clicks "Generate AI Summary" in Description field.
Flow of Events	1. System collects existing item fields (Title, Author, Category). 2. System formats prompt payload and sends to AI Engine (e.g., Gemini API). 3. AI Engine processes payload and returns summary text. 4. System populates text into Description field, automatically tagging "(AI-generated summary)". 5. Librarian reviews, edits, or requests regeneration.
Post-conditions	Item receives concise abstract summary, aiding reader discovery.
Exceptions	Ex1: Insufficient metadata input causing inaccurate AI synthesis → Generic output returned; librarian discards and clears output.

Table 10: Use-case Specification: Automated Document Summarization

4.3 Document Circulation Management

4.3.1 Overview

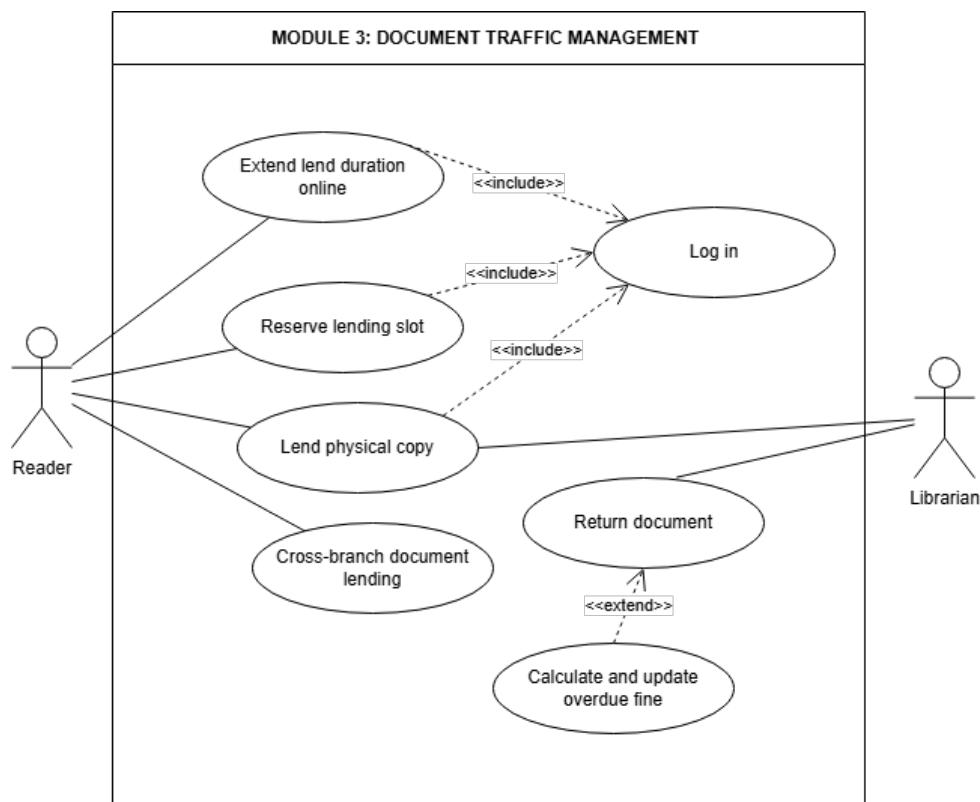


Figure 8: Document Circulation Management Use-Case Diagram

4.3.2 Detailed Specifications

Use-case Name	Physical Material Checkout
Use-case ID	UC-CIR-01
Overview	Allows readers to borrow physical materials available at the library counter assisted by a librarian.
Actor	Reader, Librarian

Pre-conditions	Reader and Librarian possess valid active accounts. Reader has no card suspension or unpaid fines exceeding threshold.
Trigger Condition	Reader presents items and membership card at circulation desk.
Flow of Events	1. [Include] Librarian logs into circulation module. 2. Librarian scans Reader membership barcode/ID. 3. System loads Reader profile, checking borrowing eligibility and fine status. 4. Librarian scans barcode on each checkout item. 5. System checks active loan limits against role allowance. 6. System creates loan record, calculating due dates based on item policy rules. 7. System updates copy status in inventory to "Checked Out" and decrements available count. 8. Librarian hands items to Reader; transaction completes.
Post-conditions	Loan record created in system; transaction logged.
Exceptions	Ex1: Reader reached maximum borrowing limit → Error alert "Loan limit exceeded"; reader must return existing items first. Ex2: Item scanned is on hold for another patron → Warning alert; librarian retains item for hold queue.

Table 11: *Use-case Specification: Physical Material Checkout*

Use-case Name	Material Return
Use-case ID	UC-CIR-02
Overview	Processes returned physical materials, closes active loan records, and updates copy availability in inventory.
Actor	Librarian
Pre-conditions	Items are associated with active open loan records.
Trigger Condition	Reader returns items at circulation desk.
Flow of Events	1. [Include] Librarian logs into circulation module. 2. Librarian scans item barcode. 3. System retrieves associated active loan record. 4. System compares return date against scheduled due date. ** Extension Points: ** - [Extend] If return date exceeds due date, system triggers "Calculate and Assess Overdue Fines" (UC-CIR-03). 5. Updates loan status to "Completed". 6. System verifies hold queue status for returned item: - If hold exists: Set status to "Awaiting Pickup" and send notification to next patron. - If no hold: Set status to "Available" and increment copy inventory count.
Post-conditions	Loan record closed. Item marked available for next transaction.
Exceptions	Ex1: Barcode damaged/unreadable → Librarian performs manual lookup by title and borrower ID.

Table 12: Use-case Specification: Material Return

Use-case Name	Calculate and Assess Overdue Fines
Use-case ID	UC-CIR-03
Overview	Automatically computes fine amounts when items are returned past due dates.
Actor	Librarian
Pre-conditions	Triggered from "Material Return" when system detects overdue return.
Trigger Condition	Return Date $\geq$ Due Date in loan record.
Flow of Events	1. System computes overdue days: (Actual Return Date - Due Date). 2. Fetches daily fine rate applicable to material type (configured by Admin). 3. Calculates total fine amount (Overdue Days $\times$ Daily Rate). 4. Displays fine calculation on librarian view. 5. Librarian notifies reader. If reader pays immediately, receipt is generated; otherwise fine is posted to reader ledger.
Post-conditions	Fine record updated on reader profile or marked as paid.
Exceptions	Ex1: Official holidays fall within overdue range → System automatically excludes official library holidays from overdue day count.

Table 13: Use-case Specification: Calculate and Assess Overdue Fines

Use-case Name	Online Loan Renewal
Use-case ID	UC-CIR-04
Overview	Allows readers to extend loan periods online without physically visiting the library.
Actor	Reader
Pre-conditions	Item is not overdue and renewal limit has not been reached.
Trigger Condition	Reader clicks "Renew" in active loans dashboard.
Flow of Events	1. [Include] Reader logs into system portal. 2. System verifies renewal eligibility: - Has maximum renewal limit been reached for item? - Does an active hold request exist from another user? 3. If eligible, system extends due date by configured extension period. 4. System updates loan record and increments renewal counter. 5. Displays "Renewal Successful" along with new due date.
Post-conditions	Item due date is extended.
Exceptions	Ex1: Item has active hold request → System rejects renewal: "Item requested by another user, please return on time".

Table 14: Use-case Specification: Online Loan Renewal

Use-case Name	Place Item Hold / Reservation
Use-case ID	UC-CIR-05
Overview	Enables readers to join a reservation queue for items currently out of stock.
Actor	Reader
Pre-conditions	Target item status is "All Copies Checked Out".
Trigger Condition	Reader clicks "Place Hold" on item detail view.
Flow of Events	1. [Include] Reader logs into system portal. 2. System verifies Reader does not currently hold an active loan for same title. 3. System appends Reader ID to item reservation queue. 4. Interface displays "Hold Placed Successfully" showing queue position. 5. Upon item return, system holds copy and alerts queue position #1 patron.
Post-conditions	Reservation saved; renewal blocked for current borrower.
Exceptions	Ex1: Reader has unpaid fines or card suspension → System rejects hold request.

Table 15: Use-case Specification: Place Item Hold

Use-case Name	Inter-Branch Loan Request
Use-case ID	UC-CIR-06
Overview	Allows readers to request item transfers from another branch library to their local campus library for pickup.
Actor	Reader
Pre-conditions	Multi-branch network enabled. Item unavailable locally but available at another branch.
Trigger Condition	Reader clicks "Request Inter-Branch Transfer" on book detail page.
Flow of Events	1. [Include] Reader logs into system portal. 2. System prompts Reader to select preferred pickup branch. 3. Reader confirms request. 4. System creates inter-branch transfer ticket sent to holding branch. 5. Holding librarian packages item; status changes to "In Transit". 6. Upon arrival at destination branch, automated notification alerts Reader for pickup.
Post-conditions	Physical transfer logged and inventory updated across branches.
Exceptions	Ex1: Last available copy borrowed before request processing → Cancel request and suggest standard hold queue.

Table 16: Use-case Specification: Inter-Branch Loan Request

4.4 Resource Access and Exploitation

4.4.1 Overview

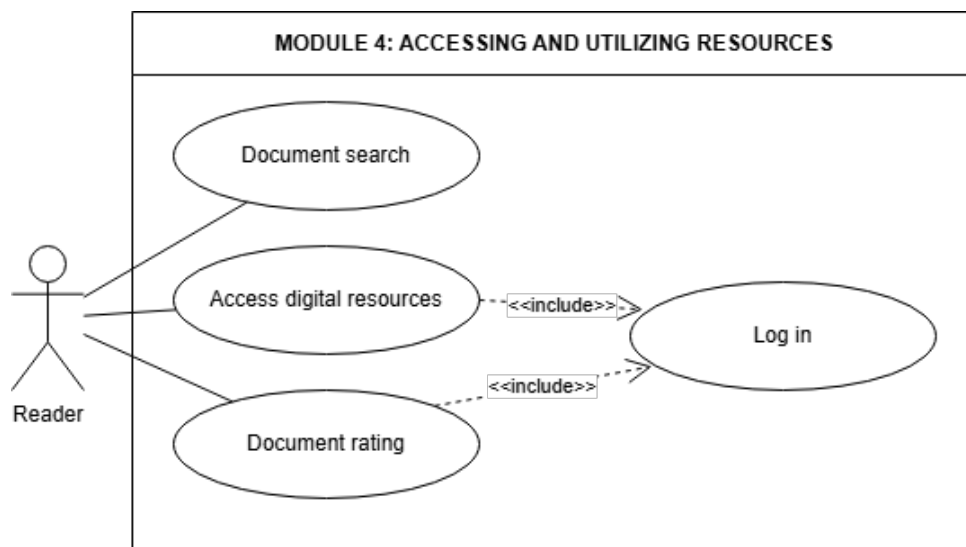


Figure 9: Resource Access & Exploitation Use-Case Diagram

4.4.2 Detailed Specifications

Use-case Name	Document Catalog Search
Use-case ID	UC-EXP-01
Overview	Allows readers to search library catalog records via keywords (Title, Author, Genre, ISBN).
Actor	Reader (Including unauthenticated guests)
Pre-conditions	System online with active DB connection.
Trigger Condition	Reader inputs query string and submits search.
Flow of Events	1. Reader selects search filter scope (All, Title, Author...) and enters query. 2. System queries catalog database indexes. 3. System returns paginated match results. 4. Renders item cards (Cover image, Title, Author, Year, Availability Status). 5. Reader clicks item card to open detail view.

Post-conditions	System state remains unchanged.
Exceptions	Ex1: No items match query → Display "No results found" with spelling/keyword suggestions.

Table 17: Use-case Specification: Document Catalog Search

Use-case Name	Rate and Review Materials
Use-case ID	UC-EXP-03
Overview	Allows readers to leave 1-5 star ratings and written reviews for items they have read.
Actor	Reader
Pre-conditions	Reader has borrowed the physical item (preventing spam).
Trigger Condition	Reader clicks "Write a Review" on book page.
Flow of Events	1. [Include] System verifies login status. 2. System verifies loan history against target item. 3. Renders rating selector (1-5 stars) and review text area. 4. Reader submits rating and optional review. 5. System checks content moderation filters. 6. Saves rating, updates item average rating score. 7. Displays success alert and refreshes view.
Post-conditions	Review saved; average item score updated.
Exceptions	Ex1: Reader has not borrowed item → Hide review button or display "You must borrow this item before submitting a review". Ex2: Review contains profanity → Block submission with moderation warning.

Table 18: Use-case Specification: Rate and Review Materials

4.5 Intelligent Interaction

4.5.1 Overview

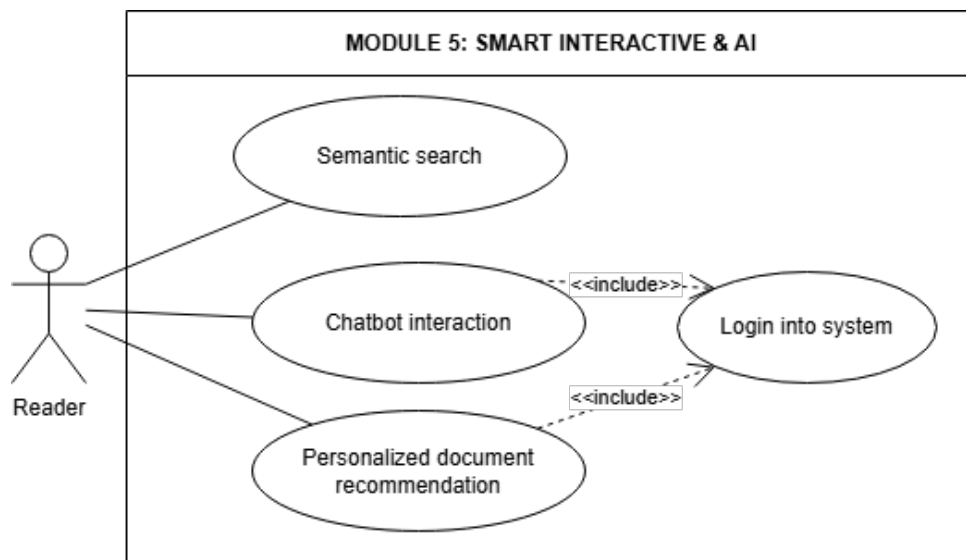


Figure 10: Intelligent Interaction & AI Use-Case Diagram

4.5.2 Detailed Specifications

Use-case Name	Natural Language Search
Use-case ID	UC-AI-01
Overview	Allows readers to search catalog items using conversational phrases (e.g., "introductory artificial intelligence textbooks") rather than exact keywords.
Actor	Reader

Pre-conditions	System AI NLP module operational.
Trigger Condition	Reader inputs phrase into Smart Search bar and submits.
Flow of Events	1. Reader enters natural language query. 2. NLP module processes query via Intent Analysis & Entity Extraction (genre, topic, target level). 3. System converts entities into vector embeddings or metadata filter trees. 4. Executes vector similarity search (pgvector) and semantic ranking. 5. Renders ranked search results sorted by semantic relevance scores.
Post-conditions	Semantic search results rendered with related topic tags.
Exceptions	Ex1: Irrelevant or meaningless phrase → AI assistant prompts "No matching documents found. Try phrases like..." Ex2: AI service offline → System falls back to full-text keyword search and alerts "Smart search temporarily under maintenance".

Table 19: *Use-case Specification: Natural Language Search*

Use-case Name	Chatbot Virtual Assistant Interaction
Use-case ID	UC-AI-02
Overview	Readers interact with an AI Chatbot for policy inquiries, opening hours, or checking personal account loan status.
Actor	Reader
Pre-conditions	AI Chatbot service online.
Trigger Condition	Reader opens Chatbot widget window.
Flow of Events	1. [Include] System prompts login if unauthenticated for personalized responses. 2. Chatbot interface displays welcome message and quick action buttons (FAQ, My Loans, Book Suggestions). 3. Reader types message or selects quick prompt. 4. Chatbot (LLM) analyzes context: - General FAQ: Retrieves answer from RAG knowledge base. - Account query (e.g., "What books do I owe?"): Chatbot queries backend API using Reader ID and synthesizes response. 5. Renders conversational response in chat widget.
Post-conditions	Chat transcript saved for audit logging and AI refinement.
Exceptions	Ex1: Query out of scope → Response: "I am a library assistant. Please contact staff at hotline for special inquiries."

Table 20: *Use-case Specification: Chatbot Virtual Assistant Interaction*

Use-case Name	View Personalized Recommendations
Use-case ID	UC-AI-03
Overview	Provides readers with tailored book recommendations generated from loan history, search logs, and collaborative user behavior.
Actor	Reader
Pre-conditions	AI Recommendation engine online.
Trigger Condition	Reader opens homepage dashboard or "Recommended For You" view.
Flow of Events	1. [Include] Reader logs into system. 2. System aggregates user signals: loan history, recent searches, ratings. 3. System sends telemetry to Recommendation Engine. 4. Engine executes Collaborative Filtering and Content-Based Filtering models. 5. Filters out items currently checked out or held by user. 6. Displays "Recommended For You" grid with brief rationale tags (e.g., "Because you read Title X...").
Post-conditions	Interface presents personalized recommendations.
Exceptions	Ex1: New account with no history (Cold Start) → Display "Trending Titles" or "New Arrivals" fallback.

Table 21: Use-case Specification: View Personalized Recommendations

4.6 Notifications and Communication

4.6.1 Overview

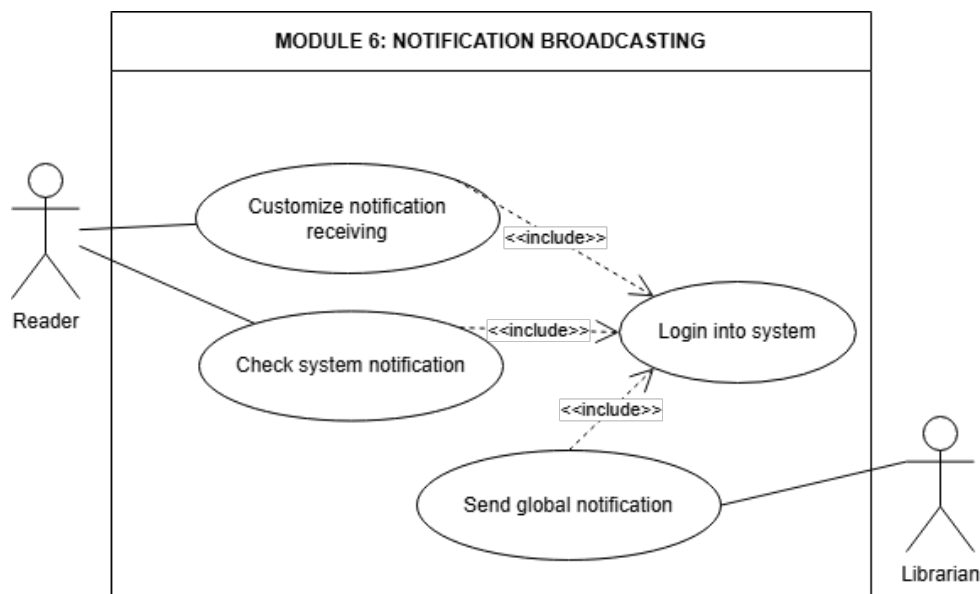


Figure 11: Notifications & Communication Use-Case Diagram

4.6.2 Detailed Specifications

Use-case Name	Configure Notification Preferences
Use-case ID	UC-NOT-01
Overview	Allows readers to customize delivery channels (Email, SMS, In-app) and event alerts (Due Reminders, Hold Pickups, System News).
Actor	Reader
Pre-conditions	Reader possesses valid account.
Trigger Condition	Reader opens "Notification Settings" in profile view.

Flow of Events	1. [Include] System prompts login verification. 2. System loads and renders current notification configuration toggles. 3. Reader toggles preferences for channels and event categories. 4. Reader clicks "Save Settings". 5. System validates and persists configuration to DB. 6. Displays "Notification preferences saved successfully".
Post-conditions	System dispatches notifications strictly per user preferences.
Exceptions	Ex1: SMS toggled on without phone number in profile → Warning alert requesting phone number update first.

Table 22: *Use-case Specification: Configure Notification Preferences*

Use-case Name	View System Notifications
Use-case ID	UC-NOT-02
Overview	Enables users to review alerts and reminders directly within the web app interface.
Actor	Reader
Pre-conditions	User has incoming notification records.
Trigger Condition	User clicks "Bell" icon (Notification Center) on top navbar.
Flow of Events	1. [Include] System verifies user login. 2. Queries notification table by User ID sorted by timestamp desc. 3. Renders notification panel distinguishing read vs unread items. 4. User clicks notification item to view details. 5. System marks notification as "Read" and decrements unread badge count.
Post-conditions	Notification status updated to "Read".
Exceptions	Ex1: Notification panel empty → Display "You have no new notifications".

Table 23: Use-case Specification: View System Notifications

Use-case Name	Broadcast Mass Notifications
Use-case ID	UC-NOT-03
Overview	Allows Librarians to create and send general announcements (holiday closures, maintenance, policy updates) to all or targeted reader groups.
Actor	Librarian
Pre-conditions	Librarian active with broadcast privileges.
Trigger Condition	Librarian accesses "Communication Management" tool in admin portal.
Flow of Events	1. [Include] System verifies Librarian session and privileges. 2. Renders broadcast creation form. 3. Librarian inputs: Title, Content, Channel (In-app, Email), Target Group (All, Students, Faculty). 4. Librarian clicks "Send Broadcast". 5. System prompts confirmation dialog. 6. System enqueues notification payload in Message Queue for dispatch. 7. Displays success confirmation and logs administrative action.
Post-conditions	Broadcast logged and dispatched to target users.
Exceptions	Ex1: Empty title or body → Disable send button and alert "Please enter complete broadcast content".

Table 24: Use-case Specification: Broadcast Mass Notifications

4.7 Reporting and System Administration

4.7.1 Overview

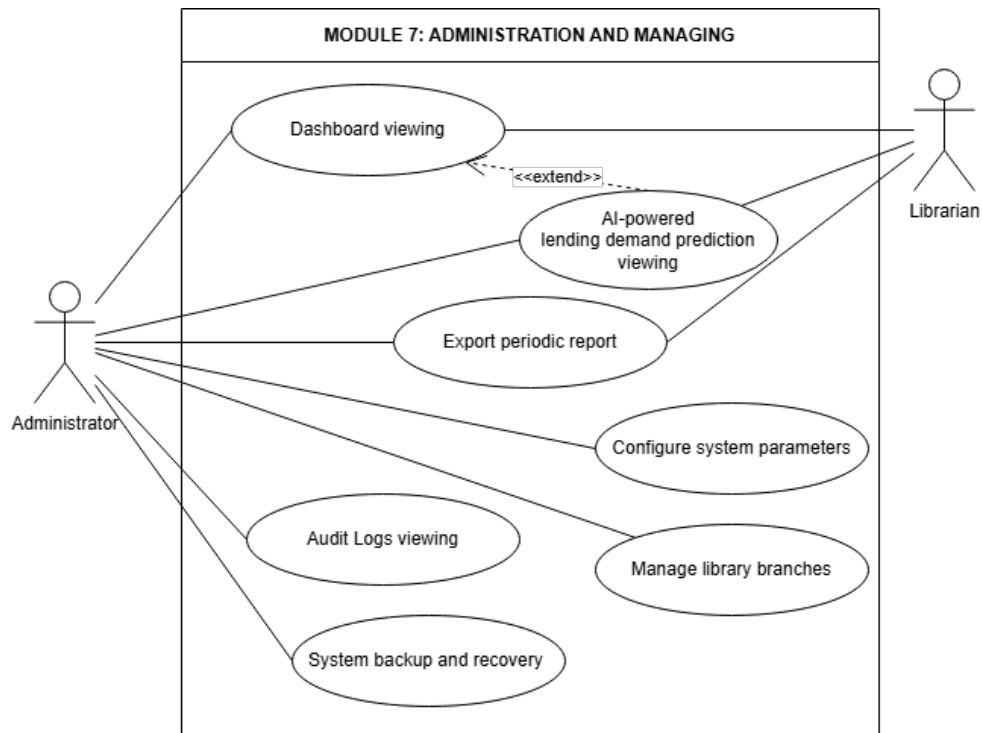


Figure 12: Reporting & Administration Use-Case Diagram

4.7.2 Detailed Specifications

Use-case Name	View Operational Dashboard
Use-case ID	UC-ADM-01
Overview	Provides real-time dashboard visualization of key operational metrics (active loans, overdue items, traffic).
Actor	Administrator (Admin), Librarian
Pre-conditions	User logged in as Admin or Librarian.
Trigger Condition	User opens administrative homepage.
Flow of Events	1. System receives request. 2. Queries DB aggregating real-time daily/weekly metrics. 3. Renders charts and metric summary cards.
Post-conditions	System data state remains unchanged.
Exceptions	Ex1: DB load high → Display cached dashboard data with refresh timestamp notice.

Table 25: Use-case Specification: View Operational Dashboard

Use-case Name	Export Periodic Statistical Reports
Use-case ID	UC-ADM-02
Overview	Supports exporting library metrics and audit data to standard formats (PDF, CSV, Excel) for archiving and reporting.
Actor	Administrator (Admin), Librarian
Pre-conditions	User logged in with report export privileges.
Trigger Condition	User opens "Statistical Reports" and selects "Export Data".
Flow of Events	1. System renders export configuration modal. 2. User selects report type (Top Borrowed, Fine Income, Overdues), date range, and export format. 3. User clicks "Export". 4. System compiles dataset and generates file artifact. 5. File download triggers automatically on client browser.
Post-conditions	Report artifact generated and downloaded.
Exceptions	Ex1: No records match date range → Warning alert "No data available for selected period".

Table 26: Use-case Specification: Export Periodic Statistical Reports

Use-case Name	Configure System Parameters
Use-case ID	UC-ADM-04
Overview	Configures core operational library rules: loan limits, loan durations, and daily overdue fine rates per item type.
Actor	Administrator (Admin)
Pre-conditions	Admin logged into system.
Trigger Condition	Admin opens "System Settings".
Flow of Events	1. System displays current global system settings. 2. Admin modifies values (e.g., fine rate adjustment). 3. Admin clicks "Save Configuration". 4. System validates parameters. 5. Updates DB and records entry in Audit Log.
Post-conditions	System calculations immediately apply updated configuration settings.
Exceptions	Ex1: Invalid parameter input (negative values) → Reject save and display validation error.

Table 27: Use-case Specification: Configure System Parameters

Use-case Name	Manage Branch Libraries
Use-case ID	UC-ADM-05
Overview	Adds, edits, or deactivates affiliated branch libraries, managing physical network topology.
Actor	Administrator (Admin)
Pre-conditions	Admin logged into system.
Trigger Condition	Admin selects "Branch Management".
Flow of Events	1. System renders active branch list. 2. Admin selects Add New, Edit, or Remove. 3. Admin enters branch attributes (Name, Address, Contact, Manager). 4. Admin clicks "Save". 5. System updates records and displays success alert.
Post-conditions	Branch directory updated.
Exceptions	Ex1: Attempting to delete branch with active inventory or staff → System prevents deletion to maintain referential integrity.

Table 28: Use-case Specification: Manage Branch Libraries

Use-case Name	View Audit Logs
Use-case ID	UC-ADM-06
Overview	Inspects historical logs of critical administrative actions (who did what, when) for security auditing.
Actor	Administrator (Admin)
Pre-conditions	Admin logged into system.
Trigger Condition	Admin accesses "Audit Logs" menu.
Flow of Events	1. System retrieves records from Audit Log table. 2. Renders event stream sorted by timestamp desc. 3. Admin filters log stream by actor, date range, or risk severity (Info, Warning, Critical). 4. Admin views detailed payload diffs (Old Data vs New Data).
Post-conditions	Read-only view; system state remains unchanged.
Exceptions	No major exceptions.

Table 29: *Use-case Specification: View Audit Logs*

5 System Analysis and Design

5.1 System Architecture

To visualize data flows and the separation of concerns across technology components in the Online Library Management System, the overall structure is modeled via the architecture diagram below:

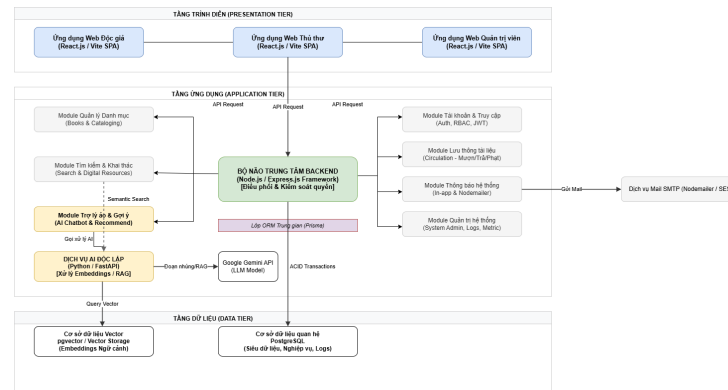


Figure 13: Overall System Architecture Diagram of the Online Library Management System

Overall, the architecture of the Online Library Management System is built upon a standard Three-Tier Architecture, combined flexibly with a Service-Oriented Architecture (SOA) paradigm to optimize independent Artificial Intelligence workloads. The three foundational tiers of the project comprise: the Presentation Tier, Application Tier, and Data Tier.

Applying this model ensures clear separation of responsibilities between user interface interactions, library business logic, and database management systems. Thanks to high inter-tier decoupling, our team can easily maintain or upgrade individual components independently (such as optimizing AI algorithms or updating the web interface) without disrupting core underlying operational processes.

5.1.1 Presentation Tier

Serving as the direct touchpoint for end-users, this tier is implemented as a robust Single Page Application (SPA) utilizing the React.js library and Vite build tool. The interface is tailored with strict Role-Based Access Control (RBAC) across three distinct user roles: Reader, Librarian, and Administrator (Admin).

All user interactions—including searching for documents, submitting checkout requests, chatting with the AI assistant, or executing librarian operations—are captured, asynchronously processed, and converted into standardized RESTful API requests before transmission to the central processing tier.

5.1.2 Application Tier

Functioning as the core orchestration hub, the Application Tier receives, processes, and executes complex business logic required for a smart library system. This tier operates primarily within a Node.js environment paired with the Express.js framework, while integrating a decoupled Python service (FastAPI) dedicated to high-performance AI workloads. To ensure modularity and scalability, the backend is partitioned into 7 functional modules matching system business domains:

- **Account and Access Management Module:** Protects system security via Role-Based Access Control (RBAC) combined with password hashing. The system employs JSON Web Tokens (JWT) to securely manage user sessions. Additionally, this module manages personal user profiles, OTP verification codes, and activity audit logs for readers and library staff.
- **Book Catalog Management Module:** Organizes hierarchical category structures (Category) and stores book metadata (Book). This sub-system assists librarians in automated cataloging by integrating third-party APIs (Open Library, Google Books API) for ISBN metadata fetching.
- **Document Circulation Management Module:** Handles the full lifecycle of physical item checkouts and returns. It tracks stack availability per copy (PhysicalCopy) via barcodes, automates reservation queues (Reservation), calculates online renewal eligibility, enforces fine rules (Fine) for overdue returns, and orchestrates inter-branch item transfers.
- **Search and Resource Exploitation Module:** Provides advanced keyword and semantic search filtering to optimize query performance across the book catalog.

- **Virtual Assistant and Recommendation Module:** Operates as a decoupled high-performance microservice leveraging Large Language Models (LLMs) via Google Generative AI (Gemini) APIs. Capabilities include: Natural language Semantic Search, AI document summarization, preference prediction algorithms for personalized recommendations, and a 24/7 Virtual Assistant Chatbot.
- **Notification Module:** Automates real-time user communications. The system listens for system events (due date reminders, fine assessments, hold availability) to dispatch multi-channel notifications via in-app alerts and automated emails using Nodemailer.
- **System Administration Module:** Provides administrative tools for system management, including dynamic system configuration key-values (`system.configs`), branch network management, real-time metric dashboards (Dashboard Metrics), automated database backups, and snapshot disaster recovery.

To maximize development velocity and type safety, the Application Tier utilizes **Prisma ORM** as the data access layer. Capitalizing on TypeScript's strict static type checking, Prisma catches query syntax errors during compile time. Prisma Migrate manages database schema versioning, ensuring seamless automated schema synchronization between development and production environments.

5.1.3 Data Tier

The data storage layer provides persistent, secure, and ACID-compliant storage for all system records. It comprises two specialized database stores:

- **Relational Database (PostgreSQL):** For library operations requiring strict financial transaction accuracy and concurrent consistency (e.g., decrementing physical inventory while generating loan tickets), **PostgreSQL** is selected for its robust ACID compliance. High-performance index structures ensure low latency under heavy query loads. Additionally, all sensitive state-mutating actions write automated entries to audit logs (AuditLog).
- **Vector Database (pgvector):** To power Retrieval-Augmented Generation (RAG) and semantic search in AI modules, the data tier incorporates vector storage mechanisms. Text summaries, book metadata, and digital document chunks converted into high-dimensional vector embeddings are stored and queried using Cosine Similarity directly in 'pgvector'.

5.2 Database Design

5.2.1 Entity Descriptions

Entity Category	Entity Name	Description
User & Access Control	users	Stores user account profiles including readers, librarians, and system administrators.
	roles	Manages system roles and authorization permissions (Admin, Librarian, Reader).
	otp_tokens	Stores OTP tokens for login authentication, password resets, and MFA.
	notification_settings	Stores user notification preferences across Email, SMS, and In-app channels.
	notifications	Manages system notifications dispatched to users (due date alerts, hold notices).
Library Resources Library Circulation	books	Stores metadata records for books, journals, and digital academic materials.
	categories	Manages hierarchical category classifications for subjects and disciplines.
	physical_copies	Tracks physical copy instances, circulation statuses, and shelf locations.
	borrow_records	Tracks loan and return lifecycles for physical library items.
	reservations	Manages item hold reservation queues when copies are out of stock.
	finest	Manages overdue fines and penalty fee assessments.
	branches	Stores branch library profiles and campus location metadata.
	branch_transfers	Manages physical copy transfers between branch libraries.
Interaction & Reviews	reviews	Stores user ratings, reviews, and feedback on library materials.
	chat_history	Stores conversation transcripts between users and the AI Chatbot assistant.
System & Administration	audit_logs	Records administrative and security audit events for system compliance.
	system_configs	Stores global dynamic configuration parameters for system operation.

Table 30: *System Entities and Database Definitions*

5.2.2 Enhanced Entity-Relationship Diagram (EERD)

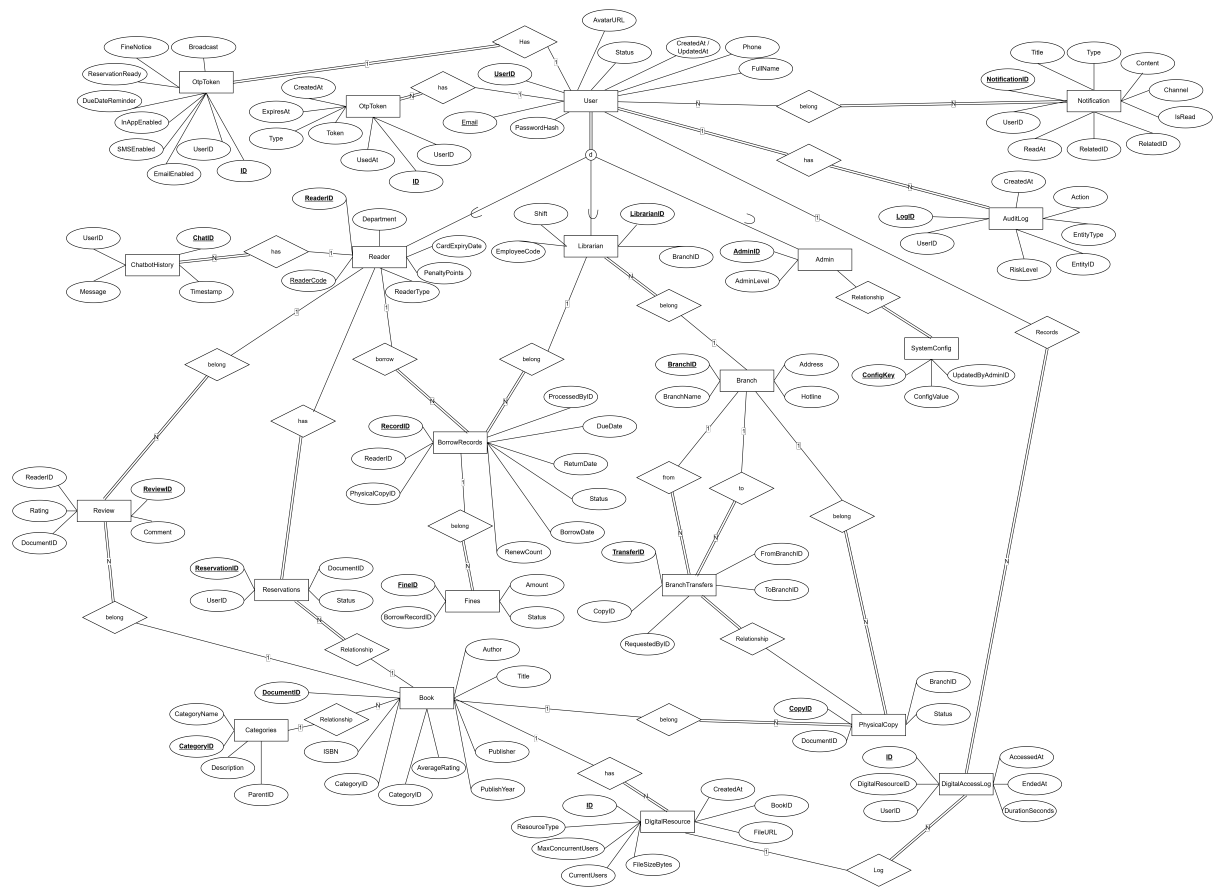


Figure 14: EERD Diagram of the Online Library Management System

Based on the Enhanced Entity-Relationship Diagram (EERD), the system operational workflows are partitioned across three primary user roles:

5.2.2.1 Reader Role End-users interacting with the platform to search, borrow, and read physical and digital items:

- **Account Management & Security:** Executes multi-factor authentication via OTP tokens (otp.tokens); configures notification channels (Email, SMS, In-app) in notification_settings.
- **Physical Borrowing & Catalog Search:** Searches and browses categories (categories). Borrows physical materials with loan records tracked in borrow_records.
- **Physical Borrowing & Returns:** Submits loan requests and tracks borrowing lifecycles in borrow_records tied to specific copies (physical_copies).
- **Reservation Queue Management:** When items are out of stock, places hold reservations (reservations) with automated queue positioning via queuePosition.
- **Feedback & Ratings:** Submits star ratings and reviews (reviews); chats with the AI Virtual Assistant (chat_history); views and pays fine ledgers (fines).

5.2.2.2 Librarian Role Operational staff managing physical stacks, catalog entries, and counter circulation:

- **Physical Stack Management:** Manages catalog records (books), copy availability (status), physical condition (condition), and shelf location (location) in physical_copies.
- **Circulation Desk Processing:** Scans barcodes (barcode) and reader cards (readerCode), updating borrow_records.

- **Reservation Queue Orchestration:** Monitors reservation queues (reservations), updating copy statuses upon return and alerting next patrons.
- **Inter-Branch Logistics:** Approves and tracks item transfer tickets between branch libraries via branch.transfers.
- **Fine Assessment & Waivers:** Reviews overdue penalties in fines, possessing authorization to grant fine waivers (waivedById, waivedReason).

5.2.2.3 Administrator Role System administrators holding global configuration, user security, and audit privileges:

- **User Management & RBAC:** Activates, locks, or suspends accounts (users), managing failed login limits (failedLoginCount) and role permissions (roles).
- **Infrastructure Administration:** Manages branch network configurations (branches) and assigns branch managers (managerId).
- **Global Parameters:** Configures operational system policies via system_configs (daily fine rates, loan limits, max file upload sizes).
- **System Audit & Compliance:** Monitors audit logs (audit_logs) for data security and regulatory compliance.

5.2.3 Database Schema

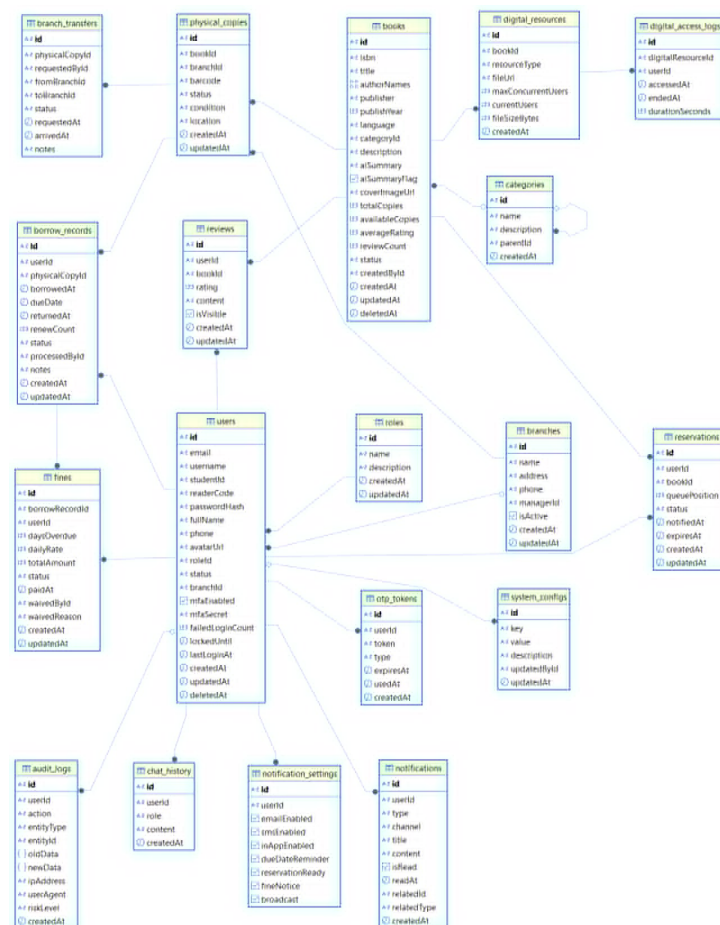


Figure 15: Relational Database Schema for the Online Library Management System

The schema above represents the relational structure implemented on PostgreSQL.

User accounts are centralized in the users table, linked to roles via foreign key roleId for RBAC permission checks. Security flags such as status, failedLoginCount, and lockedUntil enforce login security constraints.

Remaining tables map directly to the entity definitions and EERD relationships detailed above.

5.3 Class Diagram

5.3.1 Account and Access Management

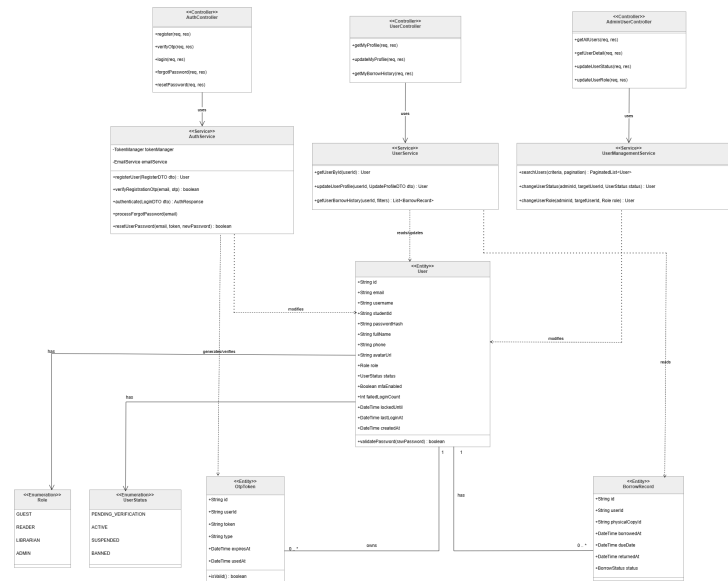


Figure 16: Class Diagram for Account and Access Management

The **Account and Access Management** class diagram models object structures, attributes, and methods for authentication and profile management.

The central entity is **User**, containing attributes `id`, `email`, `username`, `fullName`, `phone`, `avatarUrl`, `studentId`, alongside security attributes `passwordHash`, `mfaEnabled`, `failedLoginCount`, `lockedUntil`, `lastLoginAt`, and `createdAt`. The core method `validatePassword(rawPassword)` performs credential verification.

User holds associations with **Role** (GUEST, READER, LIBRARIAN, ADMIN) and **UserStatus** (PENDING_VERIFICATION, ACTIVE, SUSPENDED, BANNED). **User** associates 1 – 0..* with **OtpToken** (method `isValid()`) and 1 – 0..* with **BorrowRecord**.

Architecturally, **AuthController** relies on **AuthService** (`registerUser`, `verifyRegistrationOtp`, `authenticate`, `processForgotPassword`, `resetPasswordUser`). **UserController** relies on **UserService** (`getUserById`, `updateUserProfile`, `getUserBorrowHistory`). **AdminController** uses **UserManagementService** (`searchUsers`, `getUserDetail`, `changeUserStatus`, `changeUserRole`).

5.3.2 Book Catalog Management



Figure 17: Class Diagram for Book Catalog Management

Core data structures include Category, Book, and PhysicalCopy. Category features a self-referential hierarchy ($1 \rightarrow 0..*$) via `parentId` and associates $1 \rightarrow 0..*$ with Book.

Book holds catalog metadata (`isbn`, `title`, `authorNames`, `publisher`, `publishYear`, `language`, `totalCopies`, `availableCopies`, `aiSummary`) and associates $1 \rightarrow 0..*$ with PhysicalCopy (`barcode`, `CopyStatus` status, `condition`).

CatalogController delegates to BookCatalogService (`addBook`, `updateBook`, `deleteBook`, `autoFillByIsbn`, `generateAiSummary`, `processExcelImport`). BookCatalogService interacts with CatalogingService, which wraps `IsbnApiClient` (`queryOpenLibrary`, `queryGoogleBooks`) and `LLMEngine` (`generateBookSummary`) for automated cataloging.

5.3.3 Document Circulation Management

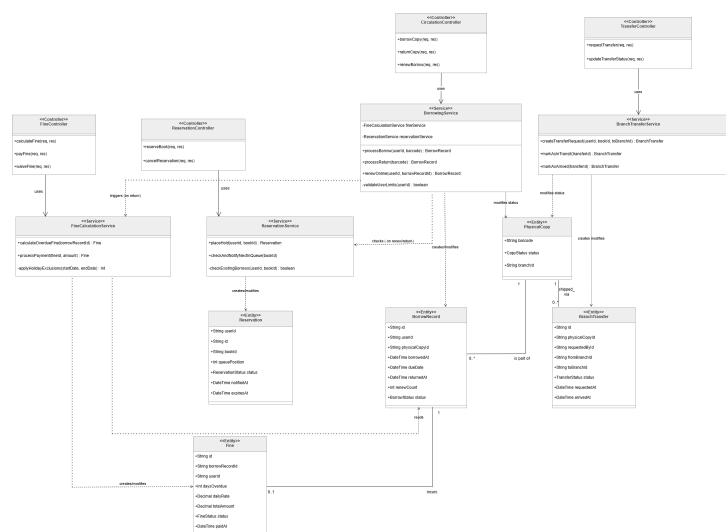


Figure 18: Class Diagram for Document Circulation Management

The central entity BorrowRecord (borrowedAt, dueDate, returnedAt, renewCount, BorrowStatus status) associates with PhysicalCopy (1 – 0..*). Reservation (queuePosition, ReservationStatus status) and Fine (1 – 0..1 with BorrowRecord) handle hold queues and fine assessments. BranchTransfer tracks inter-branch item transfers.

CirculationController uses BorrowingService (processBorrow, processReturn, renewOnline). ReservationController uses ReservationService (placeHold, cancelReservation). FineController uses FineCalculationService (calculateOverdueFine, processPayment). TransferController uses BranchTransferService (createTransferRequest, markInTransit, markAsArrived).

5.3.4 Resource Search and Exploitation

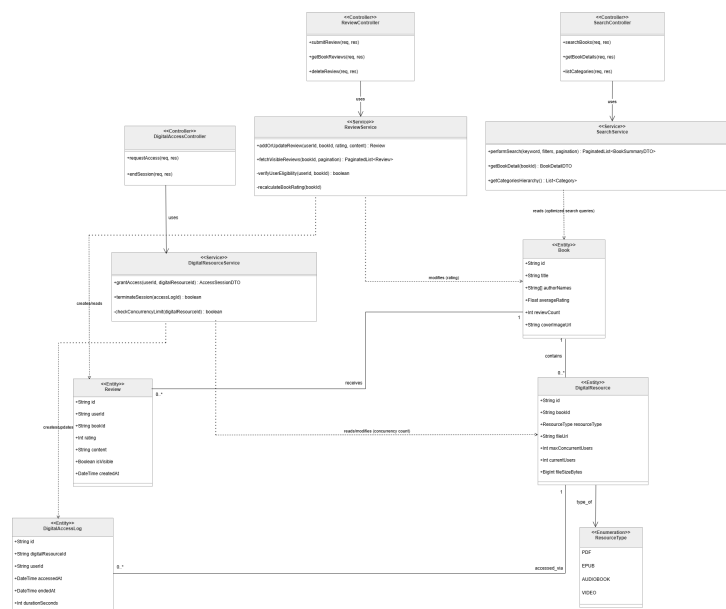


Figure 19: Class Diagram for Resource Search and Exploitation

Book associates 1 – 0..* with Review (0..* – 1 with Book) storing user ratings.

SearchController uses SearchService (performSearch, getBookDetail). ReviewController uses ReviewService (addOrUpdateReview, recalculateBookRating).

5.3.5 AI Virtual Assistant and Recommendations

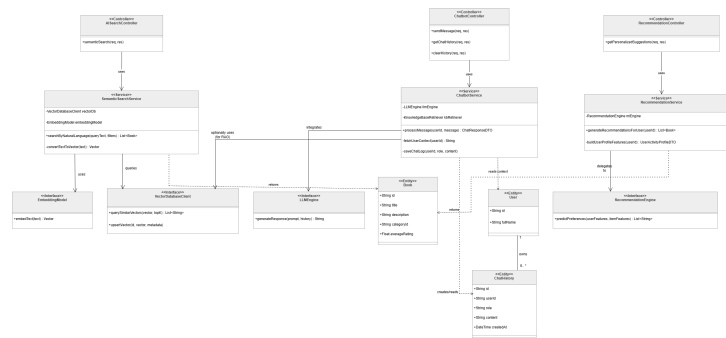


Figure 20: *Class Diagram for AI Virtual Assistant and Recommendations*

AISearchController uses SemanticSearchService (searchByNaturalLanguage) interfacing with EmbeddingModel and VectorDatabaseClient.

ChatbotController uses ChatbotService (sendMessage, getChatHistory), interfacing with LLMEngine and VectorDatabaseClient (RAG context), logging chats in ChatHistory.

RecommendationController uses RecommendationService (getPersonalizedSuggestions), leveraging RecommendationEngine (predictPreferences) to return tailored book lists.

5.3.6 Notifications and Communication

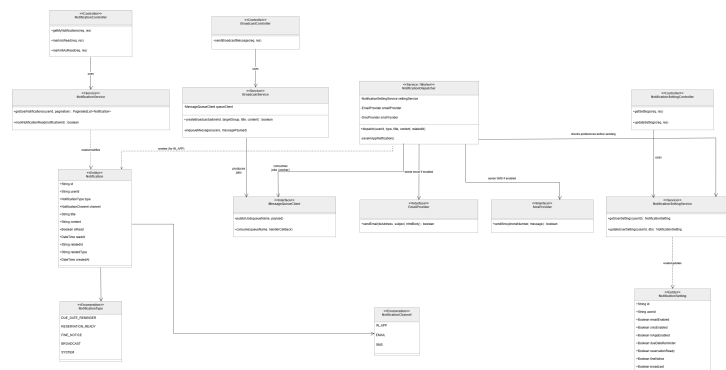


Figure 21: *Class Diagram for Notifications and Communication*

Entities comprise Notification (isRead, NotificationType, NotificationChannel) and NotificationSetting (emailEnabled, smsEnabled, inAppEnabled).

NotificationSettingController uses NotificationSettingService. NotificationController uses NotificationService. BroadcastController uses BroadcastService pushing jobs to MessageQueueClient. Async worker NotificationDispatcher consumes queue jobs and dispatches alerts via EmailProvider and SmsProvider.

5.3.7 System Administration

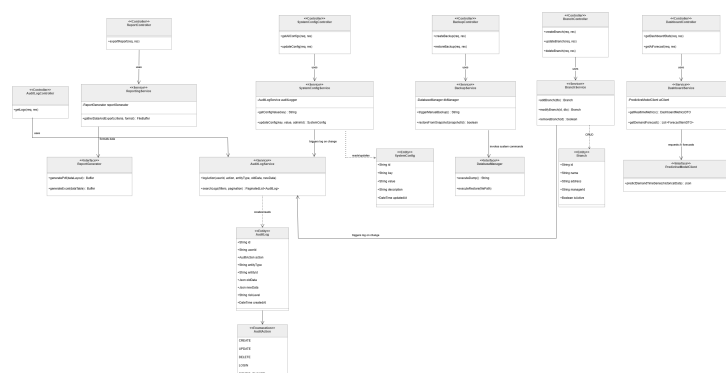


Figure 22: *Class Diagram for System Administration*

Core entities: AuditLog (action, oldData, newData, riskLevel), SystemConfig (key, value), and Branch (name, address, isActive).

AuditLogController uses AuditLogService. ReportController uses ReportingService (ReportGenerator). SystemConfigController uses SystemConfigService. BackupController uses BackupService (DatabaseManager). BranchController uses BranchService. DashboardController uses DashboardService interfacing with PredictiveModelClient for demand forecasting.

5.4 Activity Diagrams

5.4.1 Account Registration

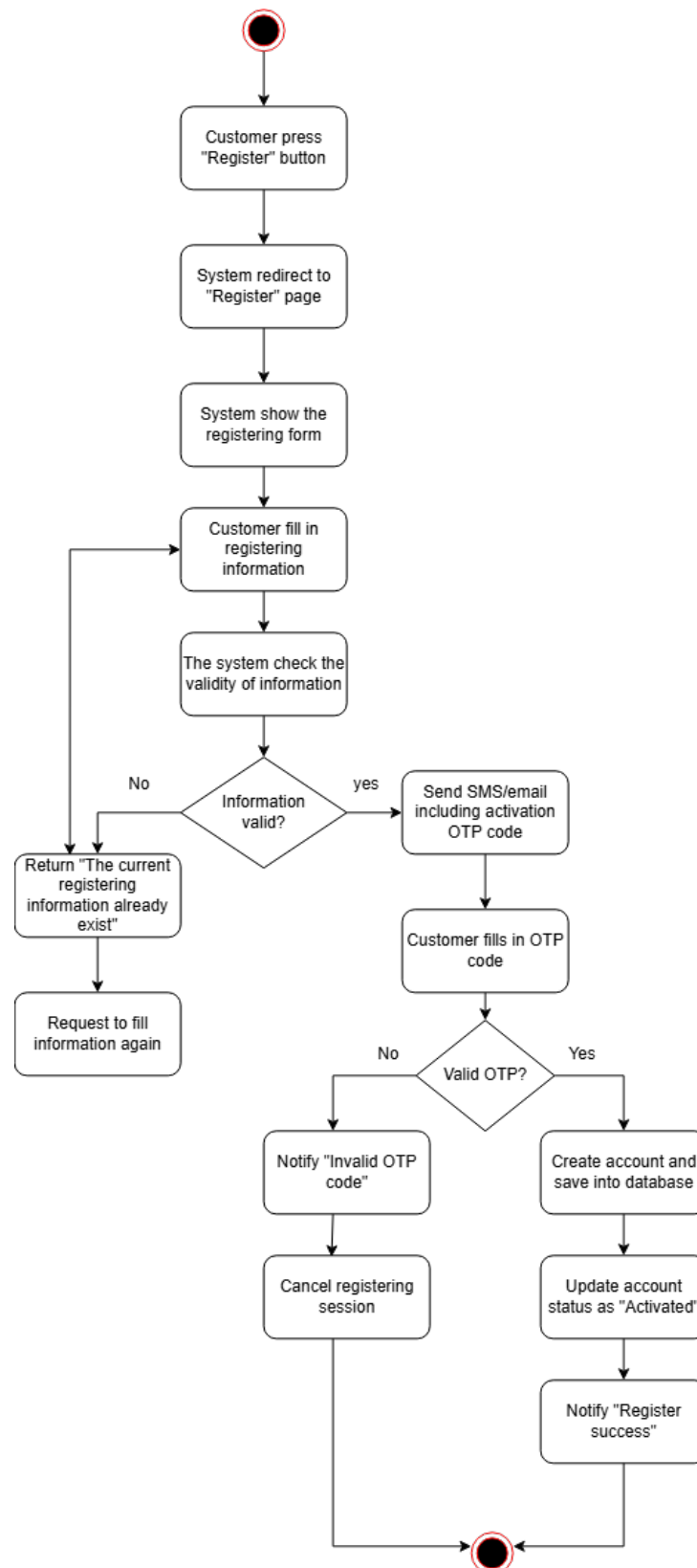


Figure 23: Activity Diagram for Account Registration

1. Guest accesses the account registration page.
2. System displays registration form.

3. Guest enters credentials (Full Name, Email/Phone, Password).
4. System validates input data syntax.
5. System checks if Email/Phone exists in database.
6. **Valid Branch:**
 - If valid, system sends Email/SMS containing OTP code or activation link.
 - Guest enters OTP code or clicks activation link.
 - System validates OTP code or link token.
 - If OTP/link is valid: System creates user account, saves to DB, sets status to "Activated", and alerts success.
7. **Invalid Branch:**
 - If data is invalid or Email/Phone exists: System displays error alert and prompts re-entry.
 - If OTP is invalid: System displays invalid OTP error and cancels registration session.

5.4.2 System Login

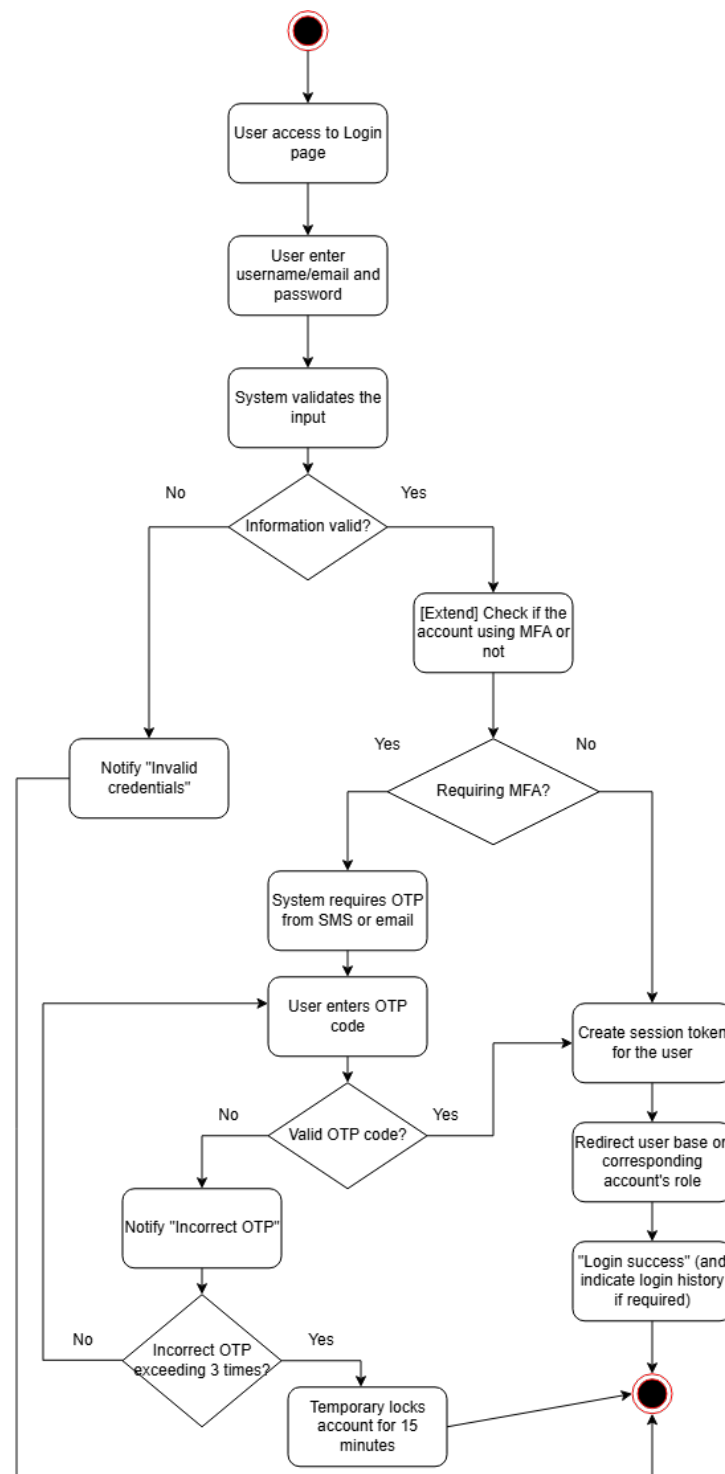


Figure 24: Activity Diagram for System Login

1. User accesses login view.
2. User enters Email/Username and Password.
3. System validates credentials against DB.
4. If credentials invalid: System displays error "Incorrect account or password".
5. If credentials valid: System checks if Multi-Factor Authentication (MFA) is required.
6. If MFA enabled: System requests OTP code; user enters OTP; system validates code.

7. If OTP invalid: Displays error "Incorrect OTP code". If failed attempts ≥ 3 , account is temporarily locked for 15 minutes.
8. If OTP valid (or MFA not required): Initializes session/token, redirects user to role dashboard, and displays success notification.

5.4.3 Password Reset

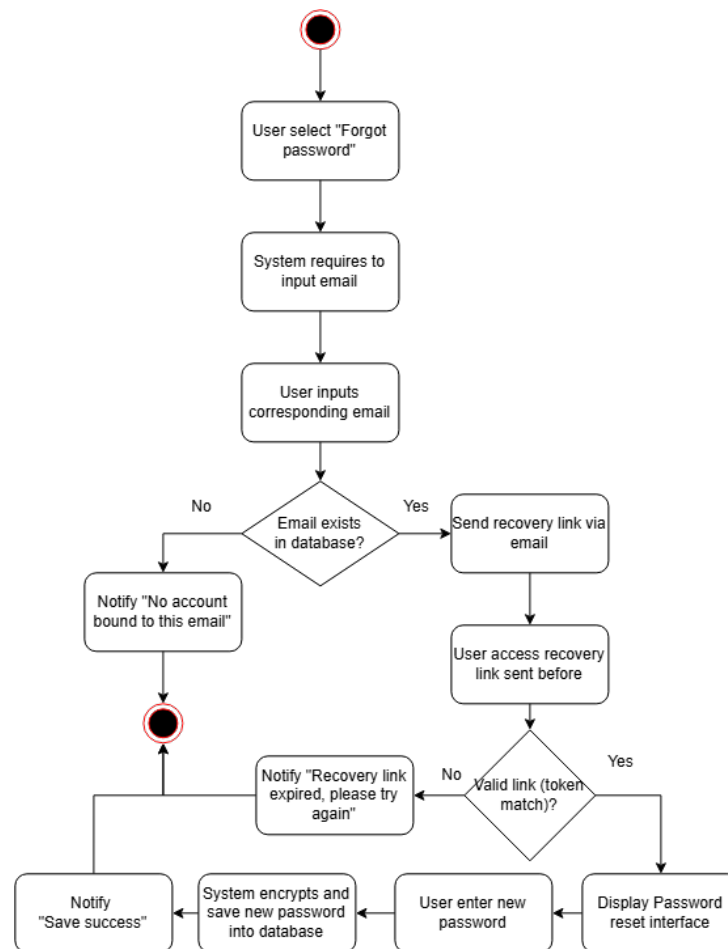


Figure 25: Activity Diagram for Password Reset

1. User selects "Forgot Password".
2. System prompts for Email.
3. User submits Email.
4. System verifies Email existence. If not found: Display "No account found with this email".
5. If found: System sends reset link via Email. User opens link; system verifies token validity.
6. If link expired/invalid: Display "Reset link expired" and prompt process restart.
7. If link valid: Display reset form; user enters new password; system hashes and updates password, alerting success.

5.4.4 View Loan/Return History

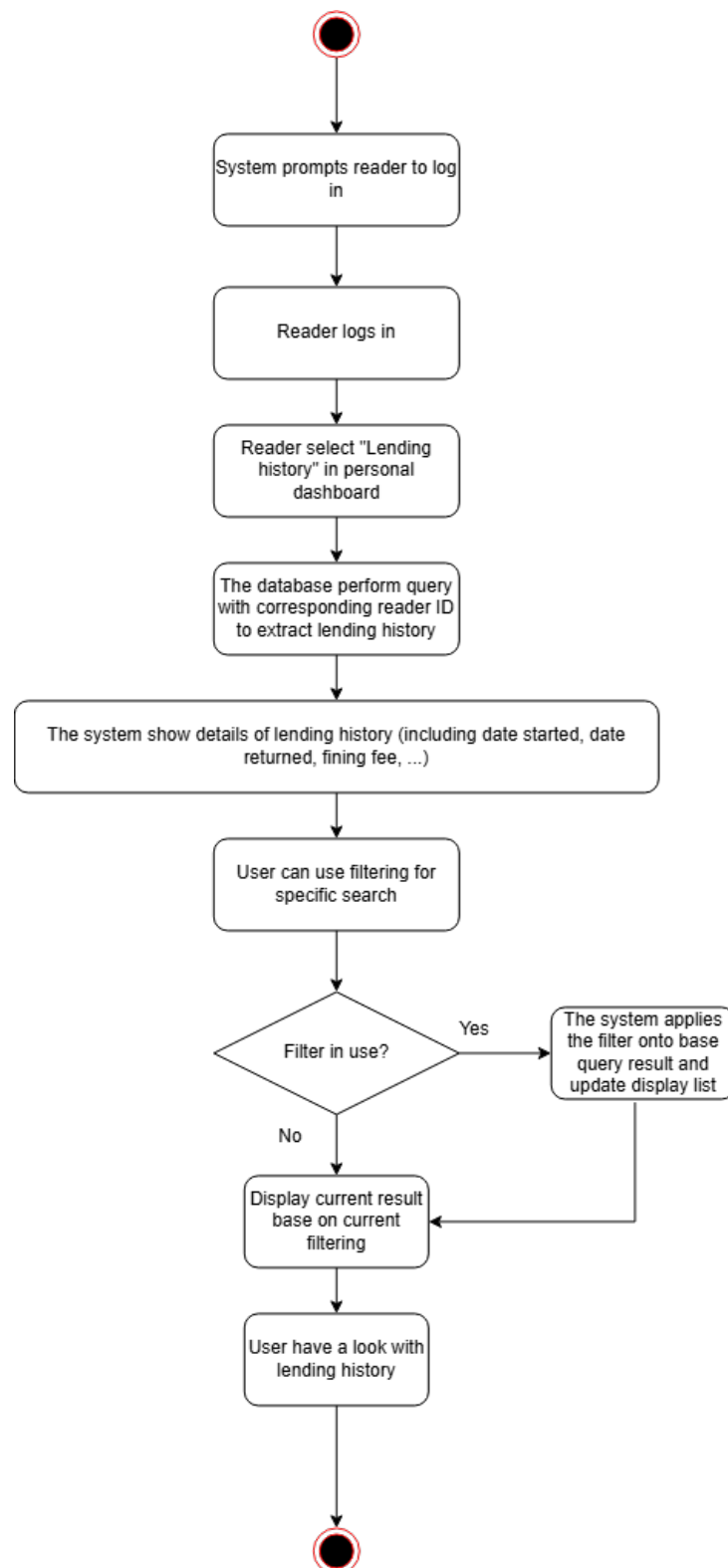


Figure 26: Activity Diagram for View Loan/Return History

1. User logs into system.

2. User opens "Borrowing History" on dashboard.
3. System queries DB for loan records matching User ID.
4. System renders loan history table (Title, Borrow Date, Due Date, Return Date, Status, Fines).
5. User can apply filters by date range or status.
6. System refreshes table view based on applied filter parameters.

5.4.5 User Account Management

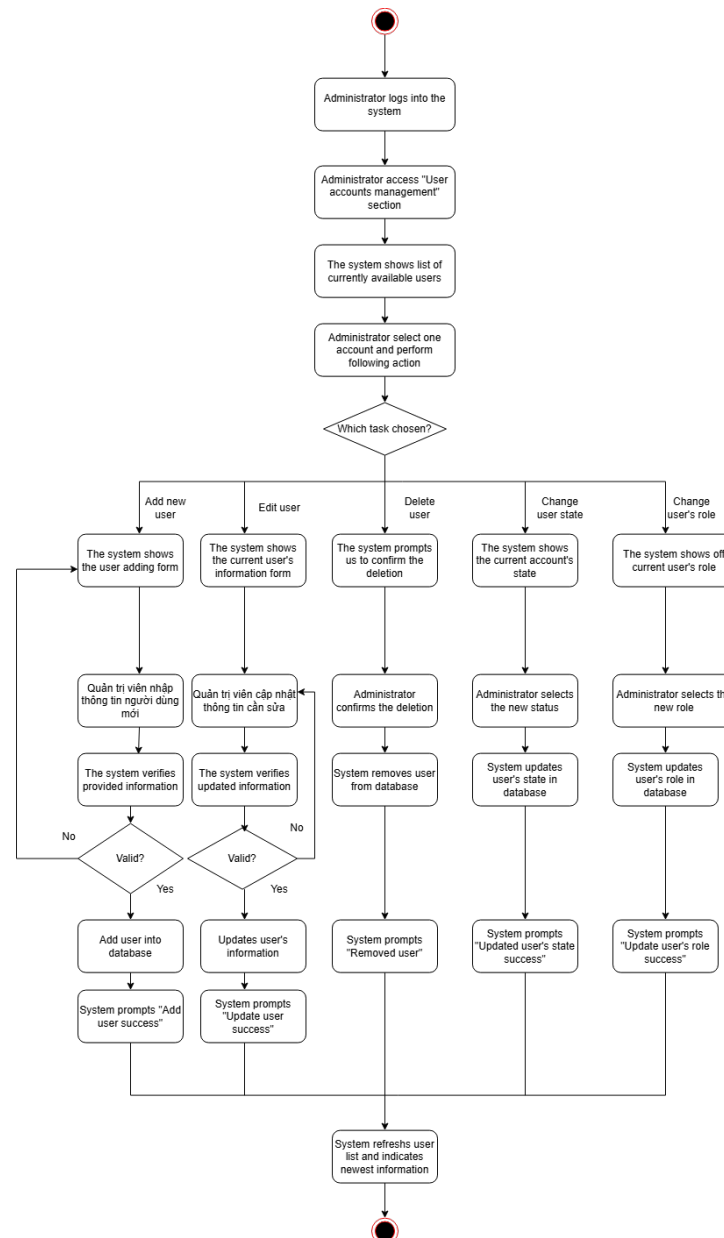


Figure 27: Activity Diagram for User Account Management

1. Admin logs in and accesses "User Management".
2. System renders user list (ID, Name, Email, Role, Status).
3. Admin selects a target user and an operation (Add User, Edit Profile, Delete User, Toggle Status, Change Role).
4. Executes selected workflow; system validates input data and updates DB, displaying success confirmation upon completion.

5.4.6 Add / Edit / Delete Catalog Record

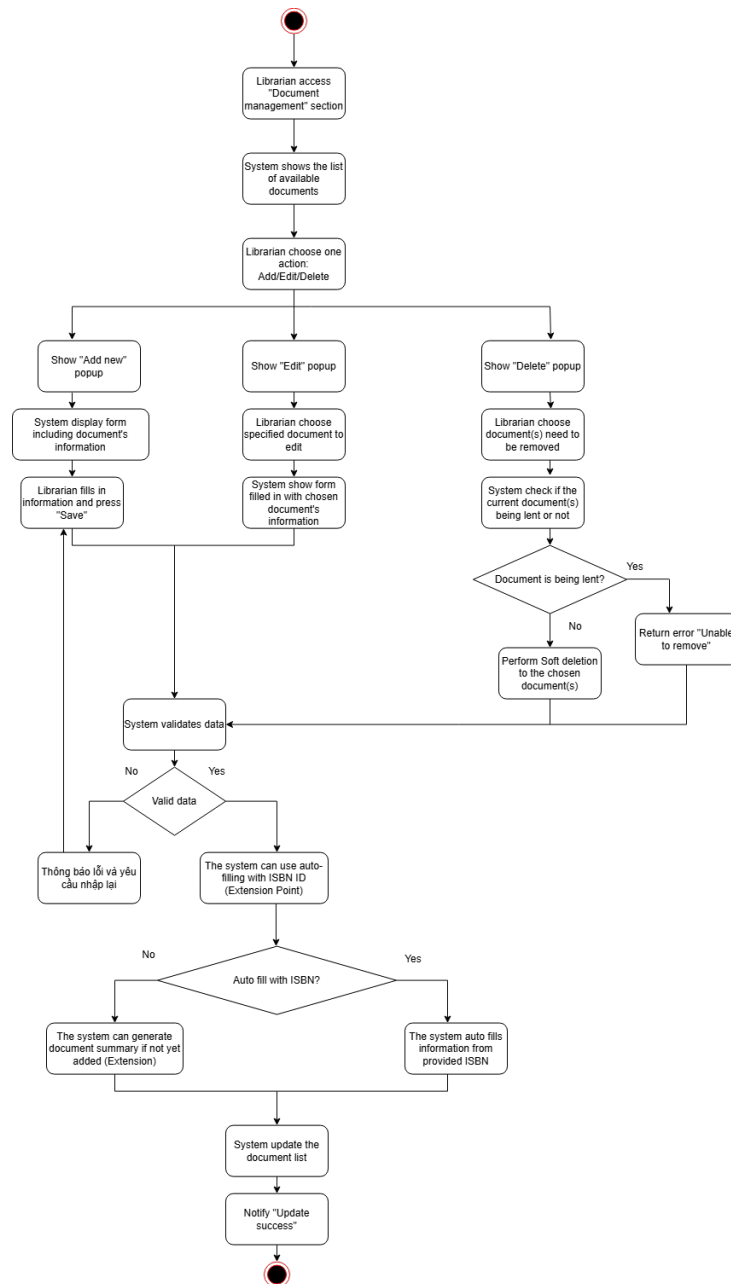


Figure 28: Activity Diagram for Add / Edit / Delete Catalog Record

1. Librarian opens "Catalog Management".
2. Librarian selects action: Add, Edit, or Delete.
3. If Delete: System verifies active loans. If item is checked out, deletion is blocked ("Cannot delete: Item checked out"). Otherwise, soft delete is performed.
4. If Add/Edit: Form is rendered; optional ISBN auto-cataloging (UC-CAT-03) or AI summary generation (UC-CAT-04) can be triggered.
5. System validates fields, persists updates, and alerts success.

5.4.7 Batch Catalog Data Ingestion

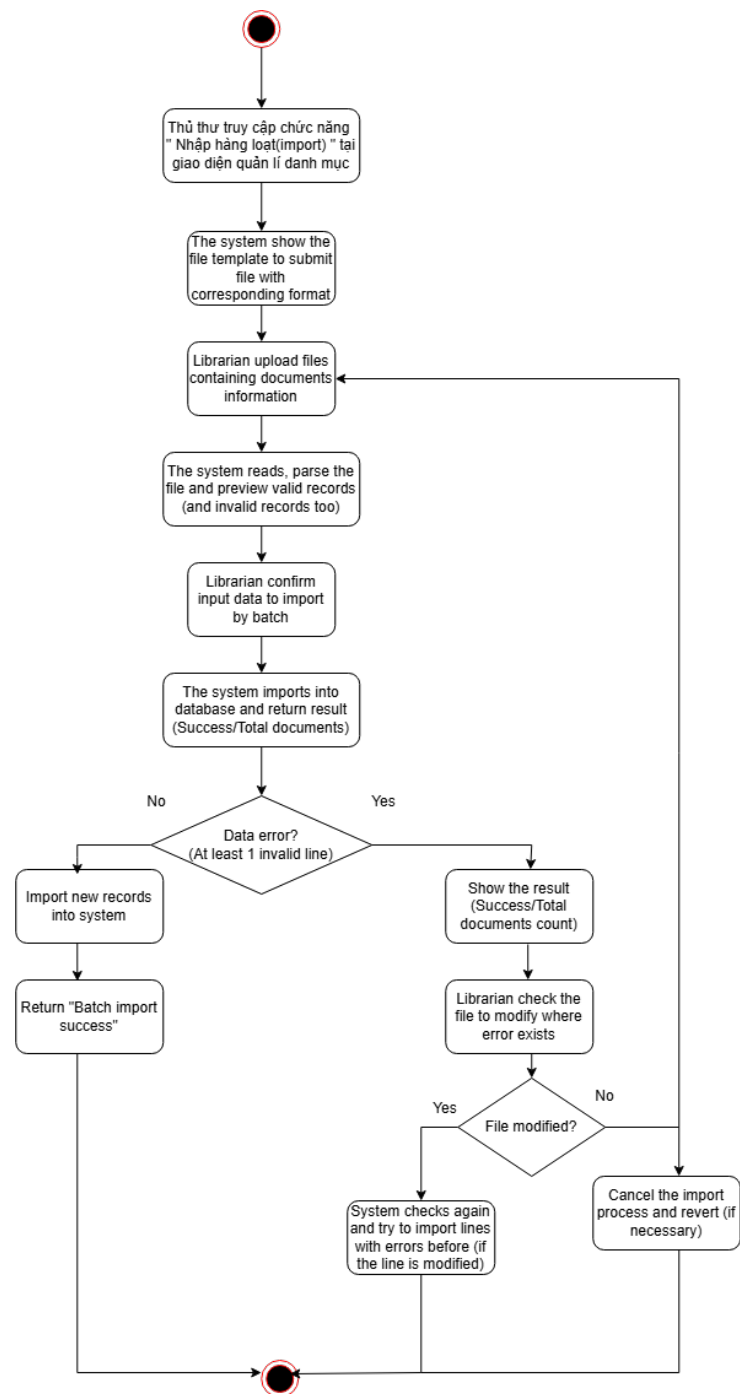


Figure 29: Activity Diagram for Batch Catalog Data Ingestion

1. Librarian selects "Batch Import".
2. Uploads catalog data file (CSV/Excel).
3. System parses file and displays preview table with valid/invalid record counts.
4. Librarian confirms import execution.
5. System writes valid rows to DB and reports results. If errors exist, librarian can fix file and re-upload or cancel operation.

5.4.8 Automated Cataloging via ISBN

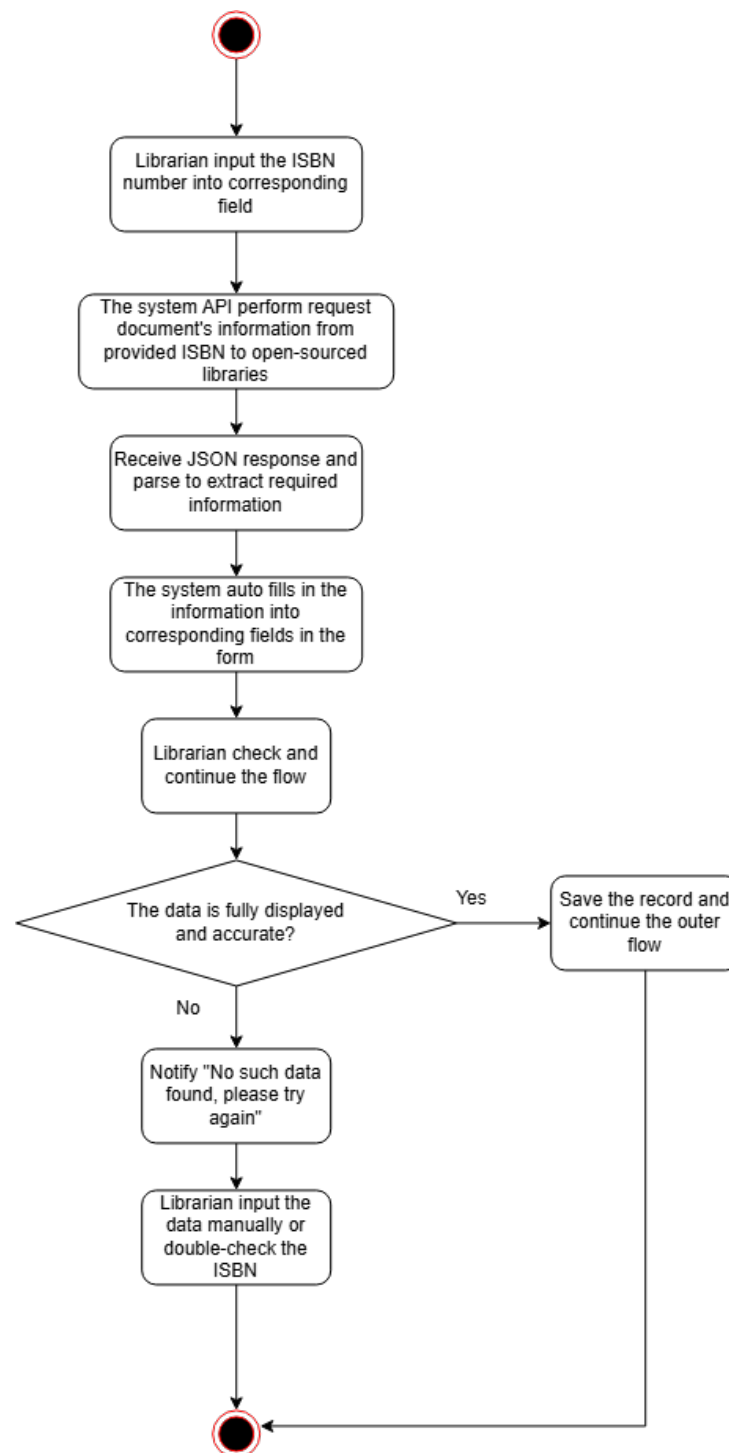


Figure 30: Activity Diagram for Automated Cataloging via ISBN

1. Librarian enters ISBN.
2. System sends API request to external sources (Google Books, Open Library).
3. Parses JSON response and populates form fields.
4. Librarian reviews and saves catalog record. If ISBN is not found, librarian switches to manual data entry.

5.4.9 Physical Material Checkout

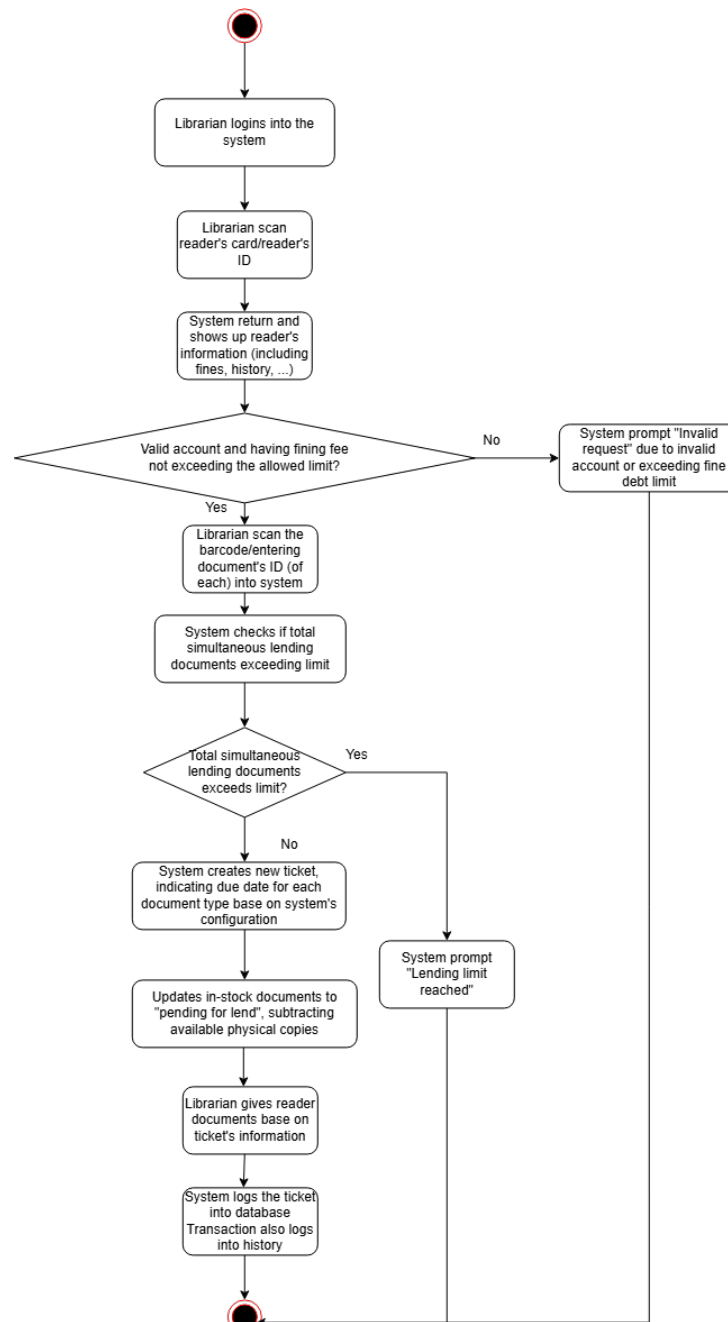


Figure 31: Activity Diagram for Physical Material Checkout

1. Librarian logs in and scans Reader card barcode.
2. System verifies reader account status and fine balance.
3. If eligible, librarian scans item barcodes.
4. System checks loan limits per role and hold queue restrictions.
5. Creates loan record, calculates due date, updates copy status to "Checked Out", and decrements inventory count.

5.4.10 Material Return

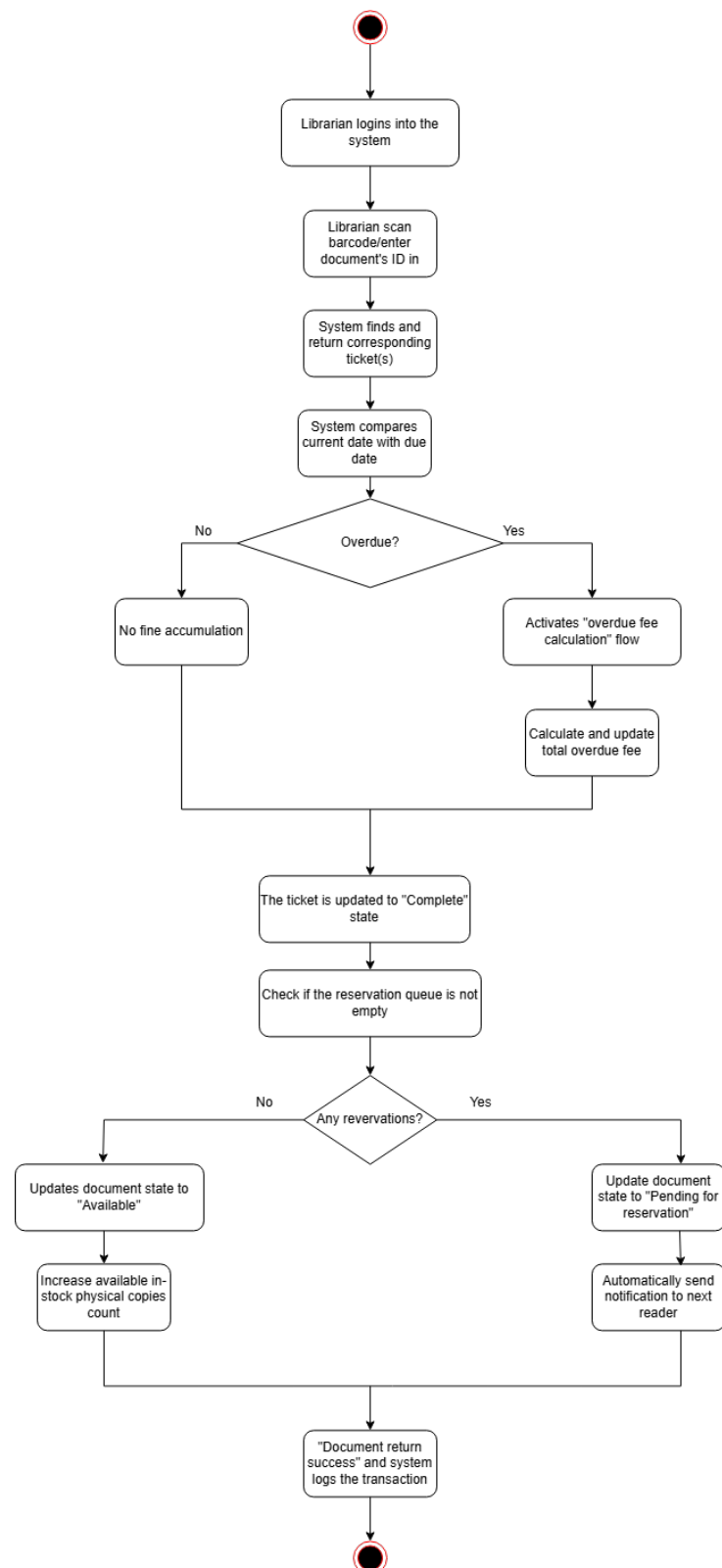


Figure 32: Activity Diagram for Material Return

1. Librarian scans returned item barcode.
2. System retrieves loan record and checks due date.
3. If overdue, triggers fine assessment workflow (UC-CIR-03).
4. Sets loan status to "Completed".

5. Checks reservation queue: if hold exists, sets copy to "Awaiting Pickup" and alerts next user; if no hold, sets copy to "Available".

5.4.11 Calculate and Assess Overdue Fines

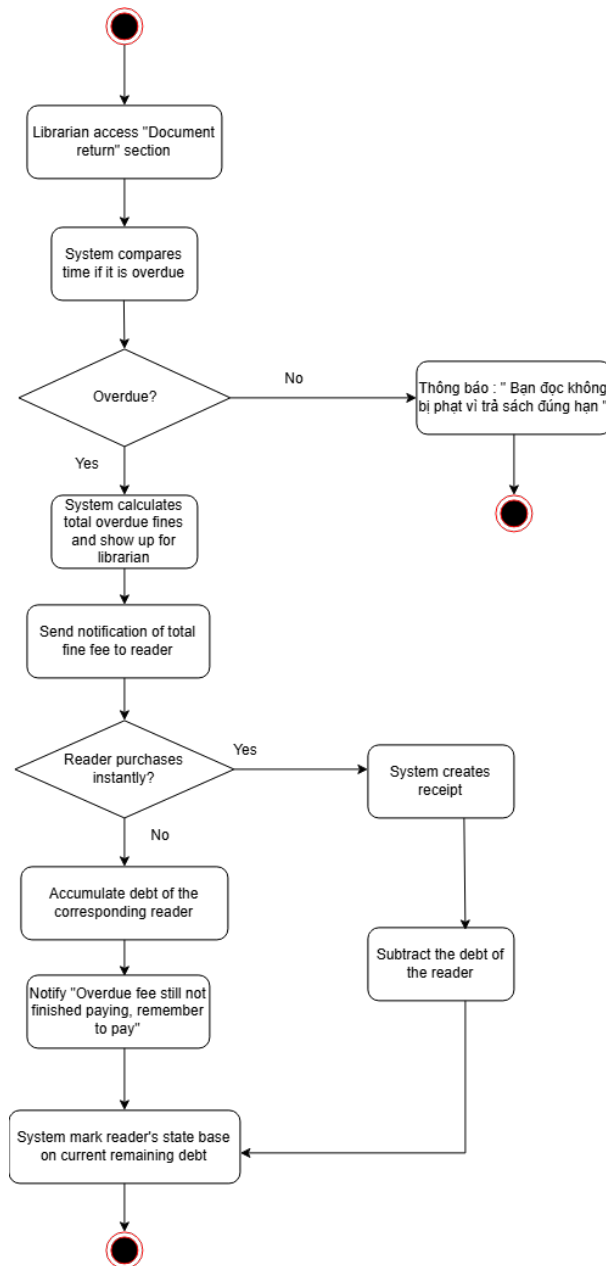


Figure 33: Activity Diagram for Calculate and Assess Overdue Fines

1. System computes overdue days (Return Date - Due Date), excluding library holidays.
2. Multiplies overdue days by daily fine rate.
3. Renders fine calculation. If reader pays immediately, generates receipt and sets ledger balance to 0; otherwise posts fine balance to reader account ledger.

5.4.12 Online Loan Renewal

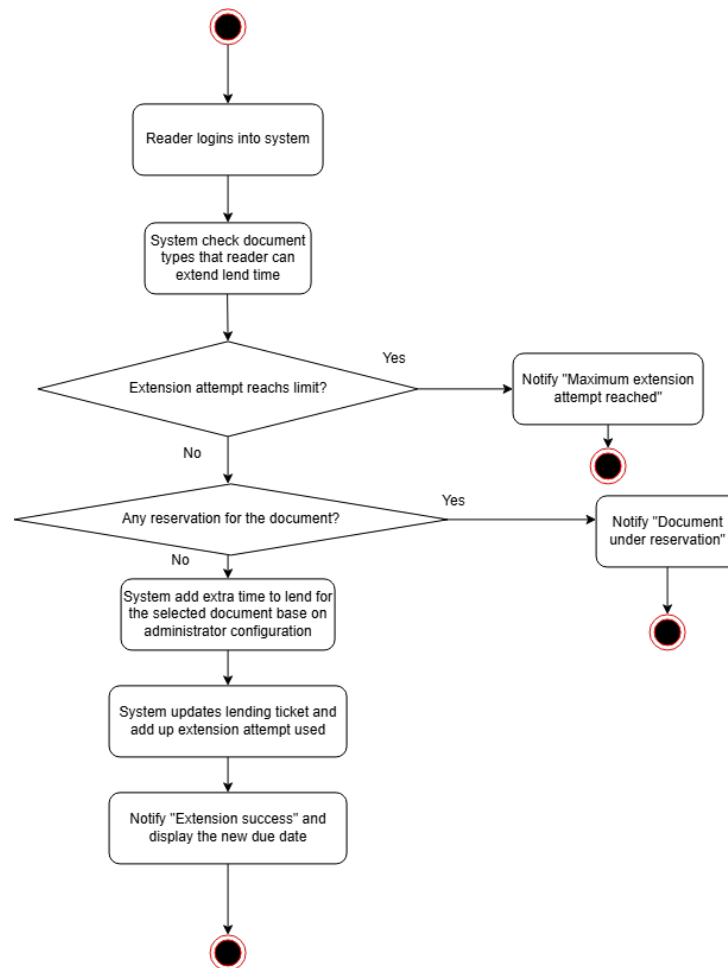


Figure 34: Activity Diagram for Online Loan Renewal

1. Reader opens active loans and clicks "Renew".
2. System verifies: maximum renewal count not reached and no hold requests exist.
3. If eligible, extends due date, increments renewal count, and displays new due date.

5.4.13 Place Item Hold

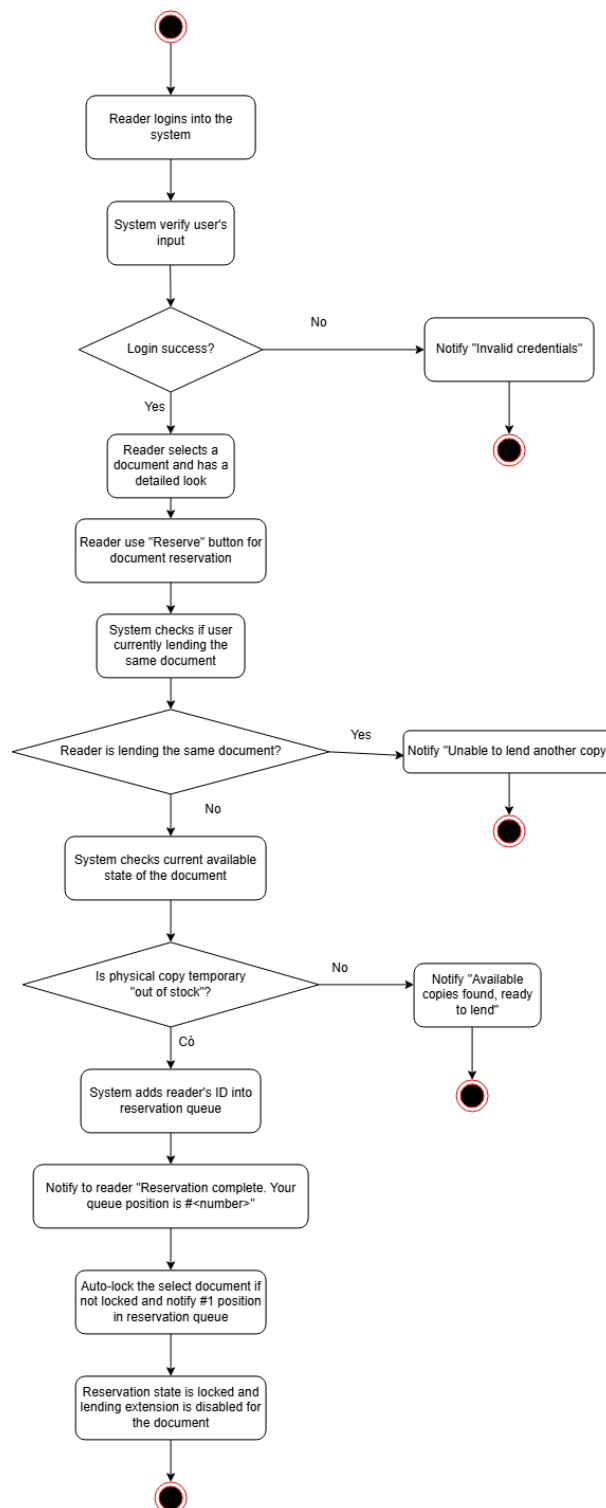


Figure 35: Activity Diagram for Place Item Hold

1. Reader opens item page and clicks "Place Hold".
2. System verifies reader does not currently hold active loan for item and item is out of stock.
3. System appends reader to reservation queue and displays queue position.

5.4.14 Document Catalog Search

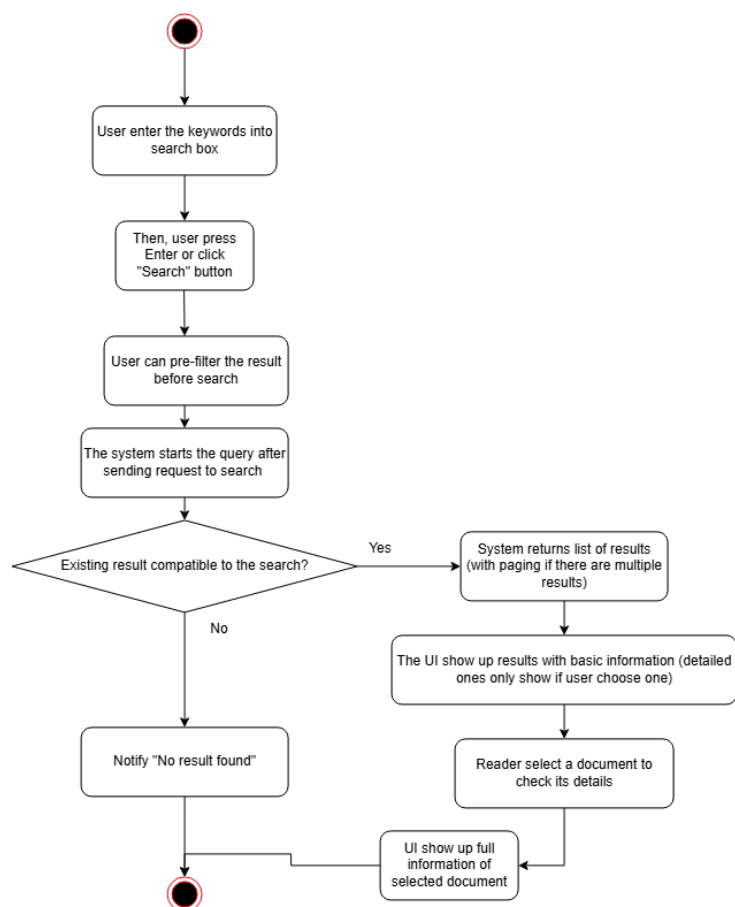


Figure 36: Activity Diagram for Document Catalog Search

1. Reader enters query keywords and selects search criteria.
2. System queries catalog DB indexes and returns paginated results.
3. Displays book cards. Reader clicks book card to view detail page.

5.5 Sequence Diagrams

5.5.1 Account Registration

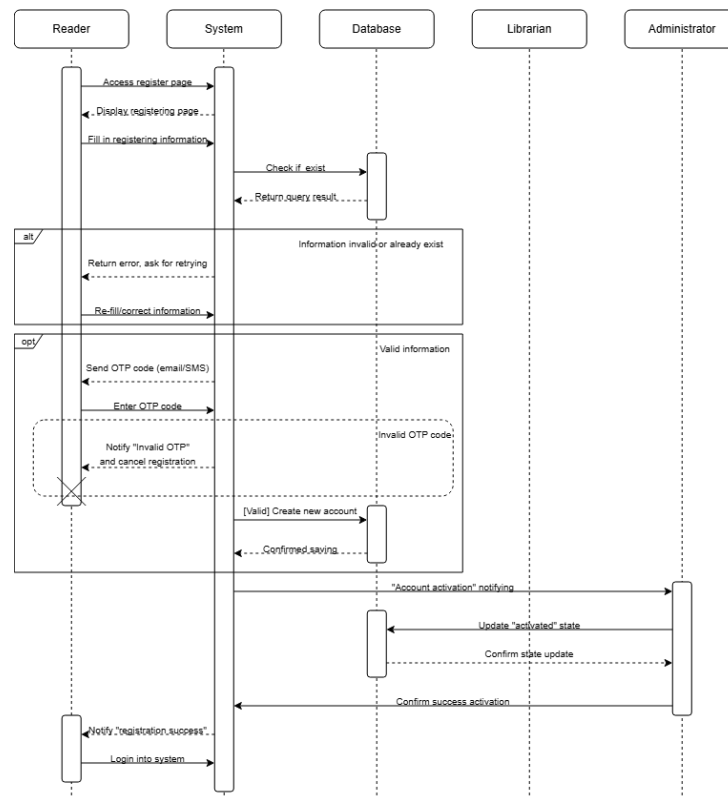


Figure 37: Sequence Diagram for Account Registration

1. Reader accesses registration view.
2. Submits details; system validates inputs and checks for duplicates.
3. System sends OTP code via email/SMS.
4. Reader enters OTP. If OTP valid, account is created and activated; otherwise registration is canceled.

5.5.2 Account Login

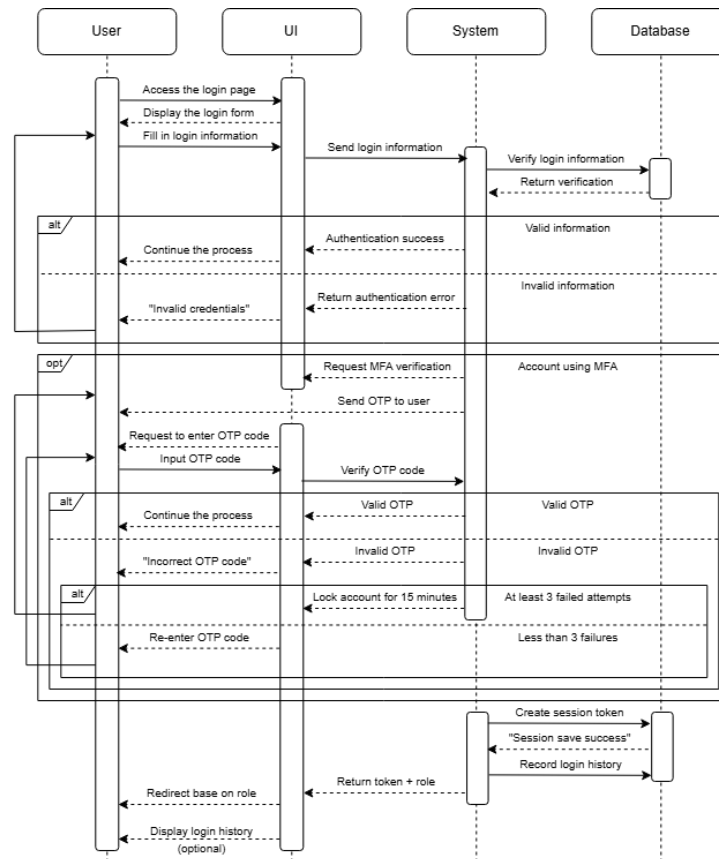


Figure 38: Sequence Diagram for Account Login

1. User submits credentials.
2. System matches credentials against DB.
3. If MFA enabled, requests OTP; upon successful OTP verification, issues session JWT token and redirects user to role dashboard.

5.5.3 Account Password Reset

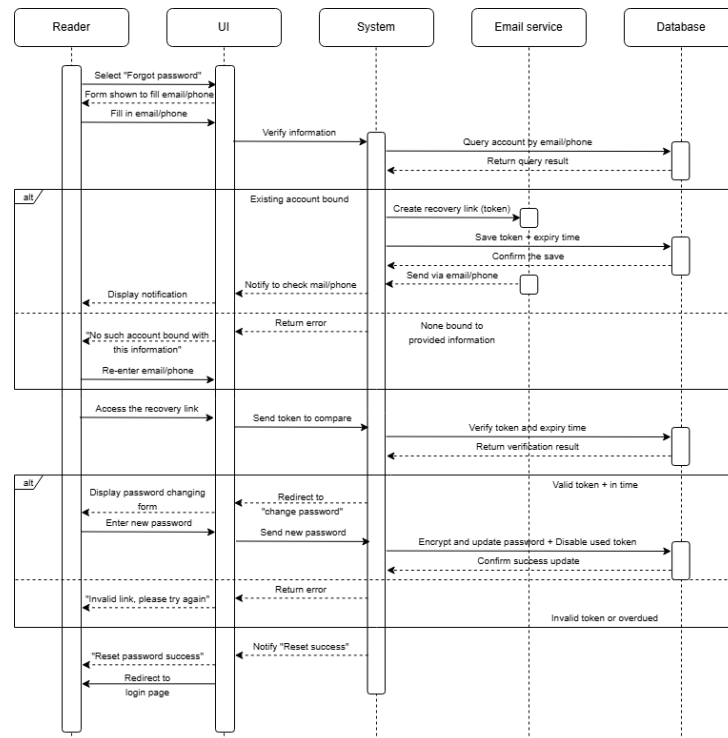


Figure 39: Sequence Diagram for Account Password Reset

1. User requests password reset by entering registered email.
2. System verifies email, generates reset token, and dispatches reset link via email.
3. User opens link; system verifies token validity. User submits new password; system updates password hash.

5.5.4 Document Search

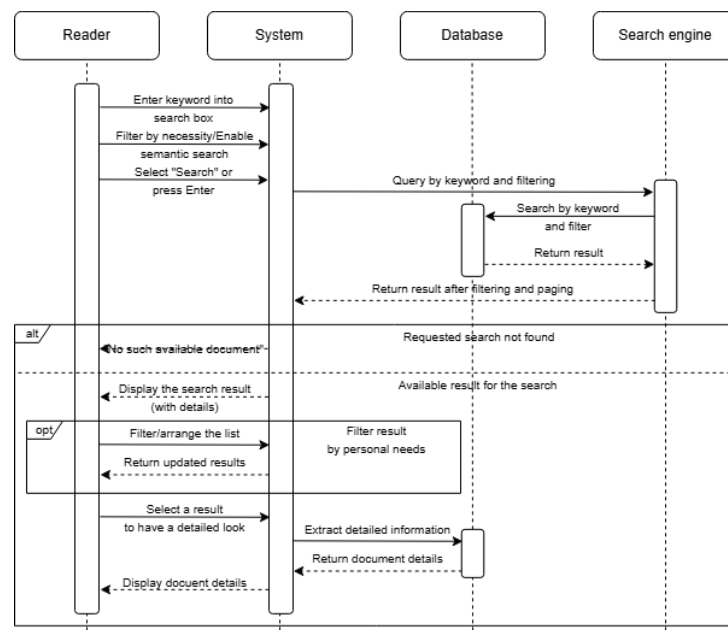


Figure 40: Sequence Diagram for Document Search

1. Reader enters search query.
2. Search engine queries catalog database and returns filtered, paginated results.
3. Renders item cards; clicking item retrieves detailed record view.

5.5.5 Add / Edit / Delete Document

5.5.5.a Add Document

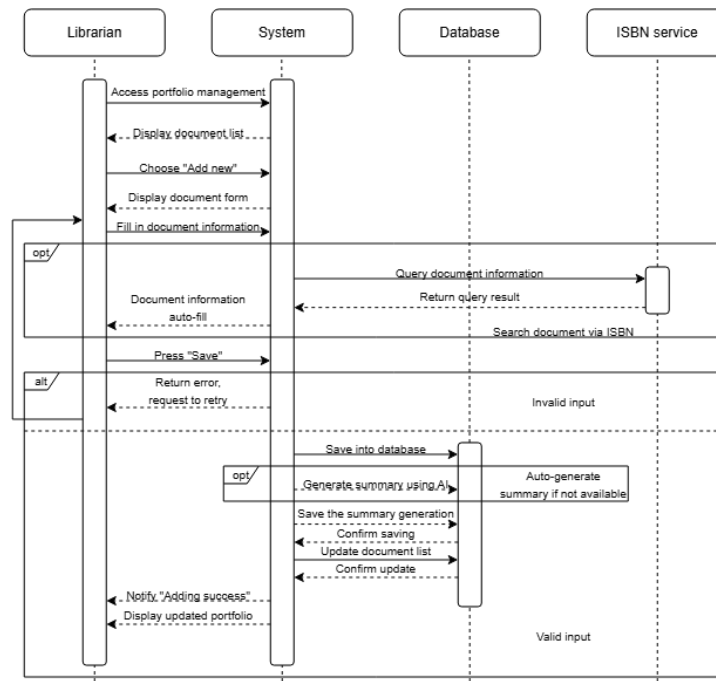


Figure 41: Sequence Diagram for Add Document

Librarian enters metadata manually or fetches via ISBN API (UC-CAT-03), optionally generating AI summary (UC-CAT-04), and saves record.

5.5.5.b Edit Document

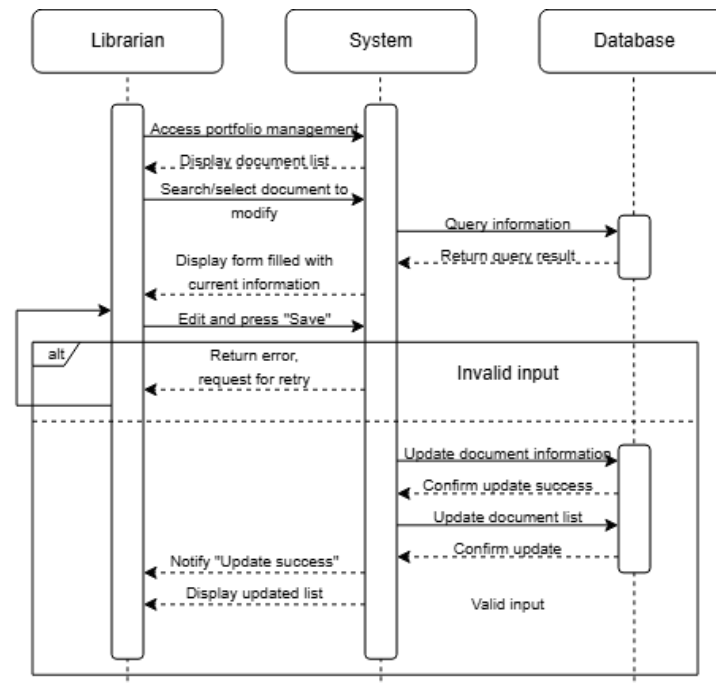


Figure 42: Sequence Diagram for Edit Document

Librarian edits pre-filled item fields, system validates inputs, and updates DB record.

5.5.5.c Delete Document

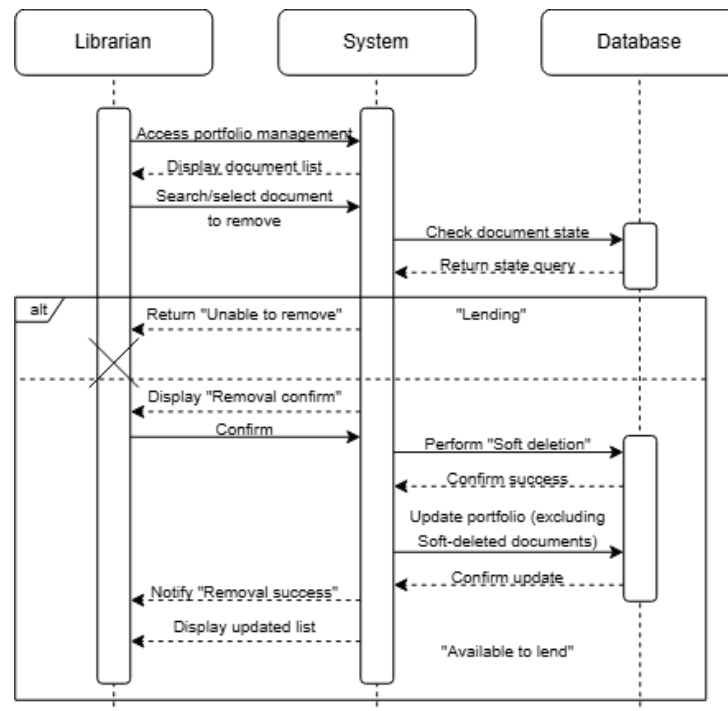


Figure 43: Sequence Diagram for Delete Document

System verifies item is not currently borrowed, librarian confirms deletion, and system performs soft delete.

5.5.6 Batch Document Import

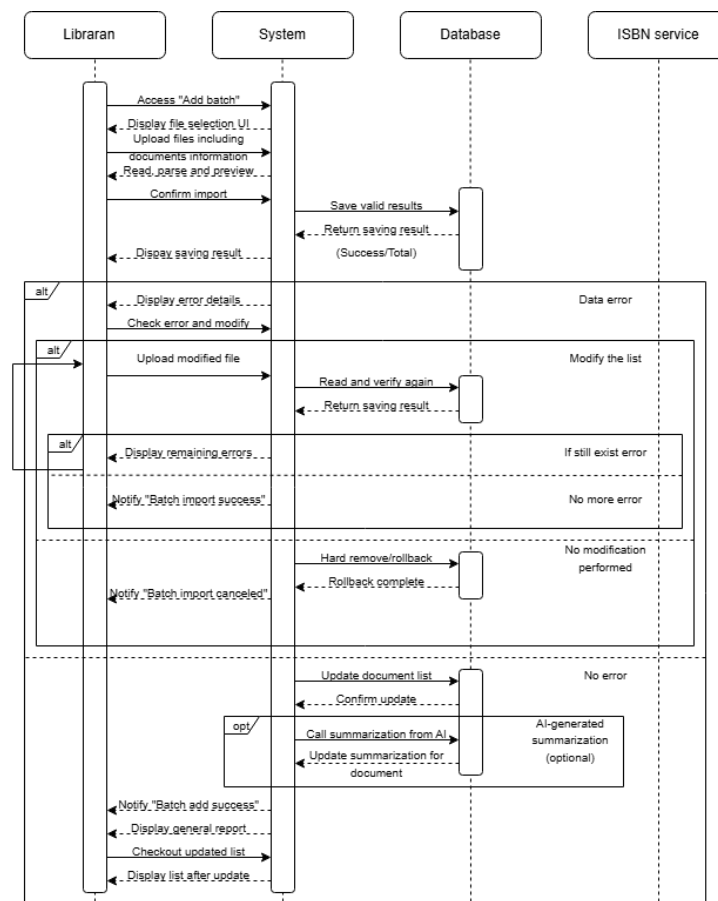


Figure 44: Sequence Diagram for Batch Document Import

Librarian uploads file; system validates rows, imports valid entries, and reports error summary.

5.5.7 Reserve Document

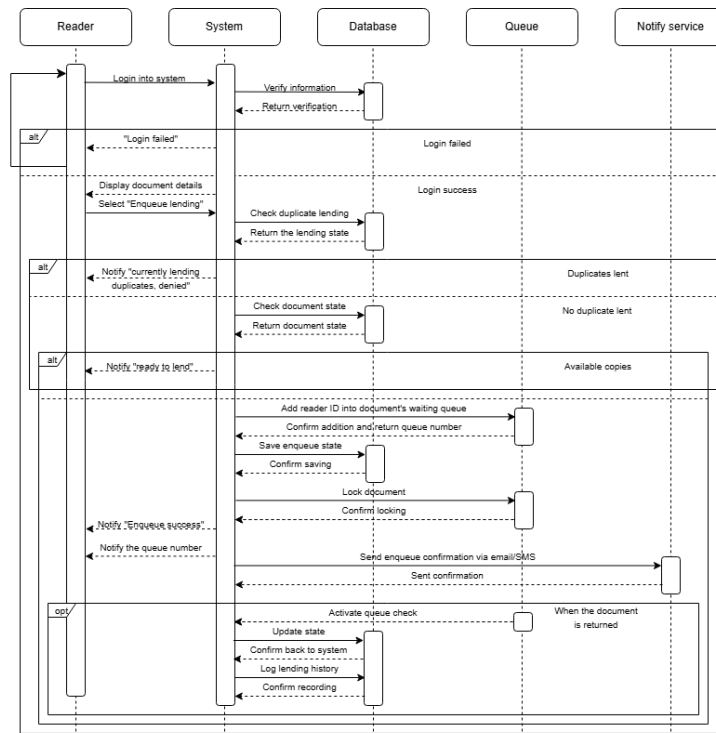


Figure 45: Sequence Diagram for Reserve Document

System checks for duplicate borrowings and copy availability; if zero available copies exist, appends user to queue and sends email confirmation.

5.5.8 Fine Calculation Update

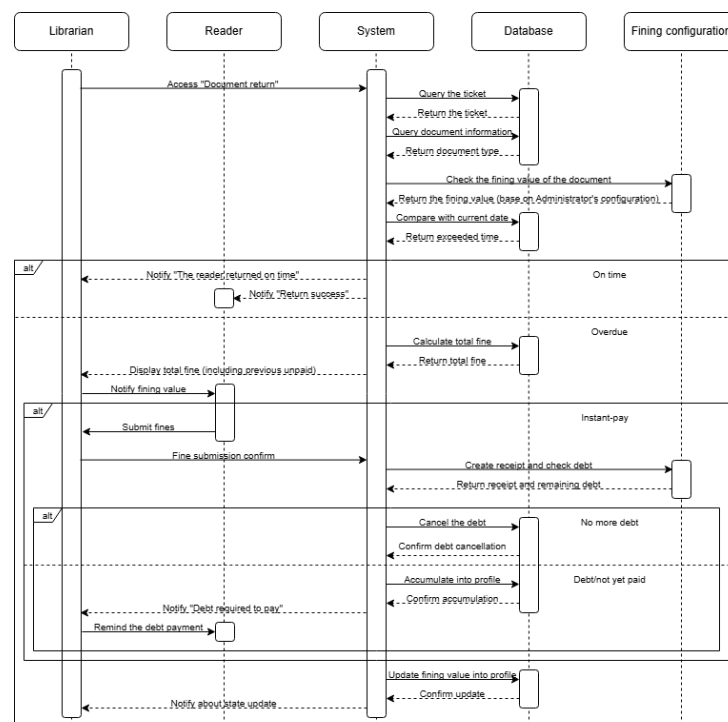


Figure 46: Sequence Diagram for Fine Calculation Update

System verifies borrower fine threshold, scans items, creates loan ticket, and decrements copy inventory.

5.5.9 Process Document Loan

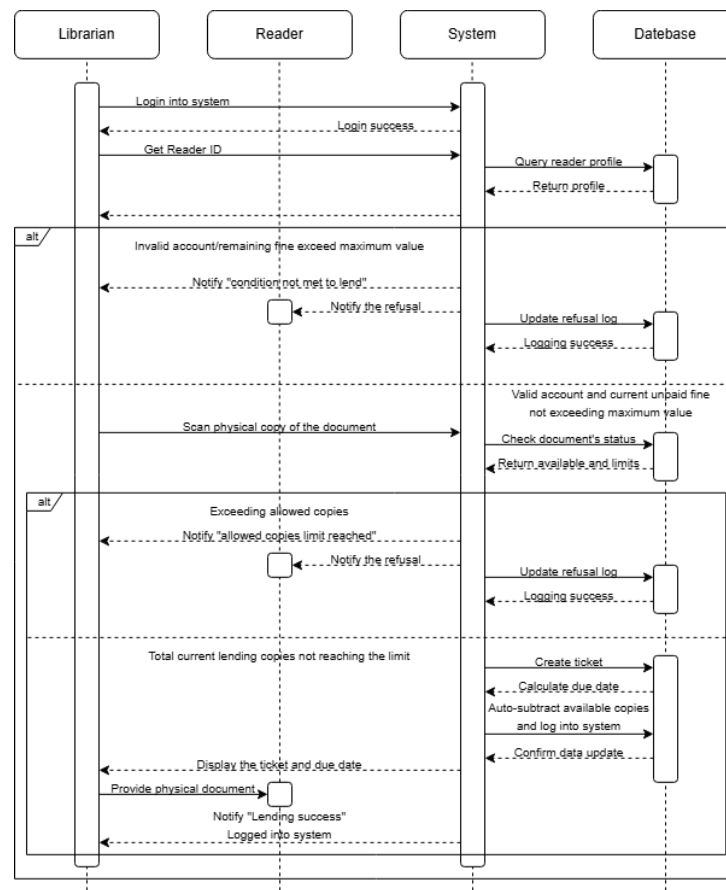


Figure 47: Sequence Diagram for Process Document Loan

Librarian validates reader eligibility, verifies item hold status, checks loan limits, and creates loan ticket.

5.5.10 Extend Document Loan

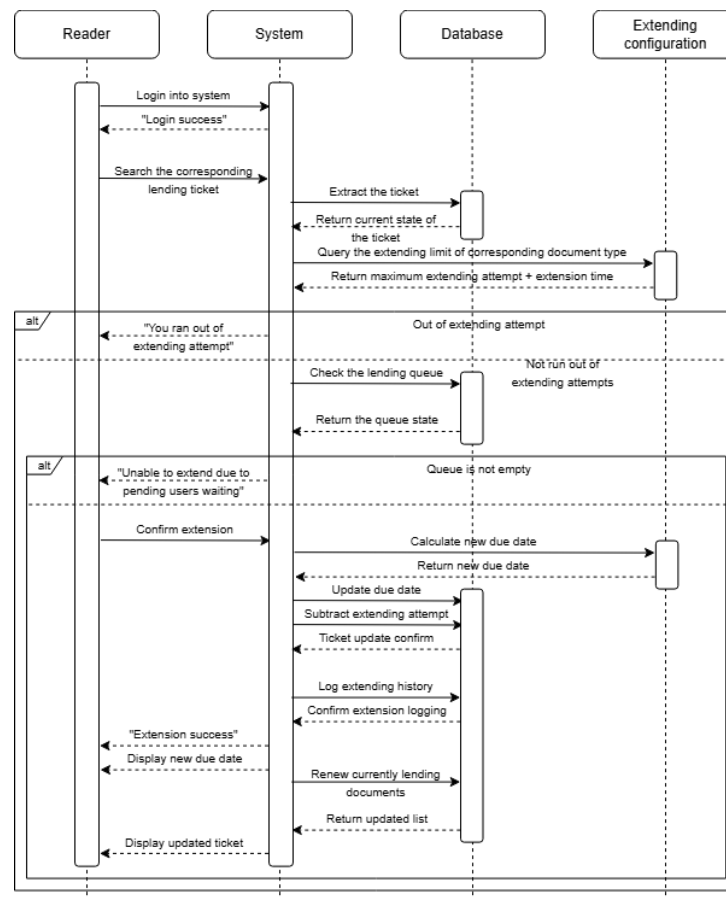


Figure 48: Sequence Diagram for Extend Document Loan

System checks renewal allowances and hold queue status; extends due date upon approval.

5.5.11 Return Document

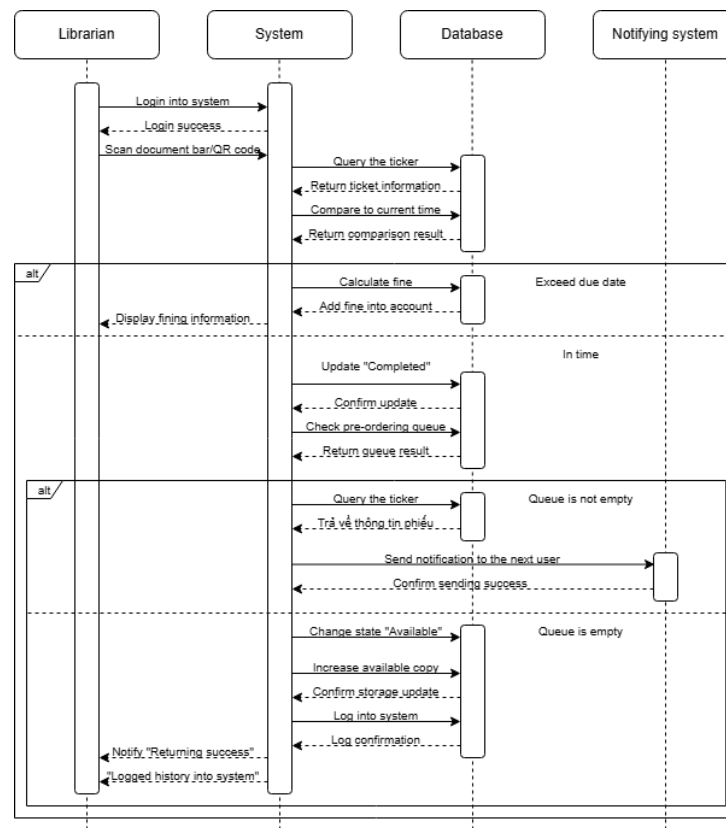


Figure 49: Sequence Diagram for Return Document

Librarian scans item, system assesses overdue fine if applicable, checks hold queue, updates copy status, and closes loan ticket.

6 Implementation

This section details the implementation of the BkLib Online Library Management System. It first presents the technology stack and architectural components powering the platform, followed by a module-by-module breakdown of both the user interface components and core technical implementations (backend services, business logic, and code snippets).

6.1 Technology Stack

6.1.1 Front-end: React.js and Vite

React.js handles the presentation layer of the online library, constructing interactive interfaces for book search, catalog browsing, and AI assistant chat interactions. Vite is employed as the build tool and dev server to provide fast startup, HMR support, and optimized production bundle compilation. On the client side, components handle rendering search results (incorporating keyword highlights and semantic matches), managing user state (caching, optimistic updates), and integrating the AI chat/assistant SDK (streaming responses, conversation state).



Vite + React

6.1.2 Back-end: Node.js and Express

Node.js + Express provides the core REST API for book catalog management, user accounts, and session administration. The server orchestrates request routing, authentication, RBAC authorization, PostgreSQL transactions via Prisma, and delegates AI workloads (embedding generation, summarization, RAG queries) to dedicated AI services. It exposes standardized endpoints for search, recommendations, and integration webhooks (e.g., metadata enrichment from Open Library).



6.1.3 Dedicated AI Microservice: Python and FastAPI

A decoupled microservice implemented in Python + FastAPI leverages NLP and Machine Learning frameworks (transformers, sentence-transformers, LLM provider client SDKs). Its primary responsibilities include embedding vector generation, document summarization, RAG-based question answering, and prompt engineering pipelines (chunking, reranking). FastAPI is ideal for low-latency concurrent endpoints and seamlessly scales based on workload demands.



6.1.4 ORM: Prisma

Prisma (TypeScript) is used to define database schemas, generate type-safe query clients, and manage PostgreSQL migrations. Prisma maintains end-to-end type safety between the Node application and the database layer, simplifying queries related to loans, transaction logs, and catalog metadata while providing ACID transaction support for data consistency.



6.1.5 Relational Database: PostgreSQL

PostgreSQL serves as the primary relational store for book records, user profiles, loan transactions, and system metadata. Key advantages include strict ACID compliance for loan-return workflows, advanced JSONB support for flexible metadata, full-text keyword indexing, and horizontal scalability via replication and partitioning.



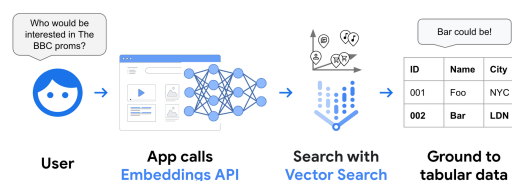
6.1.6 Vector Store and Embeddings: pgvector

Text embeddings for book contents (full texts, summaries) are stored in 'pgvector' (a vector extension for PostgreSQL) or a dedicated vector database. The vector store performs nearest-neighbor searches (cosine similarity) to power semantic search, contextual retrieval for RAG pipelines, and "similar books" recommendation features, mapping vector outputs directly to source document IDs.

6.1.7 AI / LLM Capabilities: Embeddings, RAG, Summarization, and Chatbot

The system integrates Large Language Models (LLMs) across multiple modules: embedding generation for semantic search, Retrieval-Augmented Generation (RAG) for Q&A based on library documents, automatic chapter/abstract summarization for previews, and interactive chatbot assistants. The architecture incorporates controlled prompt templates, embedding caching, and attribution mechanisms for source citation.

Grounding LLM outputs to business facts with embeddings and vector search



6.1.8 Hybrid Search Strategy

A hybrid search strategy combines: 1) full-text keyword search on PostgreSQL for exact metadata matching; and 2) vector similarity search via pgvector for semantic retrieval and contextual matching. Results are merged and reranked (based on relevance scores, popularity, and user context) before returning to the client interface.

6.1.9 Security and Authentication: JWT, RBAC, and OTP

The system utilizes JSON Web Tokens (JWT) for session management and refresh tokens for secure persistence. Role-Based Access Control (RBAC) enforces granular permissions (Admin, Librarian, Reader) verified at API gateways and endpoints. One-Time Passwords (OTP via email/SMS) secure multi-factor authentication for sensitive operations. Additional measures include HTTPS/TLS encryption, rate limiting, and comprehensive audit logging.

6.1.10 External Integrations: Open Library, Google Books, and Nodemailer

The Open Library and Google Books APIs automate metadata, cover image, and publishing data fetching via ISBN lookup. An automated ETL pipeline periodically syncs catalog data. 'Nodemailer' handles transactional notifications (loan due reminders, password resets, system alerts) with dynamic HTML templates and delivery logging.



6.2 Authentication Module

The Authentication Module serves as the primary gateway for system access across all user roles in the BkLib system. It enforces secure identity verification, role-based entry routing, self-service user onboarding, multi-step password recovery, and JSON Web Token (JWT) authorization middleware.

6.2.1 User Interface Implementation

6.2.1.1 Login Page Primary entry portal providing unified authentication with role-switching tabs.

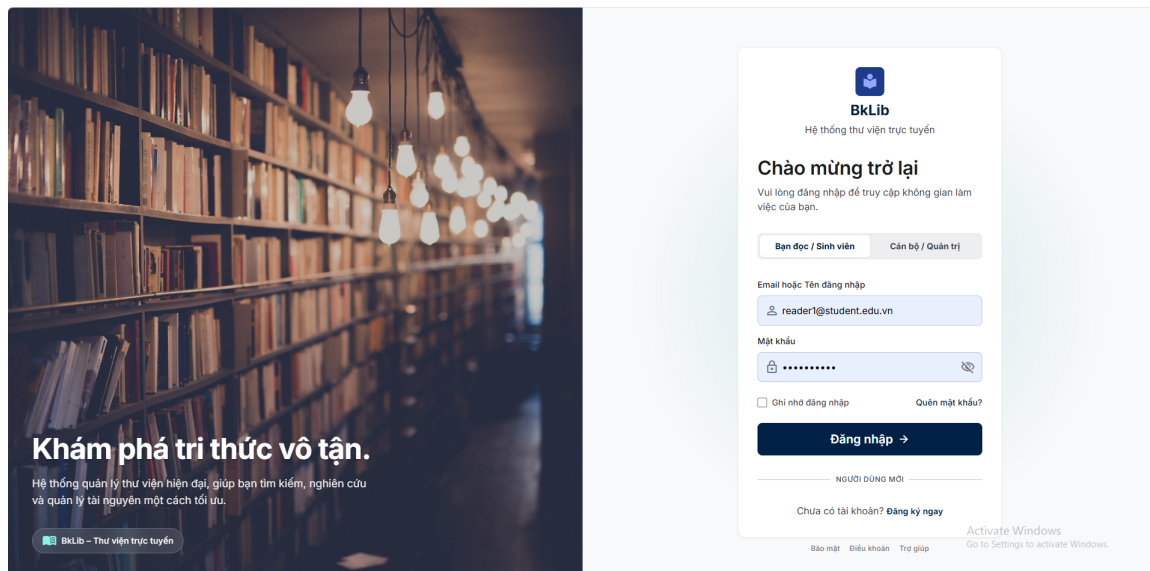


Figure 50: BkLib Login Interface

Layout & Functionality: Features a split-screen layout with branding artwork on the left and a clean authentication card on the right. Includes a role selector tab ("Bạn đọc / Sinh viên" vs. "Cán bộ / Quản trị"), credentials input fields (email or username, password with visibility toggle), a "Ghi nhớ đăng nhập" (Remember Me) checkbox, and direct triggers for password recovery ("Quên mật khẩu?") and account registration. Upon validation, users are automatically routed to their designated workspace.

6.2.1.2 Registration Page Self-service onboarding page for new library members to register accounts with branch selection.

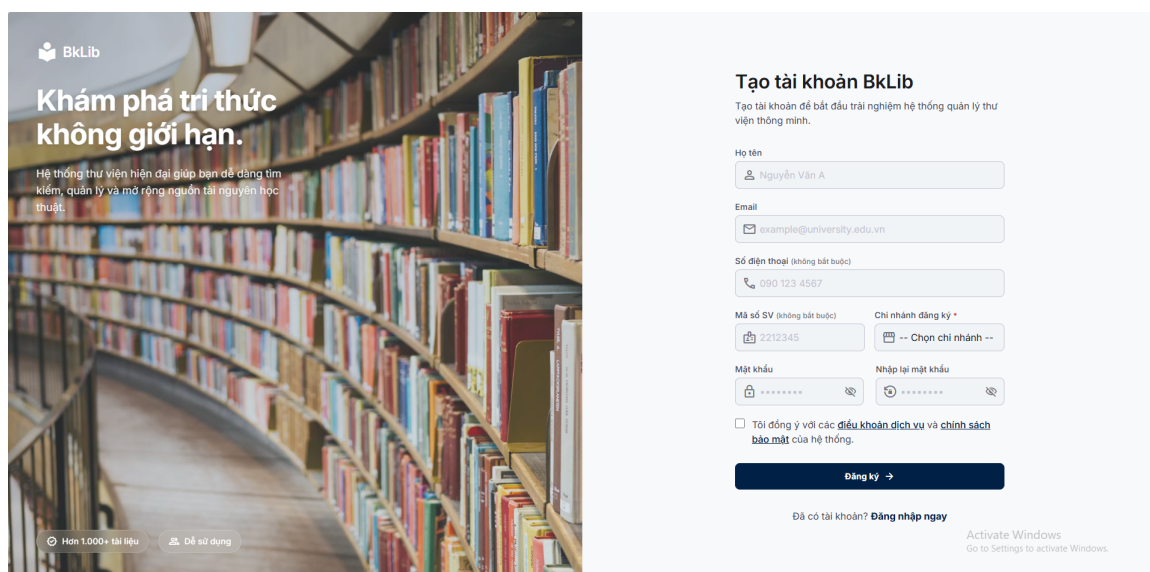
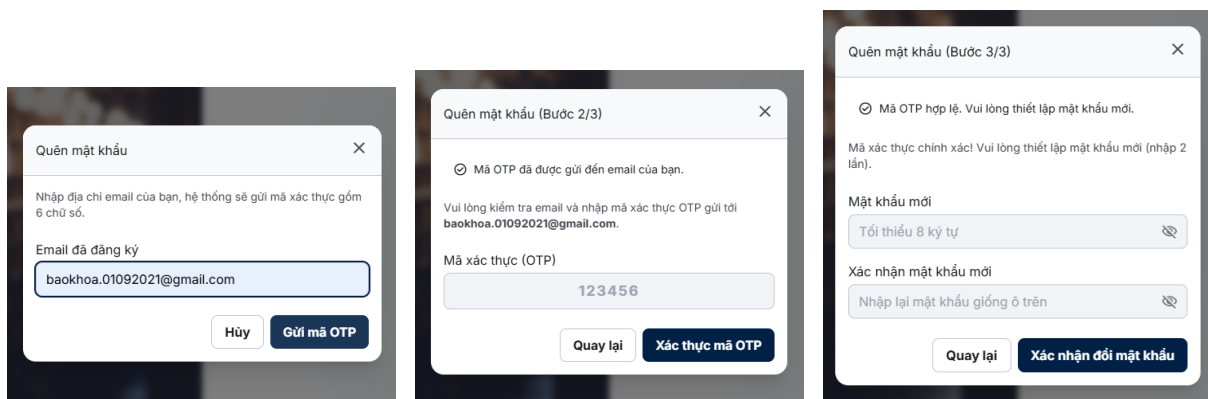


Figure 51: BkLib Account Registration Interface

Layout & Functionality: Structured form capturing essential user profile details, including Full Name, Email, Phone Number (optional), Student ID ("Mã số SV" - optional), and Branch Selection ("Chi nhánh đăng ký"). It mandates password creation with confirmation matching and terms of service consent ("Điều khoản dịch vụ" and "Chính sách bảo mật") prior to submitting the registration request.

6.2.1.3 Password Reset Workflow (3-Step Modal) Modal-based self-service credential recovery mechanism utilizing 6-digit Email OTP verification.



(a) Step 1: Email Submission

(b) Step 2: OTP Verification

(c) Step 3: Password Reset

Figure 52: Three-step Modal Workflow for Password Recovery

Layout & Functionality: Executes within an overlay modal dialog spanning three sequential stages:

- **Step 1 (Request OTP):** Users enter their registered email address to dispatch a 6-digit verification code.
- **Step 2 (OTP Verification):** Prompts input of the 6-digit code received via email, featuring validation feedback and back-navigation controls.
- **Step 3 (Password Update):** Prompts creation and confirmation of a new password (minimum 8 characters) with eye icons for masked text toggling.

6.2.2 Technical Implementation Highlights

Security and endpoint protection are enforced through middleware leveraging JSON Web Tokens (JWT) and a higher-order role authorization function.

```

1 // Middleware verifying JWT access token from Authorization header
2 export const authenticate = (req: Request, res: Response, next: NextFunction): void => {
3   const authHeader = req.headers.authorization;
4   if (!authHeader || !authHeader.startsWith('Bearer ')) {
5     res.status(401).json({ success: false, message: 'No authorization token provided' });
6     return;
7   }
8   const token = authHeader.split(' ')[1];
9   try {
10    const decoded = jwt.verify(token, env.JWT_SECRET) as AuthPayload;
11    req.user = decoded;
12    next();
13  } catch (err) {
14    res.status(401).json({ success: false, message: 'Invalid or expired token' });
15  }
16 };
17
18 // Middleware factory restricting access to specified roles
19 export const authorize = (...roles: Role[]) => {
20   return (req: Request, res: Response, next: NextFunction): void => {
21     if (!req.user || !roles.includes(req.user.role)) {
22       res.status(403).json({ success: false, message: 'Access denied' });
23       return;
24     }
25     next();
26   };
27 };

```

Listing 1: JWT Authentication and Authorization Middleware (backend-express/src/middlewares/auth.middleware.ts)

6.3 Reader Module

The Reader Module provides self-service capabilities for library patrons. It enables authenticated readers to browse the catalog, execute semantic AI-powered resource searches, track and renew active loans, monitor hold queue statuses, view historical borrowing transactions, and interact with a virtual assistant widget.

6.3.1 User Interface Implementation

6.3.1.1 Homepage Personalized central dashboard rendered upon reader authentication, serving as the operational hub for patron activities.

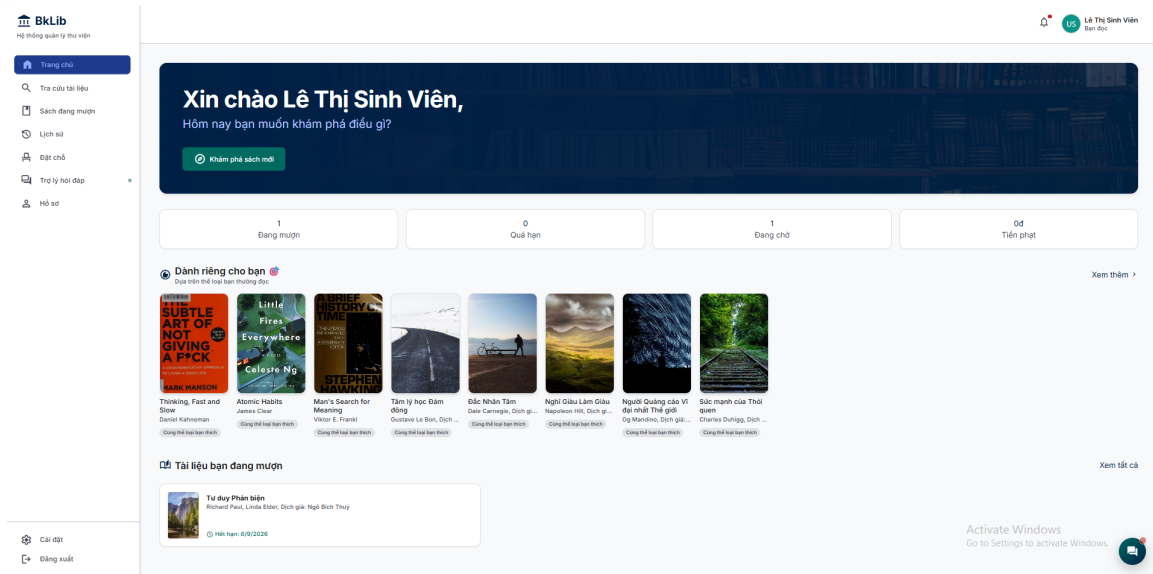


Figure 53: Reader Module Homepage Interface

Layout & Functionality: Includes a responsive left navigation sidebar, top notification center, key metric summary cards (active loans, overdue items, active holds, fine balances), and quick-access cards for currently borrowed books with real-time due date indicators.

6.3.1.2 Catalog Search Page Interface for discovery, multi-criteria filtering, and digital resource catalog browsing.

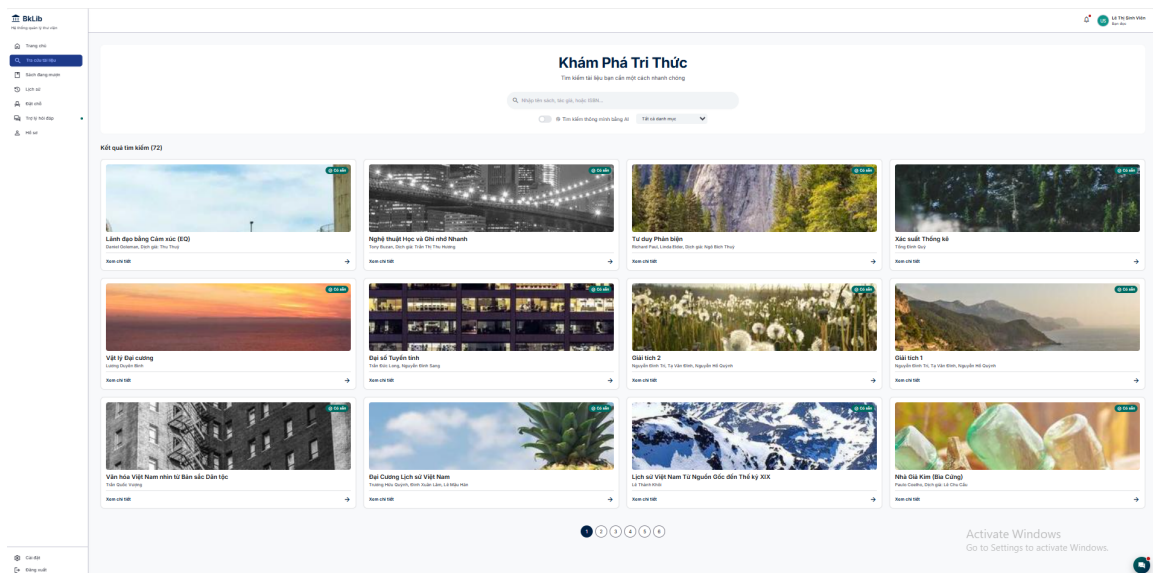


Figure 54: Catalog Search Interface

Layout & Functionality: Integrates an advanced keyword search bar with category and branch availability filters. Readers can view cover imagery, metadata (author, publisher, ISBN), real-time availability status, and initiate physical hold reservations directly from search result cards.

6.3.1.3 Active Loans Management Page Dedicated view allowing readers to monitor loan lifecycles and trigger online loan renewals.

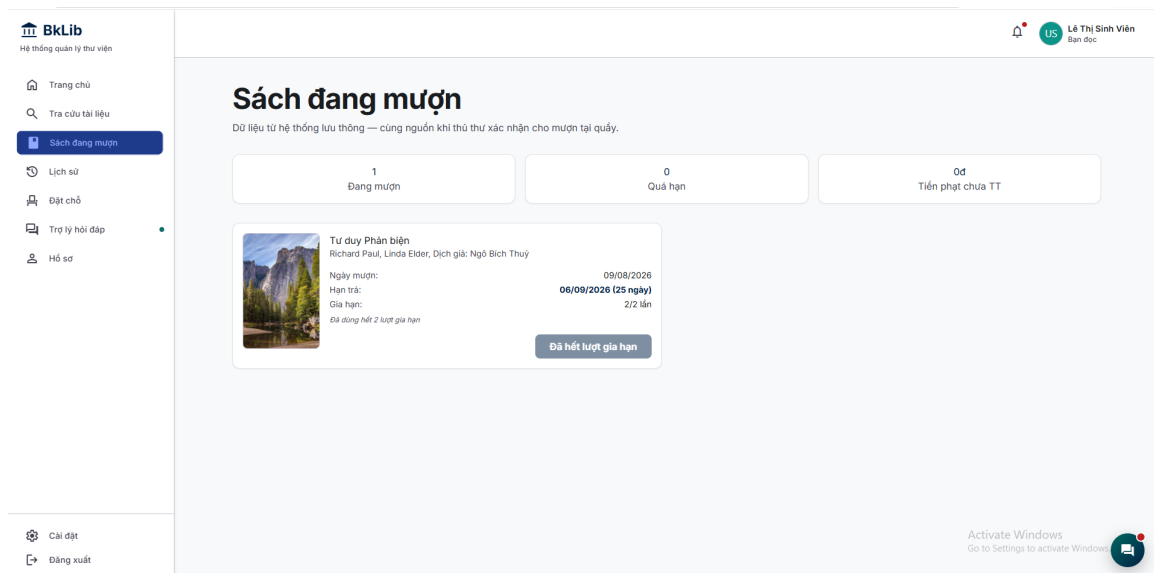


Figure 55: Active Loans Management Interface

Layout & Functionality: Displays loan summary statistics alongside detailed item cards indicating checkout dates, mandatory due dates, and renewal attempt limits. Readers can execute inline renewals, which dynamically query circulation rules to recalculate due dates or flag renewal constraints (e.g., existing holds or maximum extension limits).

6.3.1.4 Loan History Page Historical ledger recording past transactions, returned items, and financial interactions.

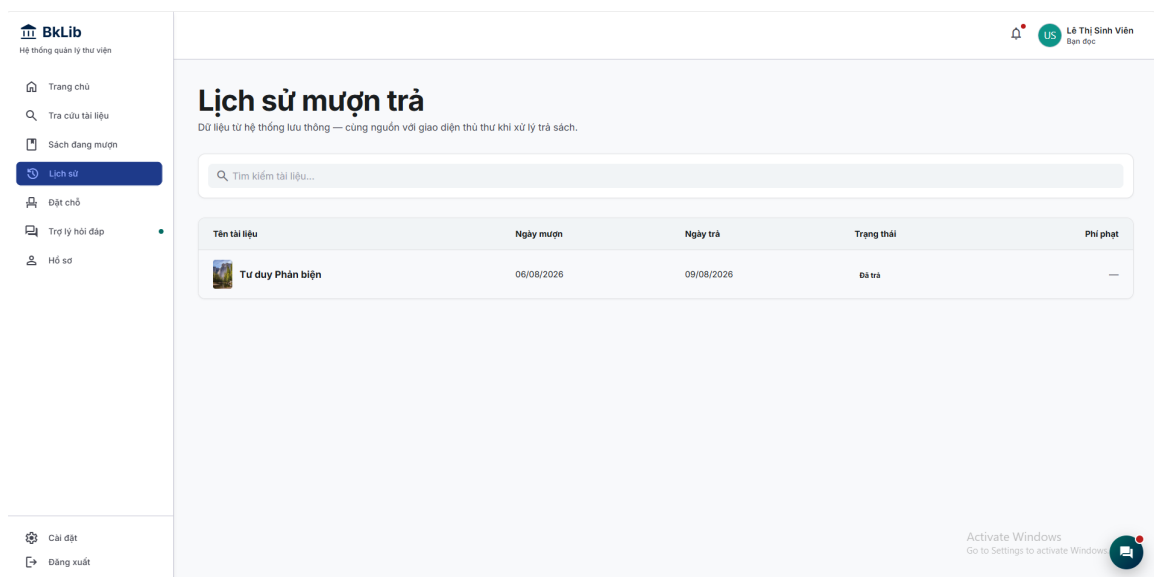


Figure 56: Loan History Interface

Layout & Functionality: Structured table containing book titles, checkout dates, actual return timestamps, fulfillment status badges, and assessed fine records. Features client-side column sorting and date-range filter triggers for personal record auditing.

6.3.1.5 Active Holds Page Monitors reservation queue positions and availability notices for out-of-stock items.

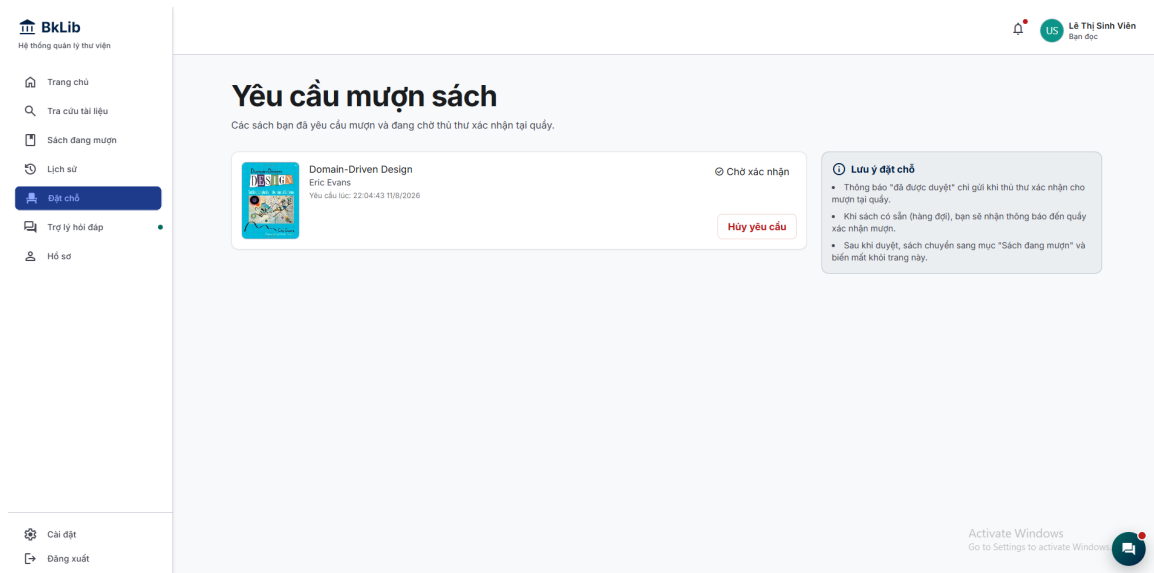


Figure 57: Active Holds Interface

Layout & Functionality: Lists active reservations with queue ordering, expiration windows for ready-for-pickup notifications, cancel hold triggers, and an informational panel detailing library reservation rules and pickup deadlines.

6.3.1.6 Virtual Assistant Chat Window Interactive AI assistant widget facilitating natural language interaction, QA, and semantic recommendation.

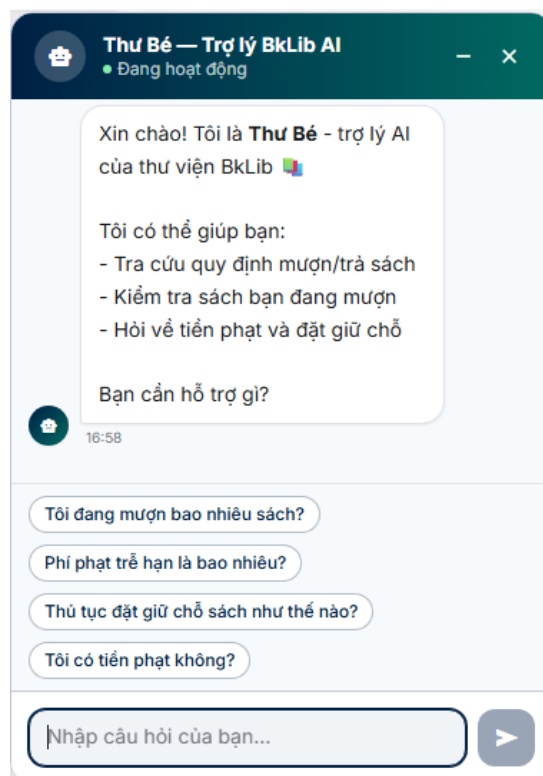


Figure 58: Virtual Assistant Chat Window Interface

Layout & Functionality: Floating chat interface utilizing a hybrid Retrieval-Augmented Generation (RAG) backend. Features prompt suggestions, streaming response bubble threads, and direct deep-links to catalog book details generated from semantic similarity queries.

6.3.1.7 Personal Profile & Settings Page Patron account management, security credentials, and communication preferences.

Figure 59: Personal Profile and Settings Interface

Layout & Functionality: Contains editable personal contact forms, password modification tools, multi-channel notification toggles (Email, SMS, In-App), and a personalized recommendation grid driven by patron reading history.

6.3.2 Technical Implementation Highlights

The Virtual Assistant utilizes vector embeddings generated via SentenceTransformer and stored in an in-memory ChromaDB collection to retrieve top context chunks for patron inquiries.

```

1 def retrieve_context(query: str, top_k: int = 3) -> str:
2     """
3     Retrieve top-k relevant text chunks for a query from ChromaDB vector collection.
4     Formats results with segment headers to feed into LLM prompt context.
5     """
6     if _collection is None or _collection.count() == 0:
7         return ""
8
9     embedder = get_embeddings()
10    query_embedding = embedder.encode([query]).tolist()
11
12    results = _collection.query(
13        query_embeddings=query_embedding,
14        n_results=min(top_k, _collection.count()),
15    )
16    docs = results.get("documents", [[]])[0]
17    if not docs:
18        return ""
19
20    return "\n\n---\n\n".join([f"[Chunk {i+1}]: {doc}" for i, doc in enumerate(docs)])

```

Listing 2: ChromaDB Vector Retrieval for RAG (ai-service/app/services/rag_service.py)

6.4 Librarian Module

The Librarian Module empowers library staff to manage daily counter operations, oversee item circulation, maintain catalog metadata, calculate fine penalties, communicate system announcements, and analyze branch performance analytics.

6.4.1 User Interface Implementation

6.4.1.1 Homepage Operational control panel highlighting daily tasks, branch status indicators, and urgent action queues.

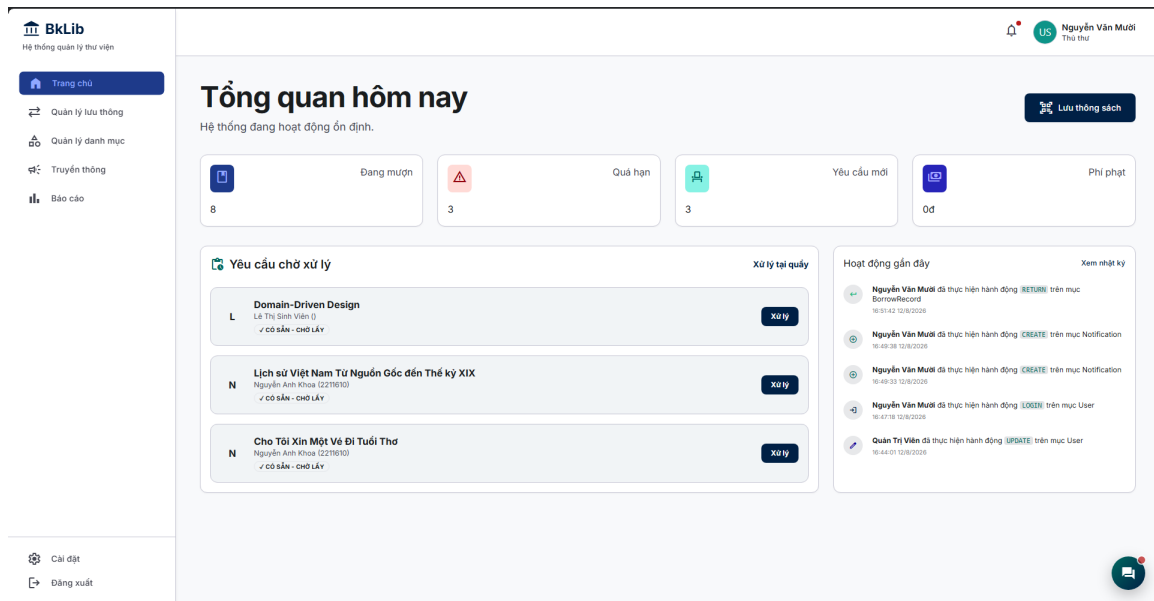


Figure 60: Librarian Module Homepage Interface

Layout & Functionality: Features quick-action navigation shortcuts (Circulation, Cataloging, Announcements, Reports), real-time counter performance metrics, pending reservation queues awaiting processing, and an activity log feed tracking recent loan activities.

6.4.1.2 Circulation Management Page Counter processing interface engineered for high-throughput check-outs, returns, and fine settlement.

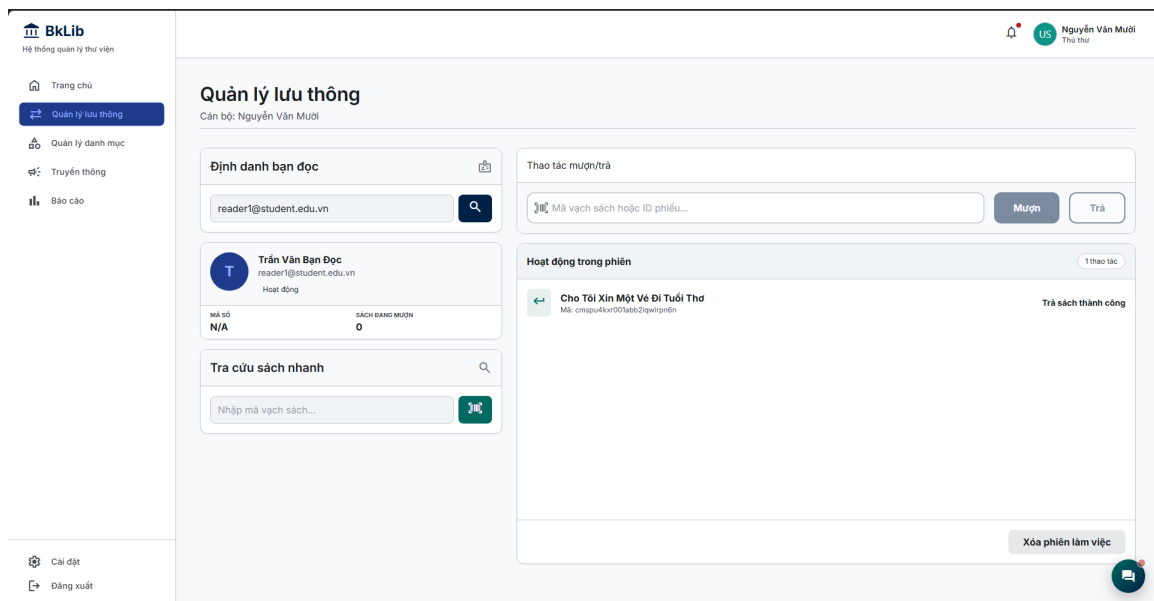


Figure 61: Circulation Management Interface

Layout & Functionality: Dual-pane interface: Patron ID/barcode lookup pane on the left, and rapid barcode scanning input area with active transaction summary and item return validation on the right. Instantly verifies patron account status, overdue holds, and automatic fine calculation upon scanning.

6.4.1.3 Catalog Management Page Administrative workspace for book metadata management, inventory tracking, and physical copy labeling.

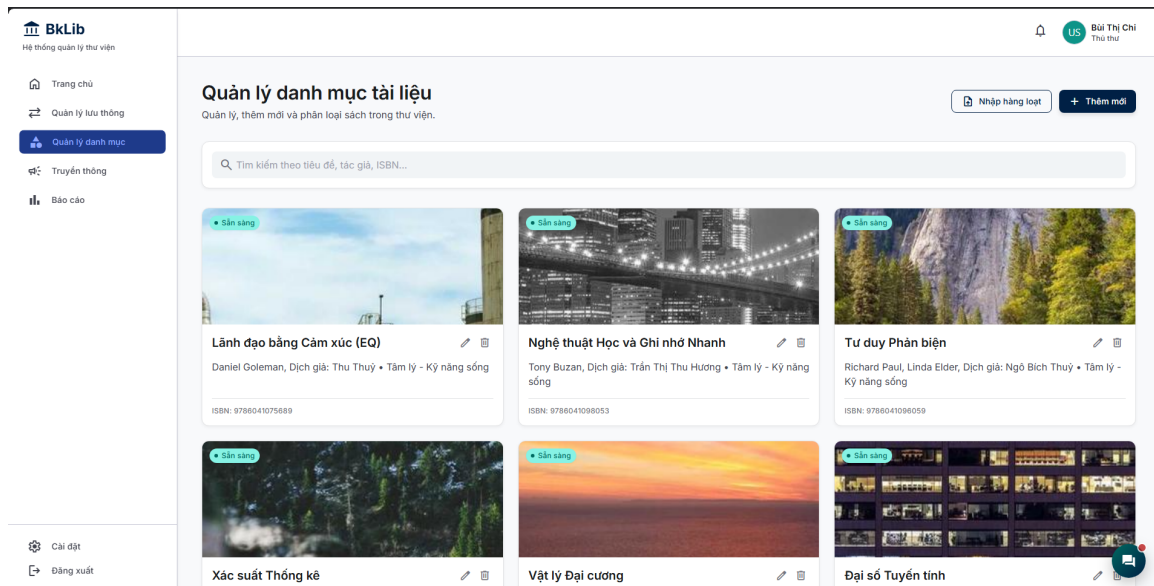


Figure 62: Catalog Management Interface

Layout & Functionality: Search control bar with ISBN auto-fetching integration, batch CSV/Excel import tools, item modal creation form, and a paginated inventory table providing inline record editing, status toggling, and copy barcode generation.

6.4.1.4 Communication Management Page Broadcast dispatch portal for library announcements and target reader notification campaigns.

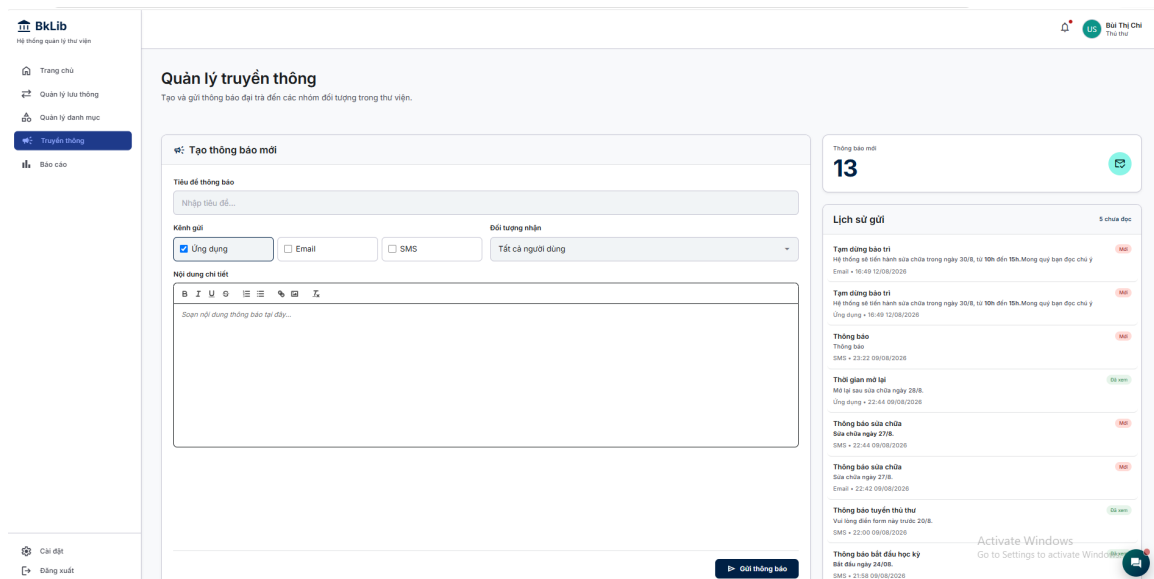


Figure 63: Communication Management Interface

Layout & Functionality: Rich-text announcement composition editor with audience targeting options (All Readers, Overdue Borrowers, Specific Member Tiers), channel selection (In-App, Email), and a historical broadcast log detailing delivery success rates.

6.4.1.5 Reports & Analytics Page Data visualization dashboard tracking circulation throughput, collection usage, and financial metrics.

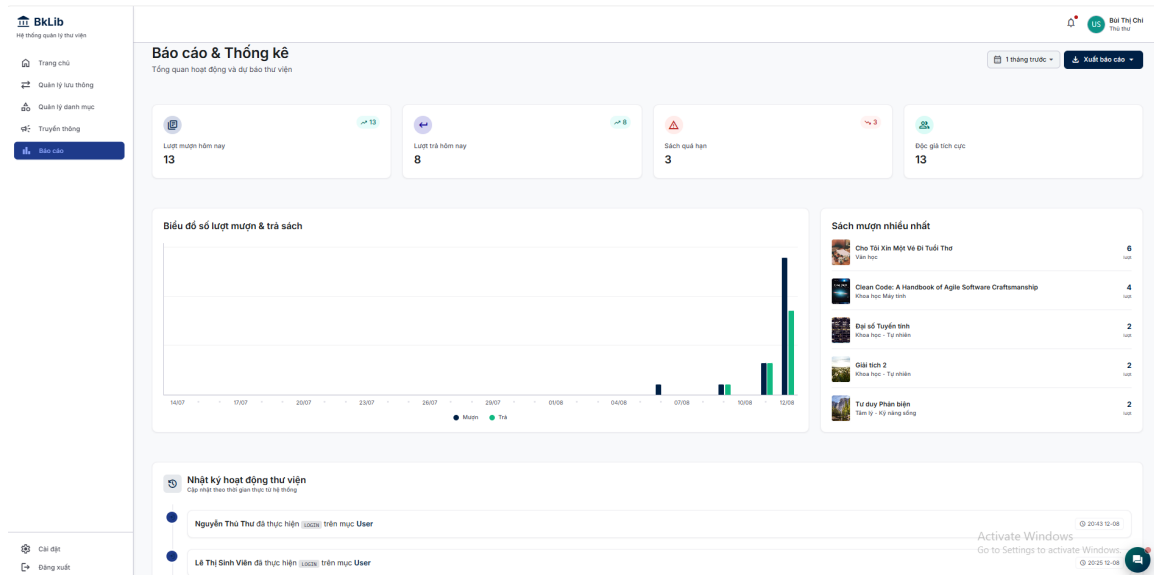


Figure 64: Librarian Reports and Analytics Interface

Layout & Functionality: Interactive date-range controls, export triggers (PDF, Excel, CSV), circulation trend charts, top-borrowed book rankings, and fine collection breakdowns to assist inventory planning.

6.4.1.6 Personal Profile & Settings Page Staff account management workspace for updating staff profile attributes, security credentials, and system notification preferences.

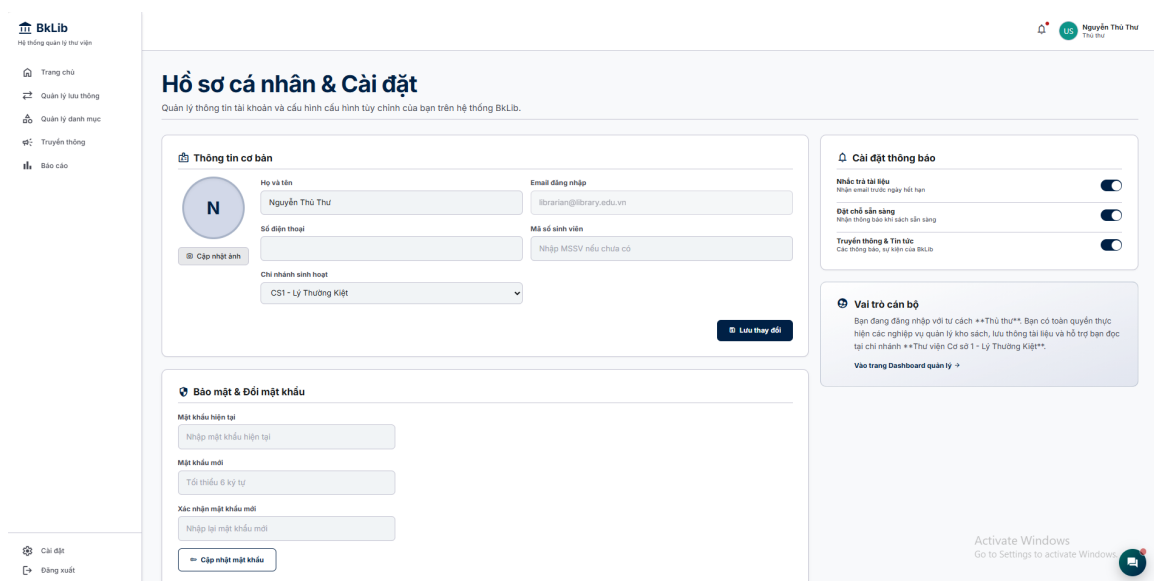


Figure 65: Librarian Personal Profile and Settings Interface

Layout & Functionality: Organized into a multi-card dashboard layout:

- **Basic Information Pane ("Thông tin cơ bản"):** Editable profile fields including full name, login email, contact phone number, student/staff ID, assigned operating branch selector (e.g., CS1 - Lý Thường Kiệt), and profile avatar update controls.
- **Security & Password Modification ("Bảo mật & Đổi mật khẩu"):** Password management utility requiring current password verification followed by new password entry and confirmation matching.
- **Notification Preferences ("Cài đặt thông báo"):** Granular toggle switches for key automated alerts, including document return reminders, hold availability notifications, and system-wide announcements.
- **Staff Role Summary Panel ("Vai trò cán bộ"):** Informational context card detailing current authorization privileges, assigned operational branch scope, and a direct navigation link to the administrative dashboard.

6.4.2 Technical Implementation Highlights

Circulation logic dynamically calculates overdue days by iterating through loan return intervals while excluding official library holiday dates configured in system settings.

```

1 const calcOverdueDays = async (dueDate: Date, returnDate: Date): Promise<number> => {
2   if (returnDate <= dueDate) return 0;
3
4   // Retrieve holiday array from system configuration
5   const holidayConfig = await getConfig(CONFIG_KEYS.HOLIDAY_DATES, '[]');
6   const holidays: string[] = JSON.parse(holidayConfig);
7
8   let overdueDays = 0;
9   const cursor = new Date(dueDate);
10  cursor.setDate(cursor.getDate() + 1);
11
12  // Iterate day-by-day excluding scheduled holidays
13  while (cursor <= returnDate) {
14    const dateStr = cursor.toISOString().split('T')[0];
15    if (!holidays.includes(dateStr)) {
16      overdueDays++;
17    }
18    cursor.setDate(cursor.getDate() + 1);
19  }
20  return overdueDays;
21 };

```

Listing 3: Holiday-Aware Overdue Day Calculation
(backend-express/src/modules/circulation/circulation.service.ts)

6.5 Administrator Module

The Administrator Module provides top-level governance for platform management. It enables system administrators to manage Role-Based Access Control (RBAC), configure system settings, manage branch networks, audit system logs, monitor infrastructure health, and execute database backup and disaster recovery operations.

6.5.1 User Interface Implementation

6.5.1.1 System Overview Dashboard High-level system health dashboard monitoring infrastructure resources, server loads, and usage statistics.

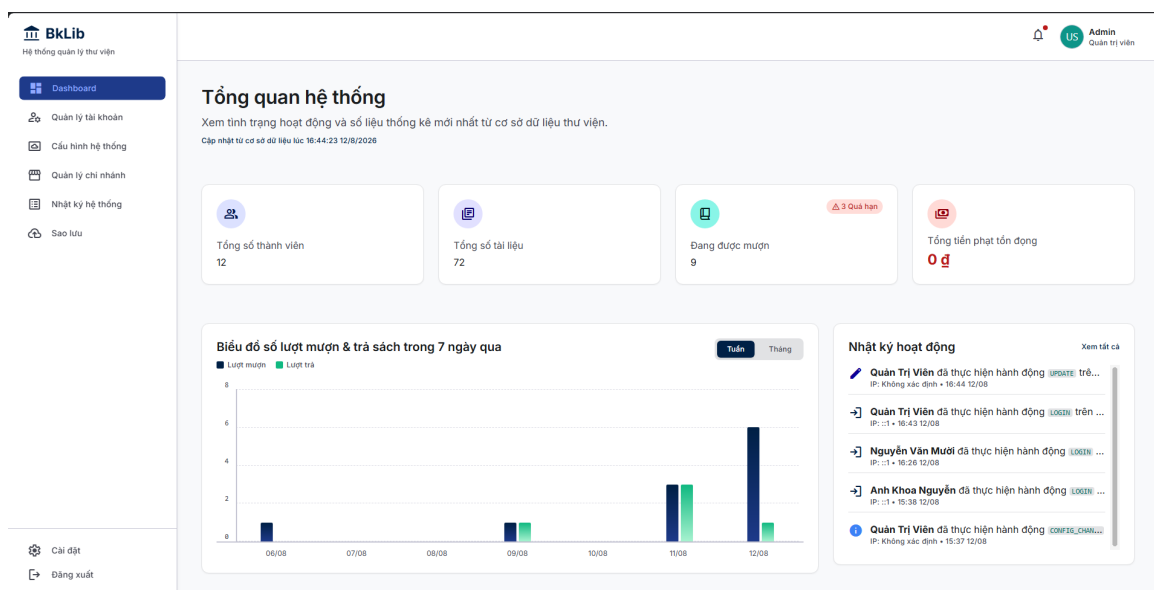


Figure 66: Administrator System Overview Dashboard

Layout & Functionality: Displays system metrics cards (Total Active Accounts, Catalog Size, Active Sessions), cloud database storage indicators, API traffic throughput graphs, and real-time system security alerts.

6.5.1.2 User Account Management Page Centralized administration workspace for user identity lifecycle, role assignments, and permission management.

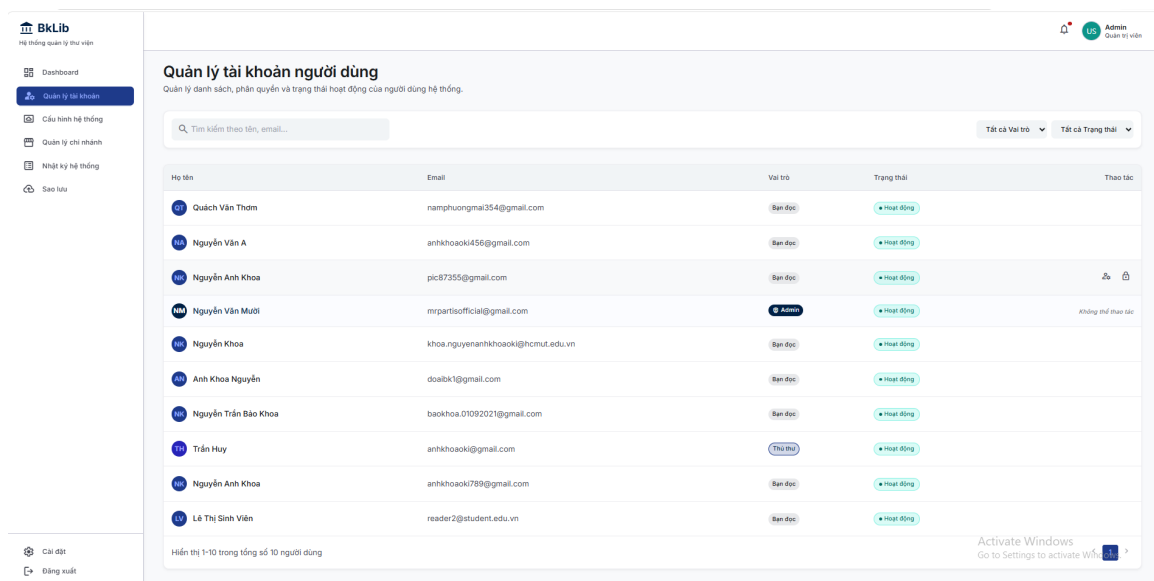


Figure 67: User Account Management Interface

Layout & Functionality: Paginated user directory with role-based filtering (Reader, Librarian, Admin), account creation modal trigger, RBAC permission assignment controls, and emergency account lock/unlock toggles.

6.5.1.3 System Configuration Page Centralized administrative interface for defining operational system parameters, circulation rules, and fine structures.

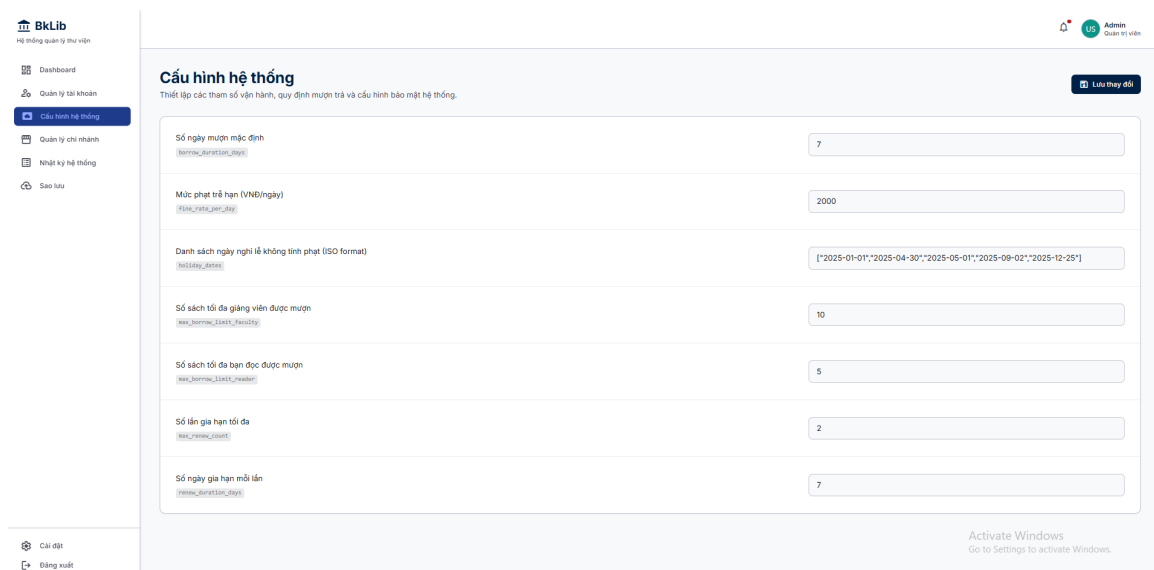


Figure 68: System Configuration Interface

Layout & Functionality: Structured configuration panel providing interactive inputs for global system environment variables, including:

- **Circulation Rules:** Default borrowing duration in days (`borrow_duration_days`), max borrowing limit for faculty members (`max_borrow_limit_faculty`), and general reader borrowing cap (`max_borrow_limit_reader`).
- **Renewal Policies:** Maximum allowable renewals per item (`max_renew_count`) and duration granted per renewal cycle (`renew_duration_days`).
- **Financial & Calendar Settings:** Daily overdue fine rate calculation in VNĐ (`fine_rate_per_day`) and an array-formatted holiday exclusion list (`holiday_dates`) in ISO format to suspend fine accruals during non-working days.

6.5.1.4 Branch Management Page

Network topology configuration page for physical library branches and inter-branch policies.

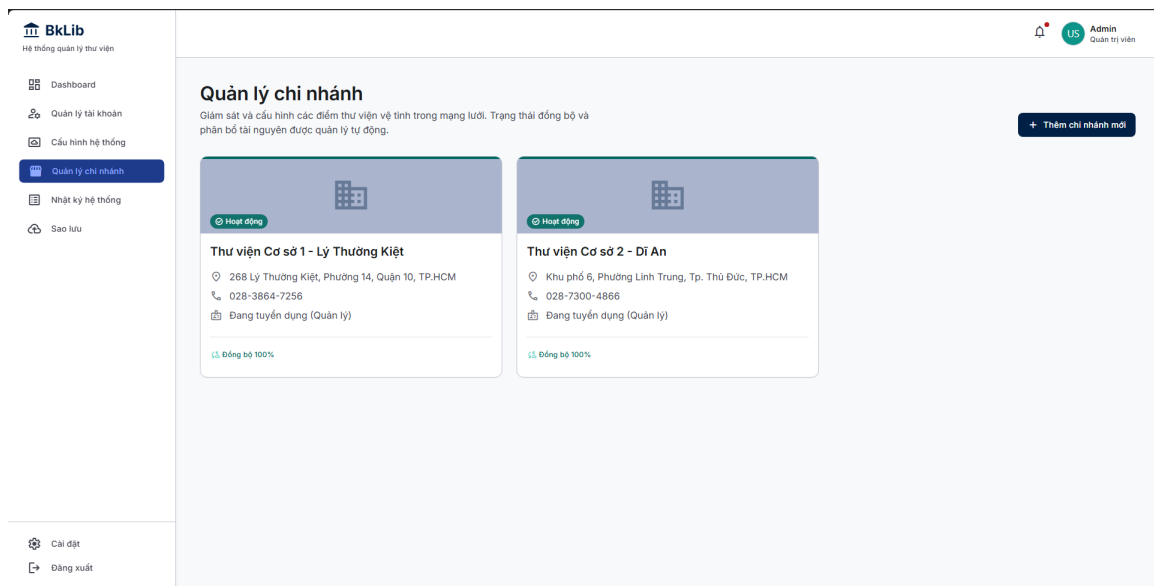


Figure 69: Branch Management Interface

Layout & Functionality: Grid of branch profile cards displaying branch addresses, contact details, assigned head librarians, operating hours, and synchronization status with the central database.

6.5.1.5 Audit Logs Page

Compliance and security audit trail viewer logging critical system actions and data state mutations.

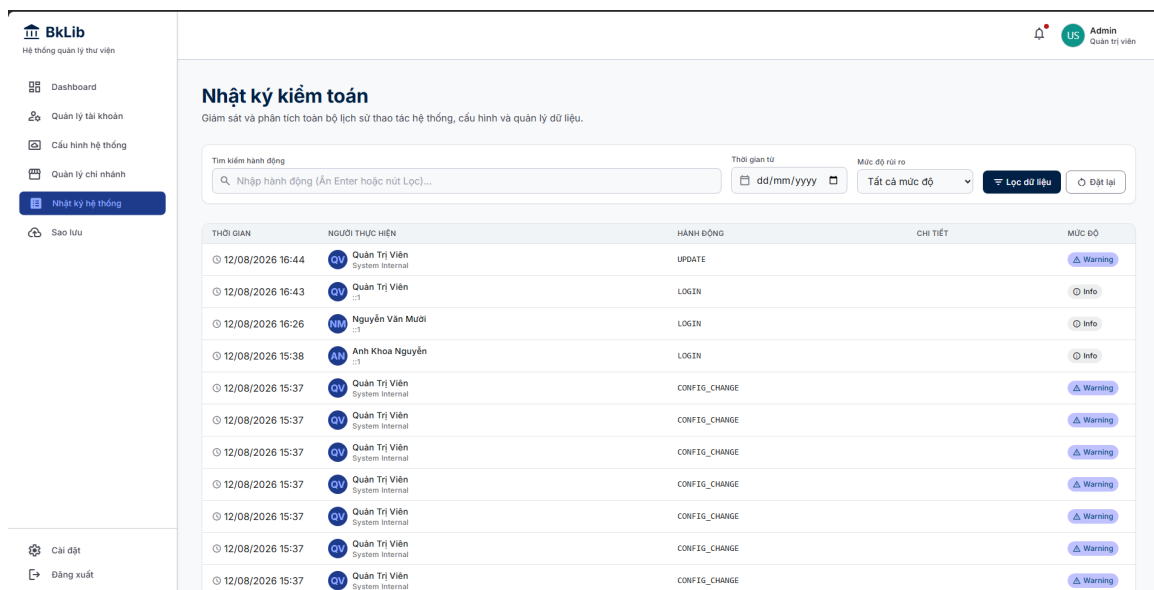


Figure 70: System Audit Logs Interface

Layout & Functionality: High-density audit event table detailing action timestamps, actor identities and IP addresses, API endpoint paths, before/after JSON payload diffs, and severity badges for anomaly tracking.

6.5.1.6 Backup & Disaster Recovery Page

Database snapshot management and disaster recovery administration tool.

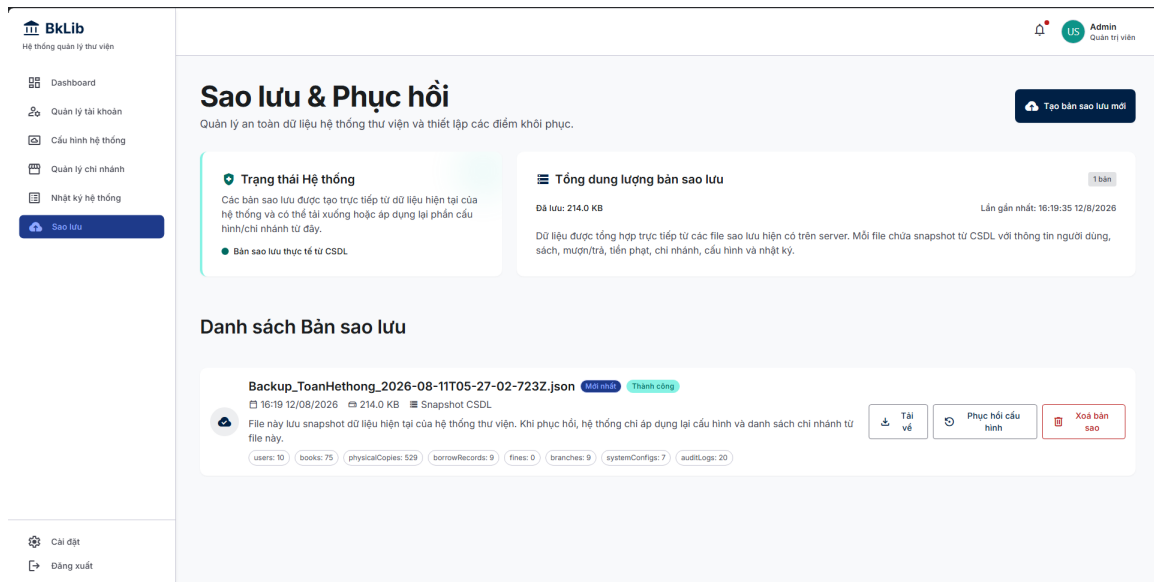


Figure 71: Backup and Disaster Recovery Interface

Layout & Functionality: Displays automated backup scheduling configurations, snapshot storage usage, manual immediate backup trigger controls, historical snapshot logs, and point-in-time database restoration controls.

6.5.2 Technical Implementation Highlights

System administrative mutations (such as system configuration adjustments, database backups, or account updates) automatically log immutable audit trail records containing actor metadata and action parameters.

```

1 await prisma.auditLog.create({
2   data: {
3     actorId: processedById,
4     action: AuditAction.SYSTEM_CONFIG_UPDATE,
5     targetResource: 'SystemConfig',
6     details: { key, oldValue, newValue },
7     ipAddress,
8   },
9 });

```

Listing 4: Audit Logging Execution (backend-express/src/modules/admin/admin.service.ts)

7 System Testing

7.1 Test Objectives and Environment

Testing was conducted to evaluate the correctness of the library management functions, the coordination between system layers, main business workflows, and requirements regarding security and error handling. The scope focused on the Express/TypeScript backend, including modules for authentication, users, books, branches, document circulation, and AI features.

Tools Used

- Jest combined with ts-jest to execute TypeScript test cases.
- Supertest to send HTTP requests to the Express application during integration and system testing.
- Mock Prisma Client and mocked external services for isolated testing, removing dependencies on live databases or email services.
- TypeScript Compiler with the `--noEmit` flag alongside Jest to verify source code quality.

Test source code is organized within the `backend-express/tests` directory, divided into four categories: unit, integration, system, and quality. Tests run in a Node.js environment and are configured centrally in the `jest.config.js` file.

7.2 Unit Testing

Objectives

Verify the correctness of individual functions or modules in isolation. Test cases focus on business logic, input validation, HTTP error codes, token verification, password handling, and exception branches. Prisma Client, mailer, and external dependencies are replaced with mocks so that results remain unaffected by the execution environment.

Methodology

Each function is tested against valid, invalid, boundary, and exception cases. Key functional groups include registration and login, OTP verification, book management, patron management, document circulation, authentication middleware, and error normalization.

Table 31: *Scope and Results of Unit Testing*

Test Group	Main Content	Suites	Result
Auth and DTO	Registration, OTP, login, input validation	3	Pass
Books	Book CRUD, book utilities, catalog management	3	Pass
Users and Branches	Patron profiles, branch business logic	2	Pass
Circulation and Middleware	Borrow/return, auth middleware, error handling	3	Pass
Total		10	168/168

The example below tests the scenario of registering with an already existing email. The expected result is that the service returns an HTTP 409 error and does not create a new account.

```

1 it('returns 409 when email already exists', async () => {
2   prismaMock.user.findUnique.mockResolvedValueOnce({ id: 'exists' });
3
4   await expect(authService.register({
5     email: 'reader@example.com',
6     password: 'password1',
7     fullName: 'A',
8   })).rejects.toMatchObject({ statusCode: 409 });
9 });

```

Listing 5: Unit test case for registration feature

Unit testing results show that 10/10 test suites passed, corresponding to 168/168 test cases passed. Code coverage for this group reached 94.33% statements, 77.58% branches, 94.44% functions, and 95.07% lines.

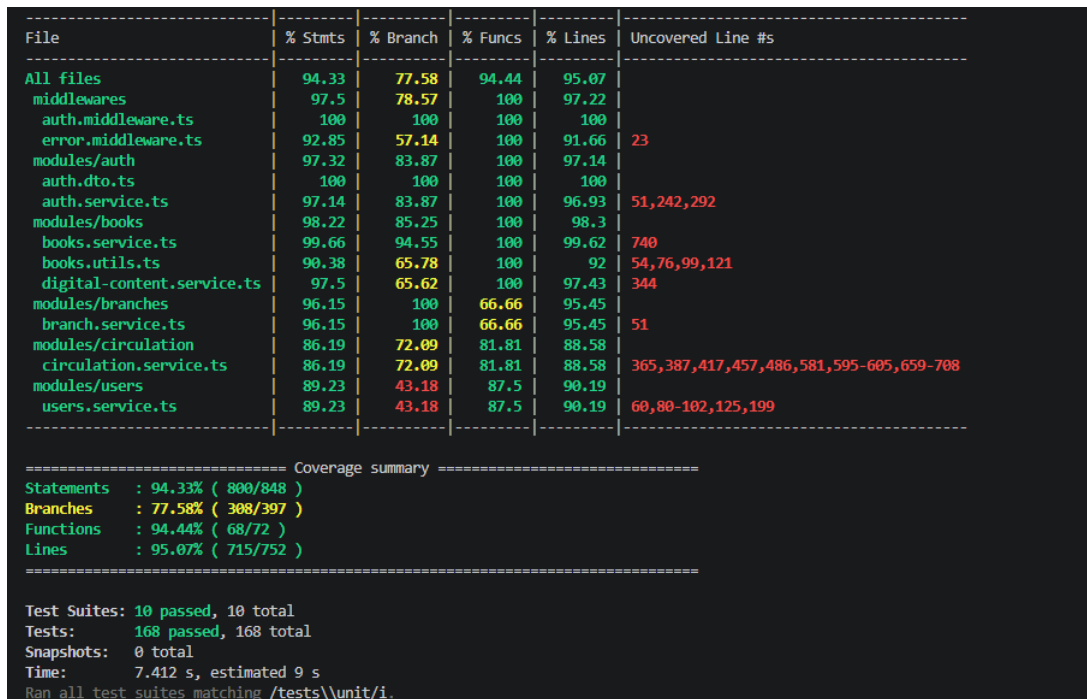


Figure 72: Unit test results

7.3 Integration Testing

Objectives

Verify the coordination between routes, middleware, controllers, services, and the data access layer through actual HTTP requests sent to the Express application. This testing suite confirms response structures, HTTP status codes, authentication mechanisms, and inter-functionality integration within the same module.

Methodology

Supertest is used to simulate GET, POST, PATCH, and DELETE requests. The Prisma Client is mocked at the data boundary, while the Express application is fully instantiated so that routes and middleware execute just as they would in a live environment.

Table 32: Scope and Results of Integration Testing

API/Module	Tested Content	Suites	Result
Auth	Registration, OTP verification, login, payload errors	1	Pass
Books and Branches	Book lookup, catalog and branch management	2	Pass
Users and Circulation	User profiles, borrowing, returning, renewals	2	Pass
AI Service	Interacting with AI service endpoints	1	Pass
Total		6	67/67

The example below tests the login API with valid credentials. In addition to HTTP status 200, the test confirms that the response payload includes an access token, refresh token, and user profile details.

```

1 const response = await request(app)
2   .post('/api/v1/auth/login')
3   .send({ email: 'reader@library.edu.vn', password: 'CorrectPass@123' });
4
5 expect(response.status).toBe(200);
6 expect(response.body.success).toBe(true);
7 expect(response.body.data.accessToken).toBeDefined();
8 expect(response.body.data.refreshToken).toBeDefined();

```

Listing 6: Integration test case for login API

Integration testing achieved 6/6 passing test suites and 67/67 passing test cases. All evaluated endpoints returned status codes and response bodies appropriate for both success and error scenarios.

```

PS C:\Users\user\library-management-system\backend-express> npm run test:integration

> library-backend@1.0.0 test:integration
> jest --testPathPattern=tests/integration --runInBand

PASS tests/integration/auth.integration.test.ts
  • Console

  console.log
    [DEV] OTP for newuser@student.edu.vn: 549780
    at sendOtpEmail (src/config/mailer.ts:68:11)

  console.log
    [MAIL - SKIPPED, blocked domain] To: newuser@student.edu.vn | Subject: [Thư viện] Mã xác thực OTP: 549780
    at sendMail (src/config/mailer.ts:40:13)

  console.log
    [DEV] OTP for reader@library.edu.vn: 675980
    at sendOtpEmail (src/config/mailer.ts:68:11)

PASS tests/integration/users.integration.test.ts
PASS tests/integration/circulation.integration.test.ts
PASS tests/integration/books.integration.test.ts
PASS tests/integration/ai.integration.test.ts
PASS tests/integration/branches.integration.test.ts

Test Suites: 6 passed, 6 total
Tests: 67 passed, 67 total
Snapshots: 0 total
Time: 4.757 s, estimated 6 s
Ran all test suites matching /tests\\integration/i.
  
```

Figure 73: Integration test results

7.4 System Testing

Objectives

Validate end-to-end business workflows from user requests to final system outputs. This group focuses on the patron lifecycle, borrow–renew–return cycles, reservation queues, overdue fine calculation, and authorization security.

Methodology

Business steps are simulated in real-world sequential order. Each step sends a request using the appropriate user role and verifies both the returned response and updated business state. The security testing group includes edge cases for missing tokens, expired tokens, tampered tokens, unauthorized role access, security headers, and non-existent endpoints.

Table 33: System Testing Workflows

Business Workflow	Tested Content	Result
Patron Lifecycle	Registration, verification, profile update, account usage	Pass
Document Circulation	Book addition, search, borrowing, history viewing, renewal, return	Pass
Reservations and Queues	Item reservation, queue ordering, readiness notifications	Pass
Overdue Fines	Overdue identification, fine creation, payment handling	Pass
System Security	JWT, RBAC, Helmet, 404 errors, invalid JSON payloads	Pass
Total	5 test suites, 36 test cases	36/36

For example, the circulation workflow sequentially checks a librarian adding a book, a patron searching for it, the librarian creating a borrow record, and the patron requesting a renewal. The case below verifies that the librarian role is authorized to create a borrow record:

```

1 const response = await request(app)
2   .post('/api/v1/circulation/borrow')
3   .set(roleAuthHeader(Role.LIBRARIAN, librarianId))
4   .send({ userId: readerId, barcode: copyBarcode });
5
6 expect(response.status).toBe(201);
7 expect(response.body.success).toBe(true);
  
```

```
8 | expect(response.body.data.borrowRecordId).toBe(borrowRecordId);
```

Listing 7: System test case for book borrowing workflow

Results show that all 5 system workflows passed, with 36/36 test cases succeeding. Cases involving missing tokens, invalid tokens, unauthorized access, and malformed JSON payloads were correctly rejected with appropriate HTTP status codes.

```
PS C:\Users\user\library-management-system\backend-express> npm run test:system
> library-backend@1.0.0 test:system
> jest --testPathPattern=tests/system --runInBand

PASS tests/system/reader-lifecycle.system.test.ts
  ● Console

    console.log
      [DEV] OTP for nguyenvana@student.edu.vn: 450827
    at sendOtpEmail (src/config/mailer.ts:68:11)

    console.log
      [MAIL - SKIPPED, blocked domain] To: nguyenvana@student.edu.vn | Subject: [Thư viện] Mã xác thực OTP: 450827
    at sendMail (src/config/mailer.ts:40:13)

PASS tests/system/circulation-workflow.system.test.ts
PASS tests/system/reservation-queue.system.test.ts
PASS tests/system/security-rbac.system.test.ts
PASS tests/system/overdue-penalties.system.test.ts

Test Suites: 5 passed, 5 total
Tests: 36 passed, 36 total
Snapshots: 0 total
Time: 4.151 s, estimated 5 s
Ran all test suites matching /tests\\system/i.
```

Figure 74: System test results

7.5 Code Quality Testing

Objectives

Evaluate static quality conditions and architectural conventions in addition to functional testing. Verification topics include TypeScript compilation, source directory integrity, hardcoded secret detection, route-service layering, API response structures, and normalized error codes.

Methodology

Tests parse and analyze TypeScript files within the `src` directory while executing `npx tsc --noEmit`. Criteria are expressed as Jest assertions so that any violation is automatically caught within the testing pipeline.

Table 34: Source Code Quality Test Results

Criterion	Tested Content	Result
TypeScript Compilation	Entire backend compiles without type errors	Pass
Codebase Integrity	The <code>src</code> directory exists; TypeScript files are non-empty	Pass
Security	No hardcoded passwords, JWT secrets, or connection strings	Pass
Layered Architecture	Routes do not invoke Prisma directly; controllers forward errors via <code>next</code>	Pass
Responses and Error Codes	Normalized success property and valid HTTP status codes	Pass
Total	2 test suites, 10 test cases	10/10

```

● PS C:\Users\user\library-management-system\backend-express> npm run test:quality

> library-backend@1.0.0 test:quality
> jest --testPathPattern=tests/quality --runInBand

PASS tests/quality/typescript-compilation.quality.test.ts
PASS tests/quality/code-quality.quality.test.ts

Test Suites: 2 passed, 2 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        4.893 s, estimated 7 s
Ran all test suites matching /tests\\quality/i.

```

Figure 75: Quality test results

7.6 Test Summary

All backend tests were executed using the command `npm test -- --runInBand`. The overall summary is presented in Table 35.

Table 35: Backend Test Summary

Test Type	Test Suites	Test Cases	Passed
Unit	10	168	168
Integration	6	67	67
System	5	36	36
Code Quality	2	10	10
Total	23	281	281

All 23 test suites executed successfully with zero failing test cases and no snapshot tests. These results confirm that key functionality, cross-layer coordination, security rules, and source code quality standards of the backend satisfy all test requirements at the time of evaluation.

```

Test Suites: 23 passed, 23 total
Tests:       281 passed, 281 total
Snapshots:   0 total
Time:        11.132 s, estimated 74 s
Ran all test suites.

```

Figure 76: Summary of test results

8 AI Features Evaluation

8.1 Why Traditional Testing is Insufficient for AI

The testing methodology described in the previous chapter follows a **deterministic** paradigm: for any given input, the system must produce a precisely defined output, and an assertion either passes or fails. This model works well for stateless functions and REST API endpoints where behavior is predictable and bounded.

However, the AI-powered features in BkLib — namely the RAG-based Chatbot and the Semantic Search engine — produce **non-deterministic** outputs in the form of natural language. Consider the following contrast:

Table 36: *Deterministic vs. Non-deterministic Testing*

Aspect	Traditional Testing	AI Feature Evaluation
Output type	Fixed value (status code, JSON field)	Natural language sentence or ranked list
Correctness check	<code>expect(x).toBe(y)</code>	Semantic similarity, faithfulness, relevance
Variability	None — identical input produces identical output	High — phrasing changes per run
Tool	Jest, Supertest	LLM-as-Judge, embedding cosine similarity

For example, when a patron asks “How many books can I borrow at once?”, a valid chatbot response may be “You can borrow up to 5 books per loan” or “The library allows a maximum of 5 items per reader” — both are semantically correct yet lexically different. A hard-coded `toBe()` assertion cannot capture this equivalence, making a dedicated AI evaluation framework necessary.

8.2 AI-as-a-Judge Methodology

Architecture of the Evaluation Pipeline

To systematically assess AI output quality, an automated **AI-as-a-Judge** pipeline was implemented. The architecture involves three roles:

- **The System Under Test (SUT):** The BkLib AI service exposing the `/chat` and `/search/semantic` endpoints.
- **The Test Suite:** A curated dataset of question–answer pairs and search queries with expected intent labels.
- **The Judge LLM:** An independent Gemini model configured with `temperature = 0.0` to ensure deterministic, reproducible scoring.

The pipeline executes as follows for each test case:

1. A query is submitted to the SUT.
2. The SUT returns a response (chatbot answer or ranked book list).
3. The Judge LLM receives the query, the SUT response, and the ground-truth context.
4. The Judge scores the response on three metrics and returns a structured JSON verdict.
5. The verdict is recorded; a case is marked *Passed* if the aggregate score meets the threshold.

Evaluation Metrics

Each chatbot response is scored on a **1–5** scale across three independent dimensions:

Table 37: *AI Evaluation Metrics Definitions*

Metric	Definition	Pass Threshold
Faithfulness	The response is grounded solely in retrieved context; no hallucinated facts are introduced.	≥ 4.0
Answer Relevancy	The response directly addresses the patron’s question without off-topic content.	≥ 4.0
Semantic Correctness	The factual content of the response matches the ground truth (e.g., borrowing limits, fine rates).	≥ 4.0

For Semantic Search, the metric is the **Semantic Match Score** — the average Judge rating of retrieved book relevance relative to the query intent.

Judge LLM Configuration

The Judge LLM is prompted with a structured template that provides (1) the original patron query, (2) the AI-generated answer or retrieval list, and (3) the reference ground truth extracted from library policy documents. The Judge is explicitly instructed *not* to penalize minor stylistic variation and *to* penalize any factual deviation from the reference. Setting `temperature = 0.0` ensures that repeated evaluation of the same case produces a consistent score, making the results reproducible and auditable.

```
1 You are an impartial evaluator for a library AI assistant .
2 Query: "{query}"
3 AI Response: "{response}"
4 Ground Truth: "{ground_truth}"
5
6 Score each metric from 1 (very poor) to 5 (excellent):
7 - faithfulness: Is the response grounded only in the provided context?
8 - answer_relevancy: Does the response directly address the query?
9 - semantic_correctness: Are the facts accurate relative to ground truth?
10
11 Return ONLY valid JSON with integer scores and a brief reason .
```

Listing 8: Judge LLM Prompt Template (simplified)

8.3 Chatbot RAG Evaluation

Test Suite Design

The evaluation dataset consists of **16 test cases** organized into two groups based on the expected processing path:

- **Group 1 — FAQ / RAG queries (10 cases):** Questions about library policy, opening hours, and procedures. These are expected to be classified as FAQ intent and answered using RAG context retrieved from the static knowledge base.
- **Group 2 — Personal / Function Calling queries (6 cases):** Questions about the reader's own account data (active loans, fines, reservations). These are expected to be classified as PERSONAL intent and trigger the appropriate backend tool call.

For each test case, the system records: (1) actual intent label assigned by the classifier, (2) tool called (for Group 2), and (3) Judge LLM scores on three RAG Triad metrics (for Group 1).

Group 1 — FAQ / RAG Test Cases

Table 38: Chatbot Evaluation — Group 1: Policy and FAQ Queries

ID	Query (Vietnamese)	Category	Intent	F	R	C
1	“Quy dinh muon sach cua thu vien nhu the nao?” (Library borrowing policy)	Borrow rules	FAQ	5	5	5
2	“Sinh vien dai hoc duoc muon toi da bao nhieu cuon sach?” (Max books for students)	Borrow limit	FAQ	5	5	5
3	“Phi phat tre han duoc tinh nhu the nao?” (How is the overdue fine calculated?)	Fine policy	FAQ	5	5	5
4	“Thu vien co so Di An mo cua luc may gio?” (Opening hours – Di An branch)	Opening hours	FAQ	5	5	5
5	“Lam the nao de huy dat giu cho sach truc tuyen?” (How to cancel a hold online)	Reservation	FAQ	5	5	5
6	“Mat sach thi phai den bu the nao?” (Compensation for lost books)	Lost/Damage	FAQ	5	5	5
7	“Thu vien co so Ly Thuong Kiet co mo cua thu bay khong?” (LTK branch Saturday hours)	Opening hours	FAQ	5	5	5
8	“Toi co the nho nguoi khac muon sach ho khong?” (Can someone borrow on my behalf?)	Card policy	FAQ	5	5	5
9	“Toi duoc gia han muon sach toi da may lan?” (Max renewals allowed)	Renewal	FAQ	5	5	5
10	“Lam sao de biet cuon sach toi can dang o co so nao?” (Which branch holds the book?)	Branch lookup	FAQ	5	5	5
Average (scale 1–5)				5.0	5.0	5.0

F = Faithfulness, R = Answer Relevancy, C = Semantic Correctness (scale 1–5)

All 10 FAQ cases were correctly classified as FAQ intent (intent accuracy: **100%**), and the RAG retrieval pipeline successfully returned relevant context in all cases (RAG retrieval rate: **100%**). The Judge LLM awarded a perfect score of **5/5** across all three metrics for every case, reflecting that the Gemini model strictly followed the retrieved context without hallucinating and produced concise, on-topic answers.

Table 39: Sample Ground Truth vs. Chatbot Answer — FAQ Case #2

Field	Content
Query	“Sinh vien dai hoc duoc muon toi da bao nhieu cuon sach?”
Ground Truth	Sinh vien dai hoc duoc muon toi da 3 cuon sach cung mot luc.
Chatbot Response	Ban co the muon toi da 3 cuon sach cung luc. Thoi han muon la 14 ngay va moi cuon duoc gia han toi da 2 lan.
Judge Verdict	{"faithfulness": 5, "relevancy": 5, "correctness": 5, "reason": "Correct 3-book limit stated, no info added beyond context"}

Group 2 — Personal / Function Calling Test Cases

For queries involving the user’s personal account data, the system must (1) detect PERSONAL intent and (2) select the correct backend tool to call from three available tools: `get_active_borrows`, `get_user_fines`, and `get_user_reservations`.

Table 40: Chatbot Evaluation — Group 2: Personal Account Queries (Function Calling)

ID	Query (Vietnamese)	Expected Tool	Actual Tool Called	Result
11	“Toi dang muon nhung cuon sach nao vay?” (What books am I currently borrowing?)	get_active_borrow	get_active_borrows	Pass
12	“Tai khoan cua toi co bi phat tien tre han khong?” (Do I have any overdue fines?)	get_user_fines	get_user_fines	Pass
13	“Kiem tra giup minh danh sach sach dang dat giu cho?” (Show my hold reservations)	get_user_reservat	gensuser reservations	Pass
14	“Cho minh xem cac cuon sach minh muon sap het han?” (Which books are due soon?)	get_active_borrow	get_active_borrows	Pass
15	“Xem giam toi tong tien phat hien tai?” (What is my total current fine amount?)	get_user_fines	get_user_fines	Pass
16	“Lich su dat giu cho cua toi co cuon nao khong?” (Any books in my reservation history?)	get_user_reservat	gensuser reservations	Pass
Tool Selection Accuracy				6/6 (100%)

All 6 personal-query cases were correctly classified as PERSONAL intent, and the LLM selected the correct backend tool in all instances. This demonstrates the robustness of the two-stage pipeline: a rule-based intent classifier (`is_personal_query`) performs reliable pre-filtering before the LLM decides on tool selection.

Table 41: Sample Function Calling Interaction — Case #12

Field	Content
Query	“Tai khoan cua toi co bi phat tien tre han khong?” (Do I have overdue fines?)
Detected Intent	PERSONAL
Tool Called	<code>get_user_fines</code>
Mock Tool Result	Nguoi dung dang no tong cong 15.000d tien phat tu 1 khoan phat tre han cuon “Clean Architecture”.
Chatbot Response	Tai khoan cua ban hien co 1 khoan phat tre han voi tong so tien la 15.000d tu cuon “Clean Architecture”. Ban co the thanh toan tai quay thu vien.

Aggregate Results

Table 42: Chatbot Evaluation — Overall Metrics Summary

Metric	Score Achieved	Target	Status
Intent Classification Accuracy	16/16 (100%)	≥90%	Pass
Tool Selection Accuracy (Personal)	6/6 (100%)	≥90%	Pass
RAG Retrieval Rate (FAQ)	10/10 (100%)	≥80%	Pass
Faithfulness (avg, scale 1–5)	5.0 / 5.0	≥4.0	Pass
Answer Relevancy (avg)	5.0 / 5.0	≥4.0	Pass
Semantic Correctness (avg)	5.0 / 5.0	≥4.0	Pass

8.4 Semantic Search Evaluation

Test Suite and Methodology

Semantic Search was evaluated on **10 representative queries** spanning: exact English queries, paraphrased descriptions, Vietnamese-language queries (cross-language scenario), and off-topic adversarial queries. For each query, the Judge LLM rated whether the returned books were semantically relevant to the query intent.

Test Cases and Results

Table 43: Semantic Search Evaluation Test Cases

ID	Query	Expected Behavior	Result
SE-01	“introduction to machine learning”	Relevant AI/ML books	Pass
SE-02	“data structures and algorithms”	Relevant CS text-books	Pass
SE-03	“books about software engineering best practices”	Software engineering titles	Pass
SE-04	“basic artificial intelligence textbooks”	Intro AI books	Fail
SE-05	“sach lap trinh Python” (Vietnamese: Python programming books)	Python programming books	Fail
SE-06	“sach hoc co so du lieu” (Vietnamese: database textbooks)	Database text-books	Fail
SE-07	“cong nghe phan mem huong doi tuong” (Vietnamese: OOP/software engineering)	OOP/SE books	Fail
SE-08	“beef noodle soup recipe” (off-topic)	No relevant results	Pass
SE-09	“how to cook Vietnamese food” (off-topic)	No relevant results	Pass
SE-10	“advanced database systems distributed computing”	DB/distributed systems	Pass
Pass Rate			6/10 (60%)

Root Cause Analysis: Data Sparsity and Embedding Language Mismatch

Two independent root causes were identified:

8.4.0.1 Cause 1 — Data Sparsity. The demo database contains only a small number of books. When the cosine similarity space is sparse, the top-K retrieval algorithm is forced to return the *nearest neighbors available*, even when they are semantically distant from the query. The system does not have the option to return “no match” — it always fills the top-K slots — so off-topic books appear as results.

Table 44: Semantic Search Failure Example — Data Sparsity

Query	Top Retrieved Books	Judge Score
“basic Python programming books”	1. Basic C++ Programming (0.65) 2. Introduction to Artificial Intelligence (0.58) 3. How to Win Friends and Influence People (0.42)	2.67 / 5.0

8.4.0.2 Cause 2 — Embedding Model Language Mismatch. The deployed embedding model is sentence-transformers/all-MiniLM-L6-v2, which was trained **primarily on English corpora**. When a patron enters a Vietnamese query, the model encodes it into a lower-quality vector that does not align well with the vector space built from the book metadata.

A natural proposed improvement was to **translate the Vietnamese query to English** before encoding. This approach was investigated but ultimately rejected for the following reason: the book embedding vectors stored in the in-memory cache were themselves generated from **Vietnamese-language text** (see `search.py`, lines 79–84 below). Translating only the query introduces a *cross-language vector mismatch*: the English query vector and the Vietnamese book vectors occupy different regions of the embedding space, producing worse similarity scores, not better ones.

```

1 text_to_embed = (
2     f"Ten sach: {title}."
3     f"Tac gia: {author_str}."
4     f"Danh muc: {category_name}."
5     f"Tom tat: {summary}"
6 )

```

Listing 9: Book text fed to the embedding model — Vietnamese language

The correct architectural fix requires **consistency**: both the indexed book text and the query must be encoded using the same language or the same multilingual model. Two viable approaches are compared below:

Table 45: *Architectural Fix Options for Language Mismatch*

Approach	Description	Trade-offs
Translate book metadata to English at index time	Translate all title/summary fields before embedding; store English vectors	One-time cost; requires re-indexing all books
Replace model with multilingual embedding	Switch to <code>paraphrase-multilingual-MiniLM-L12-v2</code> which maps Vietnamese and English into the same vector space	Model download ~120 MB; no translation needed

The multilingual model approach is preferred for future work because it eliminates the translation dependency and maintains consistency without requiring a re-index pipeline.

8.5 Limitations and Future Work

The current AI evaluation framework is functional but subject to the following limitations:

- **Data sparsity:** The demo corpus is too small to produce meaningful semantic coverage. Expanding the catalog to thousands of actual library records would significantly improve Semantic Search performance.
- **Language coverage:** The monolingual English embedding model underserves Vietnamese-language patrons. Adopting a multilingual model is the highest-priority improvement for the search subsystem.
- **Judge model dependency:** The AI-as-a-Judge pipeline relies on the Gemini API, which is subject to rate limiting and quota constraints. A local open-source judge model (e.g., Llama-3-based) would make the evaluation infrastructure fully self-contained.
- **Chatbot context window:** The RAG pipeline retrieves only the top-3 context chunks. For complex multi-part questions, increasing `top_k` to 5 or 7 may improve completeness without significantly increasing latency.

9 Summary and Conclusion

9.1 Project Accomplishments and Implementation Summary

This capstone graduation project has successfully completed the comprehensive design, development, security optimization, and cloud server deployment of the Online Library Management System. By modernizing traditional circulation workflows and embedding artificial intelligence capabilities, the platform provides an end-to-end, intelligent solution serving readers, librarians, and system administrators. The primary accomplishments of the project include:

- **Comprehensive System Analysis and Design:** Formulated complete technical architecture documentation, including a Three-Tier Architecture model, Enhanced Entity-Relationship Diagrams (EERD), class diagrams, activity diagrams, and sequence diagrams covering all operational workflows.
- **Full-Stack Platform Implementation:** Built a robust machine server utilizing Node.js, Express.js, TypeScript, and Prisma ORM connected to PostgreSQL, alongside a high-performance single-page web application (SPA) developed using React.js and Vite.
- **AI-Powered Smart Modules:** Integrated Google Generative AI (Gemini API) and pgvector to deliver natural language semantic search, automated cataloging via ISBN lookup, automated document summarization, personalized recommendation algorithms, and a 24/7 AI virtual assistant (Chatbot with RAG context).
- **Core Circulation Management:** Implemented complete checkouts, returns, hold queue management, online loan renewals, inter-branch physical transfers, automated overdue fine calculations, and multi-channel notifications (In-app and Email via Nodemailer).
- **Security Hardening, Optimization, and Deployment:** Executed rigorous security audits against Insecure Direct Object References (IDOR) and N+1 query bottlenecks; configured Cloudflare CDN, iptables rate limiting for DoS/DDoS mitigation, request timeout constraints, and production server deployment.

9.2 Summary of Completed Project Lifecycle

The execution lifecycle of the graduation project—covering architectural design, full-stack implementation, AI microservice integration, security hardening, and final server deployment—is summarized in Table 46.

Phase	Core Deliverables	Summary of Accomplishments
Phase 1	System Analysis & Foundational Infrastructure	Conducted domain analysis, designed database EERD, established Node.js/Express.js backend, built React.js frontend, and implemented basic cataloging and user authentication modules.
Phase 2	Advanced Circulation	Developed physical copy inventory tracking, hold queue reservations, online loan renewals, inter-branch transfer logistics, and fine calculation engines.
Phase 3	AI Microservices & Intelligent Features	Configured pgvector for vector embeddings, built natural language semantic search, integrated Gemini API for document summarization, deployed recommendation algorithms, and created RAG-assisted Chatbot.
Phase 4	Security, Performance & Production Deployment	Conducted security audits (IDOR prevention, transaction atomicity, input sanitization), query optimization, iptables rate limiting, Cloudflare CDN integration, and production server deployment.

Table 46: Summary of Completed Project Deliverables

10 References

- [1] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill, 2014.
- [2] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2004.

- [3] E. Brown, *Web Development with Node and Express: Leveraging the JavaScript Stack*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.
- [4] B. Ramirez, *Building Data Science Applications with FastAPI: Develop, test, and deploy powerful web applications with Python*, Birmingham, UK: Packt Publishing, 2021.
- [5] Prisma Team, "Prisma Documentation: Working with PostgreSQL," *Prisma.io*, 2024. [Online]. Available: <https://www.prisma.io/docs/orm/overview/databases/postgresql>. [Accessed: October 2026].
- [6] A. Avdeev, "pgvector: Open-source vector similarity search for PostgreSQL," *GitHub Repository*, 2023. [Online]. Available: <https://github.com/pgvector/pgvector>. [Accessed: October 2026].
- [7] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [8] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," *Google API Technical Report*, 2023. [Online]. Available: <https://ai.google.dev/gemini-api/docs>. [Accessed: October 2026].

11 Appendix